УДК 004.2 (075.8)
ББК 32.971.32-02
А 87

Архитектура вычислительных машин и систем [Электронный ресурс]: Учебно-методическое пособие / Мусихин А.Г., Морозов В.А., Гололобов А.А. [и др.]. — М.: МИРЭА – Российский технологический университет, 2024. — 1 электрон. опт. диск (CD-ROM)

Учебно-методическое пособие содержит необходимые для решения практических заданий теоретические сведения по архитектуре вычислительных машин и систем, примеры решения практических заданий и варианты контрольных заданий к практическим занятиям.

Предназначено для студентов 2-го курса квалификации бакалавр, обучающихся по направлениям 09.03.01 «Информатика и вычислительная техника», 09.03.03 «Прикладная информатика», 09.03.04 «Программная инженерия», 01.03.04 «Прикладная математика» и поддерживает дисциплину «Архитектура вычислительных машин и систем», а также может быть использовано студентами других направлений подготовки для получения знаний и навыков в области информационных технологий.

Учебно-методическое пособие издается в авторской редакции.

Авторский коллектив: Мусихин Александр Григорьевич, Морозов Василий Алексеевич, Гололобов Александр Алексеевич, Пономарев Артем Николаевич.

Рецензенты:

Болбаков Роман Геннадьевич, к.т.н., доцент, зав. каф. ИППО РТУ МИРЭА

Андреева Ольга Николаевна д.т.н., доцент, начальник отдела научной работы АО "Концерн "Моринсис-Агат"

Системные требования:
Наличие операционной системы Windows, поддерживаемой производителем.
Наличие свободного места в оперативной памяти не менее 128 Мб.
Наличие свободного места в памяти постоянного хранения (на жестком диске) не менее 30 Мб.
Наличие интерфейса ввода информации.
Дополнительные программные средства: программа для чтения pdf-файлов (Adobe Reader).
Подписано к использованию по решению Редакционно-издательского совета
МИРЭА — Российский технологический университет.
Обьем: 4.71 мб
Тираж: 10

ISBN 978-5-7339-2252-2

© МИРЭА – Российский
технологический университет, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
ПРАКТИЧЕСКАЯ РАБОТА № 1 6
1.1. Стек и стековая память 6
1.2. Польская обратная запись (ПОЛИЗ) 9
1.3. Перевод в ПОЛИЗ через ненаправленный граф 10
1.4. Перевод в ПОЛИЗ по алгоритму Дейкстры 12
1.5. Задания для самостоятельного выполнения 15
ПРАКТИЧЕСКАЯ РАБОТА № 2 19
2.1. D-триггер 19
2.2. T-триггер 22
2.3. JK-триггер 23
2.4. Счетчики 25
2.5. Пример построения счетчика на D-триггерах 26
2.6. Пример построения счетчика на T-триггерах 30
2.7. Пример построения счетчика на JK-триггерах 33
2.8. Задания для самостоятельного решения 39
ПРАКТИЧЕСКАЯ РАБОТА № 3 41
3.1. Учебное операционное устройство и его компоненты 41
3.2. Запись управляющего слова ЦП 47
3.3. Задания для самостоятельного выполнения 52
ПРАКТИЧЕСКАЯ РАБОТА № 4 54
4.1. Устройство управления ЦП учебного ОУ 54
4.2. Управляющие слова для выполнения команды ADD 57
4.3. Изменения в регистрах при выполнении команды ADD 60
4.4. Задания для самостоятельного выполнения 63
ПРАКТИЧЕСКАЯ РАБОТА №5 71
5.1. Описание 8-разрядного процессора типа КР580ВМ80 71
5.2. Описание эмулятора CPU580 72
5.3. Пример решения задачи 76
5.4. Задания для самостоятельного выполнения 76
ПРАКТИЧЕСКАЯ РАБОТА № 6 77
6.1. Понятие конвейера 77
6.2. RISC-конвейер 79
6.3. Описание эмулятора RISC-конвейера 80
6.4. Примеры конфликтов в эмуляторе RISC-конвейера 83
6.5. Варианты для самостоятельной работы 85

ПРАКТИЧЕСКАЯ РАБОТА № 7 86
7.1. Параллельные вычисления 86
7.2. Метрики эффективности параллельных вычислений 87
7.3. Закономерности параллельных вычислений 89
7.3.1. Закон Амдала 90
7.3.2. Закон Густафсона 91
7.3.3. Закон Сана-Ная 92
7.4. Пример решения задачи 93
7.5. Задания для самостоятельного выполнения 95
ПРАКТИЧЕСКАЯ РАБОТА № 8 97
8.1. Общие сведения 97
8.2. Типы адресов стека TCP/IP 97
8.3. Классы IP-адресов 99
8.4. Использование масок 100
8.5. Протокол IPv6 102
8.6. Протокол ARP 102
8.7. Пример задания 103
8.8. Задания для самостоятельного решения 105
ПРАКТИЧЕСКАЯ РАБОТА №9 107
9.1. Виртуальная и физическая память 107
9.2. Страничное распределение памяти 108
9.3. Сегментное распределение памяти 109
9.4. Сегментно-страничное распределение памяти 110
9.5. Перевод виртуального адреса в физический 111
9.6. Задания для самостоятельного решения 113
ПРАКТИЧЕСКАЯ РАБОТА №10 114
10.1. Общая организация прерываний 114
10.2. Процесс обработки прерываний, со стороны памяти 116
10.3. Задания для самостоятельного выполнения 118
ПРАКТИЧЕСКАЯ РАБОТА №11 123
11.1. Оперативное запоминающее устройство 123
11.2. Пример решения задачи 127
11.3. Задачи для самостоятельного выполнения 130
ЗАКЛЮЧЕНИЕ 133
СПИСОК ИСТОЧНИКОВ И ЛИТЕРАТУРЫ 134

ВВЕДЕНИЕ

Настоящее учебное издание представляет собой учебно-методическое пособие по дисциплине «Архитектура вычислительных машин и систем» для различных профилей (специальностей) следующих направлений подготовки 09.03.01 «Информатика и вычислительная техника», 09.03.03 «Прикладная информатика», 09.03.04 «Программная инженерия», 01.03.04 «Прикладная математика».

Издание предназначено для помощи студентам второго курса в обучении по данным направлениям подготовки при изучении теоретических основ и выполнении практических работ, предусмотренных рабочими программами дисциплины «Архитектура вычислительных машин и систем». Пособие содержит необходимую часть теоретических сведений, примеры выполнения заданий и задания по практическим работам. Учебно-методическое пособие поможет обучающимся освоить знания и получить умения и практические навыки в изучении теоретико-практических основ построения вычислительных систем, что необходимо при выполнении определенных видов работ, связанных с будущей профессиональной деятельностью.

ПРАКТИЧЕСКАЯ РАБОТА № 1

1.1. Стек и стековая память

Стек — метод организации памяти в компьютере, представляющий собой список элементов, работающий по принципу LIFO (англ. «Last In-First Out» — «Последний Вошел — Первый Вышел»). Стековая память может быть реализована аппаратным или программным способом. При аппаратной реализации стека содержимое стека перемещается целиком при выполнении каждой операции. Поэтому проще в ВМ является программная реализация стека.

На рисунке ниже приведена обобщённая структура стековой памяти (рис. 1). Она содержит определённое количество ячеек, причём доступ к такой памяти осуществляется только через верхнюю я к.т.н., доцент, зав. каф. ИППО РТУ МИРЭА чейку стека — вершину стека. Указатель стека показывает на вершину стека, то есть на верхнюю заполненную ячейку стека. Когда стек пуст, указатель стека показывает на одну ячейку ниже дна стека [1].

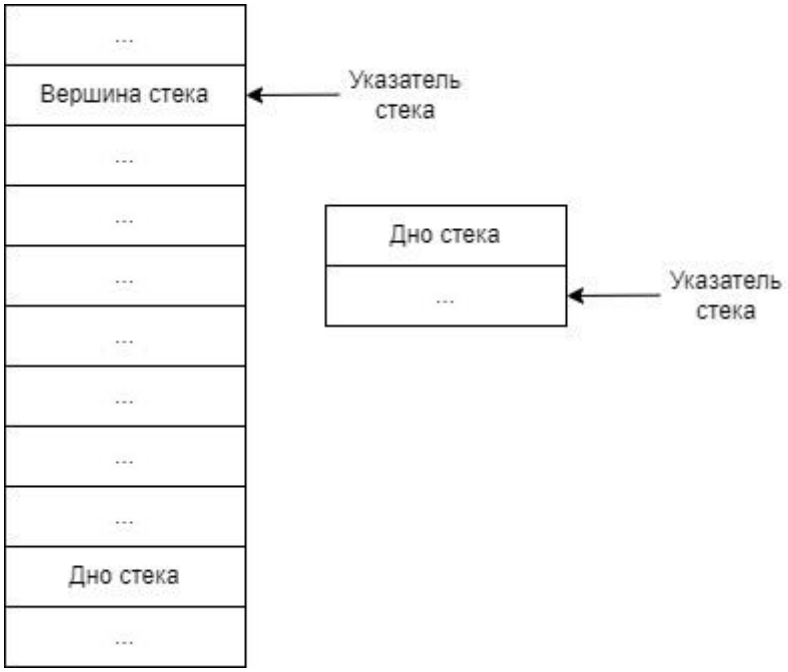


Рисунок 1. Обобщенная структура стека

Реализовать стековую память можно двумя основными способами: программно и аппаратно. У обоих подходов принцип работы одинаков — LIFO, однако в самой реализации есть различия. Например, при аппаратной организации запись нового значения в стек приводит к смещению всего содержимого стека на одну ячейку вниз и к записи в верхнюю ячейку стека (рис. 2) [2]. При считывании слово читается из вершины стека, а всё содержимое стека сдвигается на одну ячейку вверх (рис. 3).

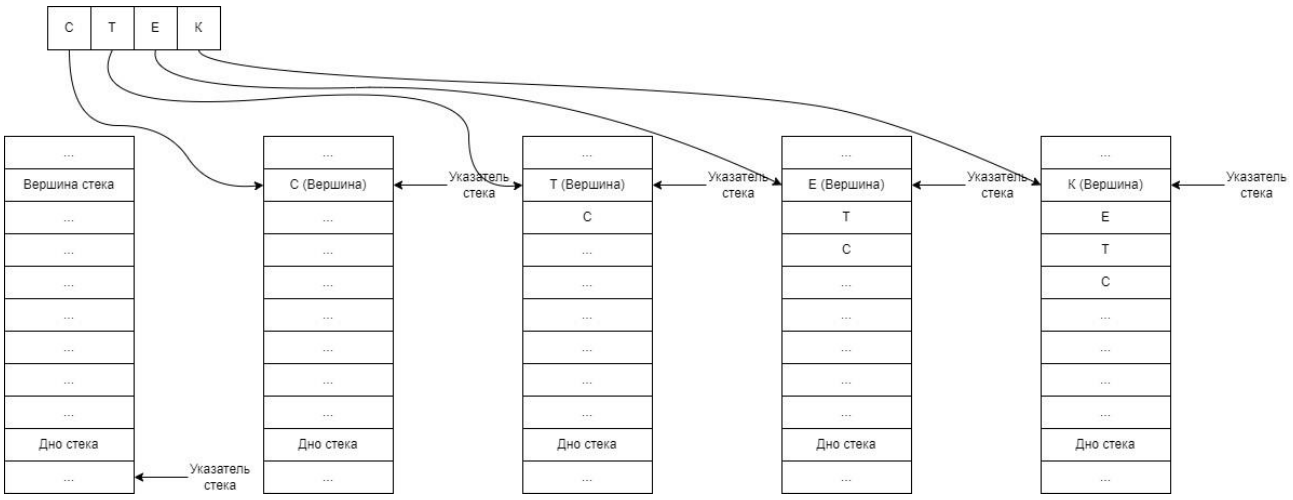


Рисунок 2. Аппаратная реализация стека (Запись)

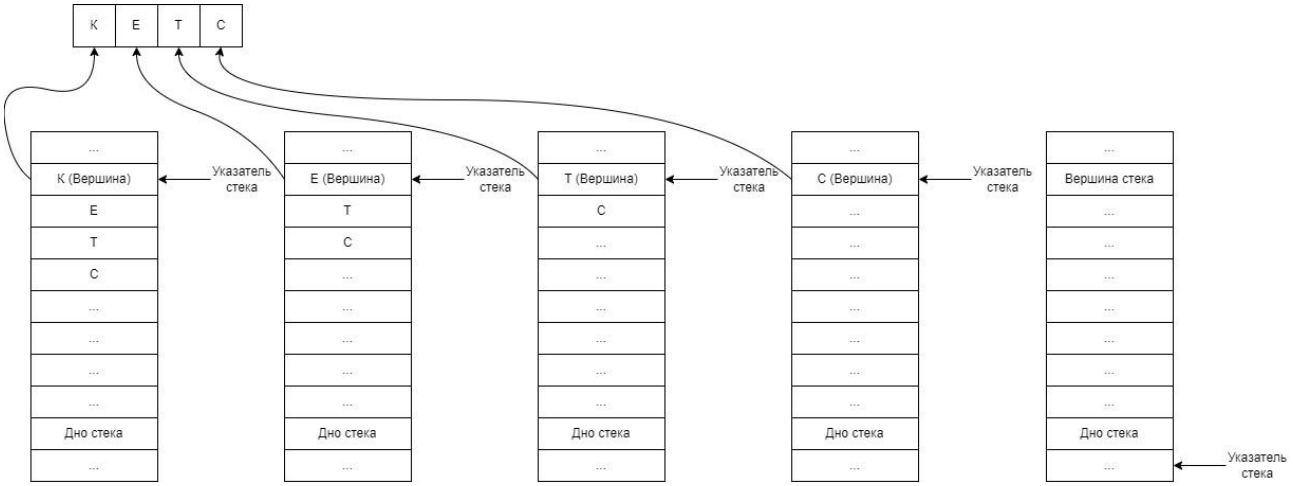


Рисунок 3. Аппаратная реализация стека (Извлечение)

В программной модели вершина стека плавающая: адрес вершины стека содержится в регистре-указателе стека. Такой стек растет не сверху вниз (ко дну стека), а снизу-вверх (от дна стека). Из рассмотрения рис. 4 видно, что при записи в стек сначала происходит его настройка на новую вершину (указатель стека уменьшает своё значение), а затем производится загрузка значения в стек. При считывании (рис. 5) указатель стека увеличивает свое значение после извлечения данного из вершины стека. Поэтому, если стек свободен, значение указателя соответствует ячейке, следующей за дном стека.

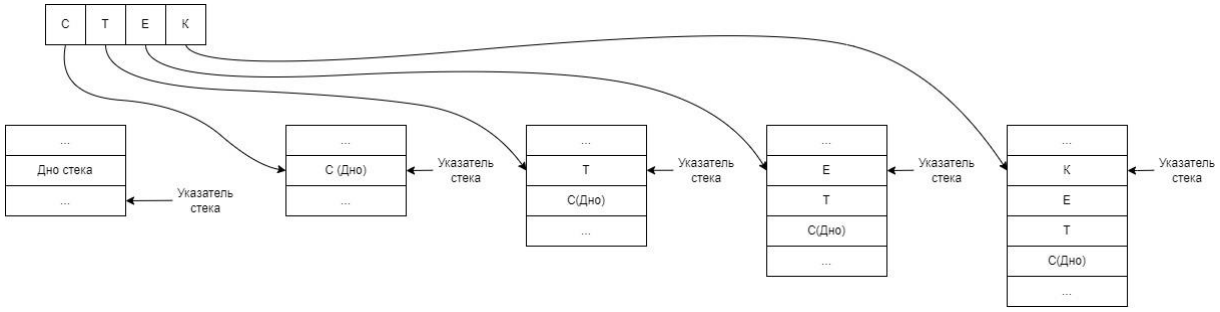


Рисунок 4. Программная реализация стека (Запись)

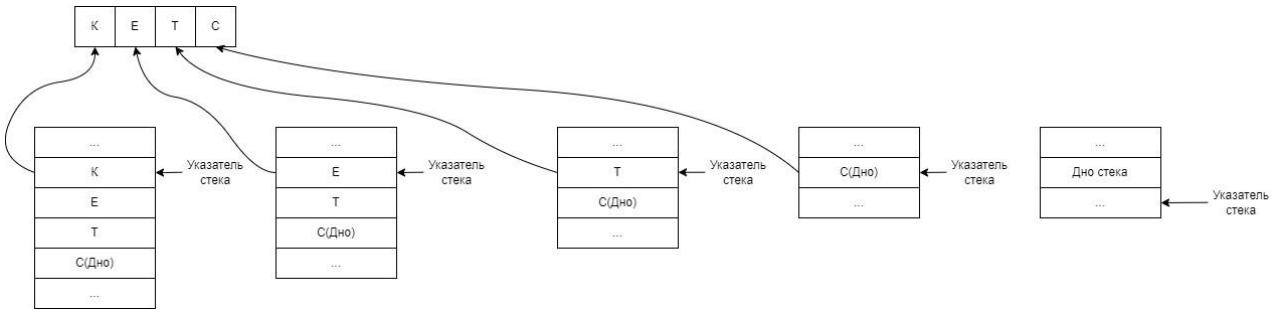


Рисунок 5. Программная реализация стека (Извлечение)

Как можно заметить, из принципа работы любого стека изначальная запись, которая вводилась, была «СТЕК», а на выводе мы получили «КЕТС», произошло это именно из-за организации памяти LIFO. Первым прочитанным символом для записи в стек, была буква «С», следовательно, она попадет на дно стека и будет извлечена последней. Для правильной организации вывода, чтобы входное сообщение было равно тому, которое мы получим в итоге, необходимо заранее развернуть входное сообщение. Пусть входное сообщение будет не «СТЕК», а «КЕТС», тогда при записи в стековую память, будет результат, показанный на рис. 6, а при извлечении на рис. 7.

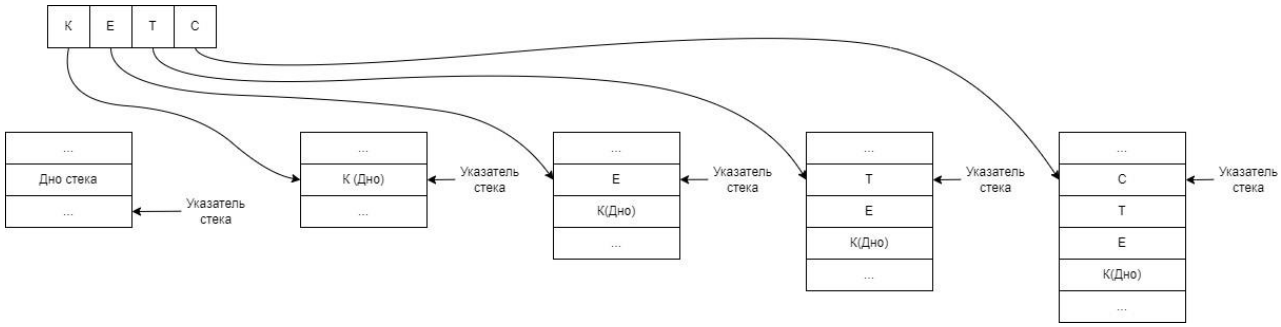


Рисунок 6. Запись слова «КЕТС»

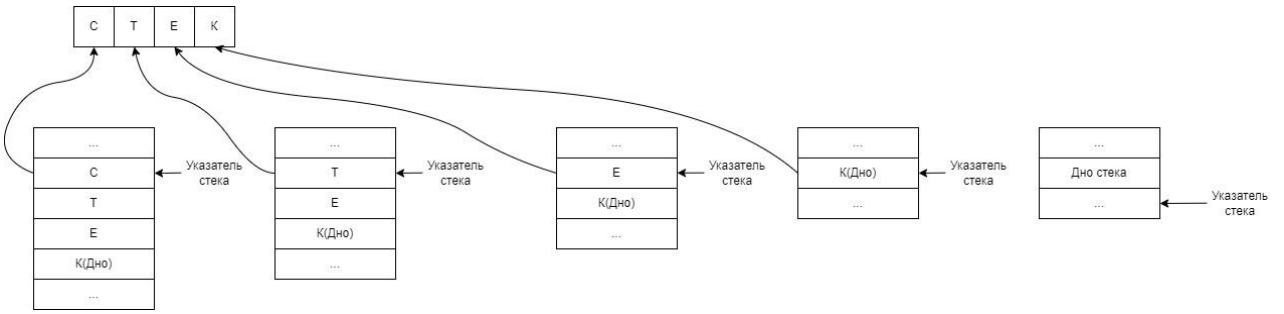


Рисунок 7. Извлечение слова «СТЕК»

На стековой памяти можно выполнять математические и логические операции. Однако, как было показано в примерах выше, просто выполнить математические операции не получится, сначала потребуется перевести изначальное уравнение в постфиксную запись, а уже после, выполнить вычисления на машине.

1.2. Польская обратная запись (ПОЛИЗ)

Постфиксная запись или ПОЛИЗ (Польская обратная запись) — особый вид записи уравнения, при котором все знаки операций вынесены за пределы операндов. Например, классическая (прямая) запись любого уравнения: A + B — это, так называемая инфиксная запись, когда знак операции находится между операндами. Запись вида: AB+ — это постфиксная запись, или запись в виде ПОЛИЗ. Как видно, знак операции (+), вынесен за пределы операндов, и находится после них. Рассмотрим процесс выполнения такой математической операции на стековой ВМ. Процесс изображен на рис. 8. При выполнении считывается первый символ, если это операнд, то он заносится в стек. Первый считанный символ (это «A») — это операнд, значит заносим это значение в стек. Считывается второй символ, это «B» — это также операнд, значит также заносим его в стек. Считывается третий символ — это символ «+» — это математическая операция сложения, она бинарная, то есть требует два числа для выполнения. Операнды хранятся в стеке, указатель стека установлен на вершину, в ней хранится значение «B», изымаем его, и ставим во второй (правый) регистр в АЛУ (Арифметико-Логическое Устройство). Затем считываем второе значение «A», заносим его в первый (левый) регистр АЛУ. Далее АЛУ производит операцию суммирования и возвращает полученное значение обратно в стек, после чего, можно использовать полученное значение суммы для последующих вычислений [3,4].

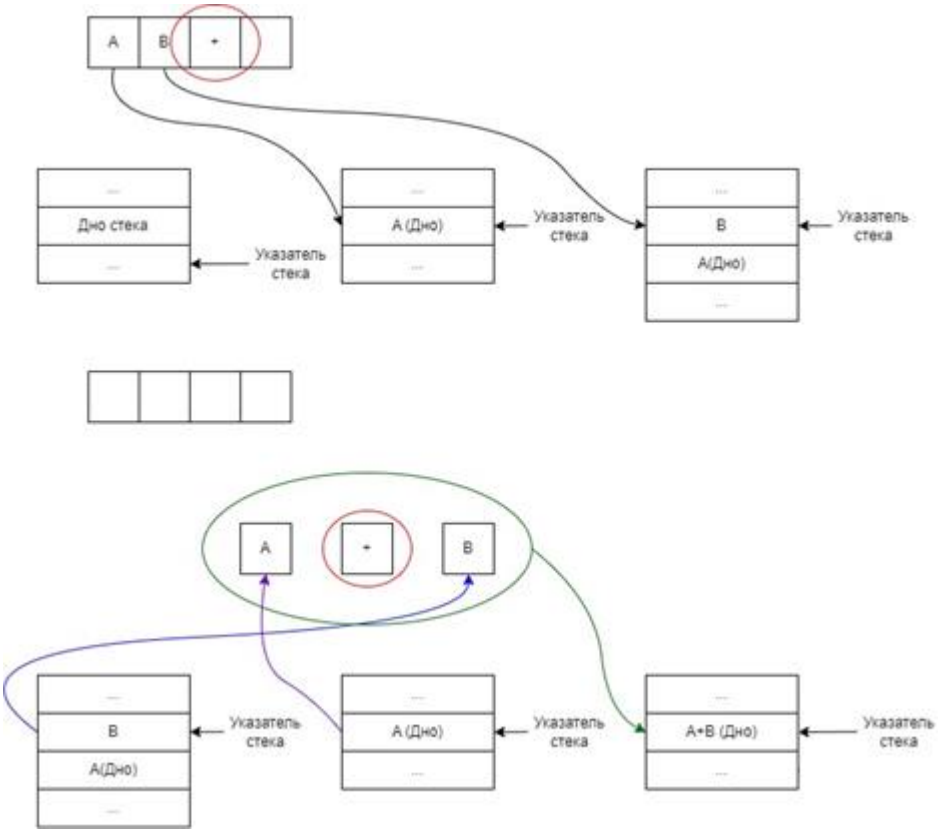


Рисунок 8. Процесс выполнения операции A+B на стеке

1.3. Перевод в ПОЛИЗ через ненаправленный граф

Рассмотрев процесс выполнения уравнений, записанных в польской обратной нотации, необходимо разобраться, как перевести прямую запись уравнения в обратную. Есть несколько способов перевести прямую запись в ПОЛИЗ. Рассмотрим метод перевода в ПОЛИЗ через граф. Пусть задано выражение вида:

$$(A + B) * (C + D) - E. \quad (1)$$

Представим это выражение в виде графа, в котором узлам соответствуют операции, а ветвям — операнды. Построение начнем с корня, в качестве которого выбирается операция, выполняющаяся последней. Левой ветви соответствует левый операнд операции, а правой ветви — правый. Дерево выражения (1) показано на рис. 9.

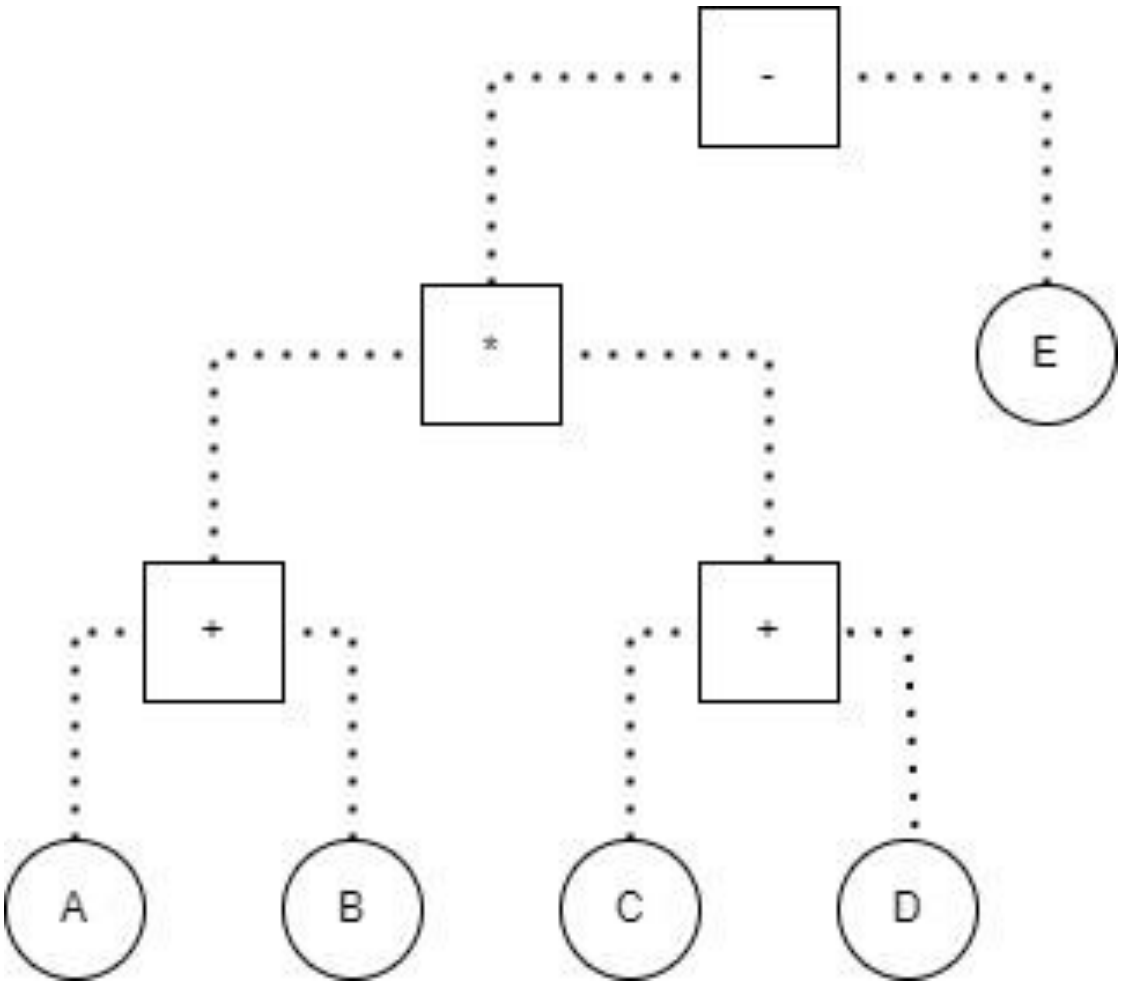


Рисунок 9. Уравнение, представленное в виде дерева

Для того, чтобы перевести граф, обратно в уравнение, будем идти от крайней левой ветки «A», к центру. Ветка операнда «A» соединена с операцией «+», которая в свою очередь соединена с операндом «B» и с операцией умножения, значит в выходную строку записывается «AB+», а граф приобретает вид, как на рис. 10.

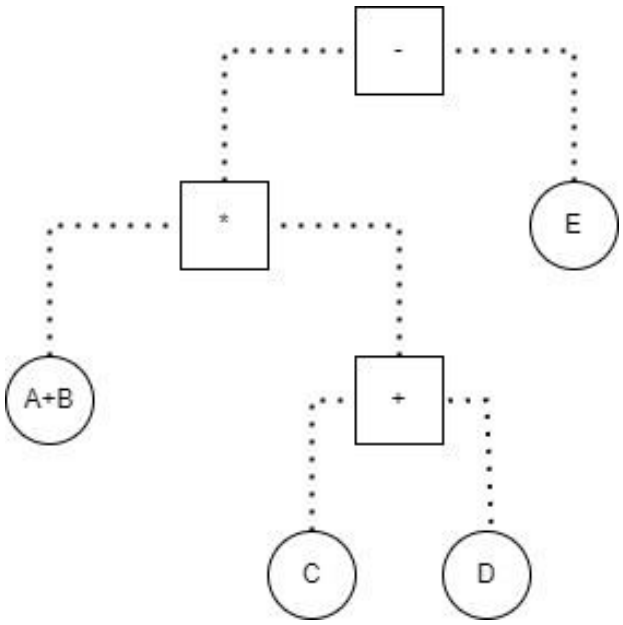


Рисунок 10. Граф после первого прохода

Заметим, что для операции умножения также необходимо два операнда. Значит нам нужно повторить предыдущий шаг с операндами «C» и «D», тем самым получив выходную строку равную «AB+CD+», а граф получит вид, представленный на рис. 11.

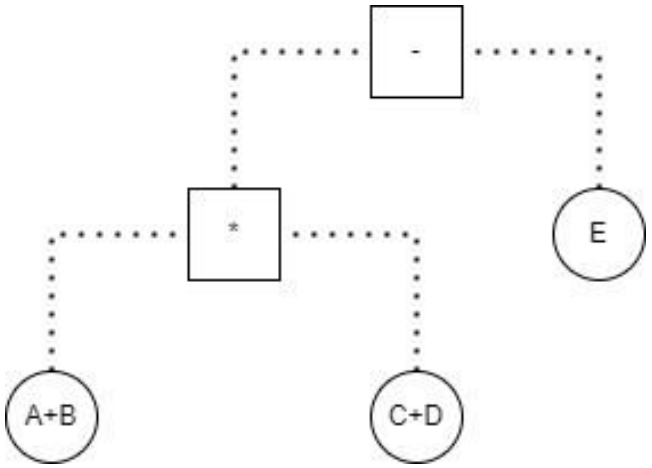


Рисунок 11. Граф после второго прохода

Продолжая описанный алгоритм, мы получим выражение вида (2):

$$AB + CD + * E -. \quad (2)$$

Следующий этап, это проверка полученной записи, с помощью выполнения полученного уравнения на стековой ВМ, и сравнения полученных результатов, с прямыми вычислениями. Пусть A = 5, B = 7, C = –5, D = 7 и E = 8, тогда при подстановке значений в прямую запись получим следующее равенство (3):

$$(5 + 7) * (-5 + 7) - 8 = 16. \quad (3)$$

Произведем подсчет значений уравнения, записанного в польской обратной записи, для этого используем табл. 1:

Таблица 1. Подсчет результата полученного уравнения (2)

Шаг	Формула, подлежащая расчету	Левый знак операции	Операнды	Результат	Итоговая формула
1	5 7 + (–5)7 +*8–	+	5 и 7	12	12 (–5) 7 + * 8 –
2	12 (–5) 7 +*8-	+	(–5) и 7	2	12 2 * 8 –
3	12 2 * 8 –	*	12 и 2	24	24 8 –
4	24 8 –	–	24 и 8	16	16

Как видно из табл. 1, результат, полученный при вычислении уравнения (2), записанного в ПОЛИЗ, совпал с результатом (3), полученным при подсчете прямого уравнения (1), значит, перевод в ПОЛИЗ был верным.

1.4. Перевод в ПОЛИЗ по алгоритму Дейкстры

Второй вариант перевода из прямой записи в ПОЛИЗ — это использование алгоритма Дейкстры. Алгоритм заключается в назначении приоритетов операциям и анализе прямой записи слева направо, по одному символу. Если при анализе выявляется операнд, то он сразу записывается в выходную строку, если встречается математическая операция, то она заносится в стек, в зависимости от ее приоритета (табл. 2).

Таблица 2. Таблица приоритетов математических операций

Операция	Приоритет
(	0
)	1
+ или –	2
* или /	3
^	4

Важно отметить, что символы скобочек, хоть и имеют приоритет, в стек заносятся с некоторыми оговорками, о которых будет сказано позже.

Обобщенный пошаговый алгоритм Дейкстры выглядит следующим образом:

a) если очередной символ из исходной строки есть операнд, то он заносится в выходную строку;

b) если очередной символ из исходной строки есть операция и стек пустой, то операция записывается в стек;

c) операция выталкивает из стека все операции с большим или равным приоритетом в выходную строку;
d) если очередной символ из исходной строки есть открывающая скобка, то он заносится в стек;
e) закрывающая круглая скобка выталкивает все операции из стека до ближайшей открывающей скобки, сами скобки в выходную строку не переписываются, а уничтожают друг друга.

За основу возьмем уравнение (1), которое рассматривалось ранее, и разделим его на отдельные символы (рис. 12).



Рисунок 12. Уравнение, разбитое на символы

Первый символ, «(» — открывающая скобка, заносим его в стек, так как он пустой. Второй символ, «A» — это операнд, отправляем его в выходную строку. Третий символ, «+» — операция сложения, ее приоритет 2, у предыдущего символа приоритет 0, значит добавляем его в стек. Четвертый, «B» — операнд, записываем в выходную строку. На данном этапе, стек должен выглядеть как на рис. 13. Следующий символ — это закрывающая скобка «)», значит, необходимо извлечь и из стека все, до ближайшей открывающей скобки. Сами скобки в выходной строке присутствовать не будут, поэтому они просто уничтожаются.

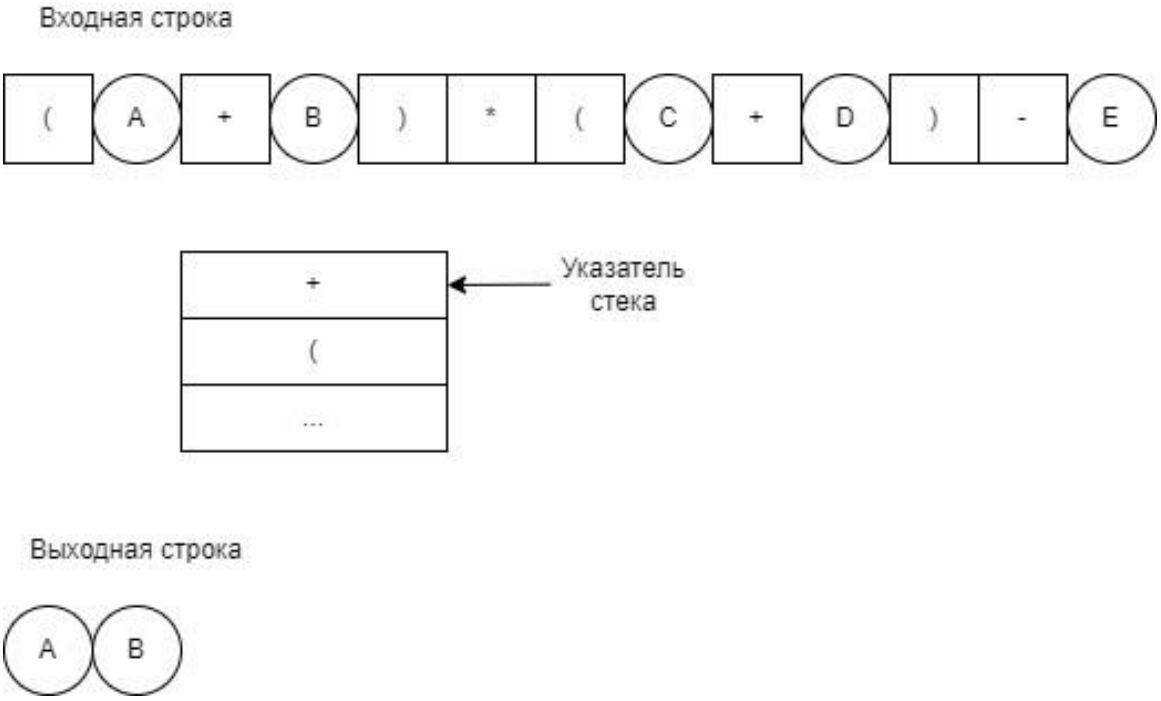


Рисунок 13. Промежуточный итог перевода уравнения в ПОЛИЗ

Анализируем следующий символ, это операция умножения «*», стек — пустой, заносим операцию в стек. Следующий символ — открывающая скобка, ее тоже заносим в стек, несмотря что ее приоритет меньше, чем у умножения. После чего заносим в стек операцию сложения, а в выходную строку записываем «CD». Промежуточный результат изображен на рис. 14.

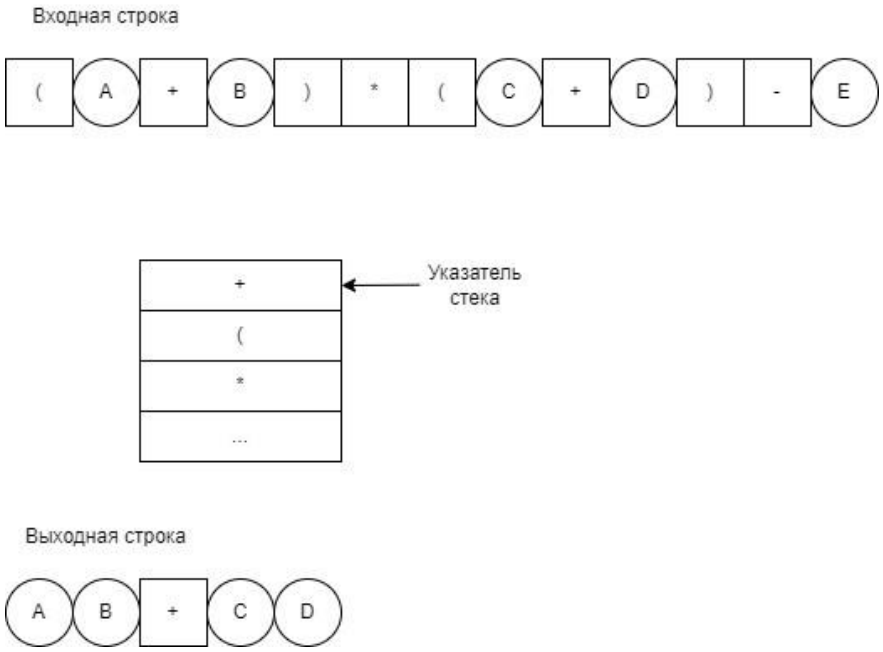


Рисунок 14. Промежуточный итог перевода уравнения в ПОЛИЗ

Далее, символ закрывающей скобочки выталкивает из стека операцию сложения, оставляя операцию умножения. Берем следующий символ — операция вычитания — у нее приоритет ниже, чем у операции умножения, следовательно, операция умножения извлекается из стека, а операция вычитания, наоборот, заносится в стек. Операнд «E», заносится в выходную строку. Больше символов во входной строке нет, значит извлекаем из стека оставшиеся символы. Итоговый результат можно увидеть на рис. 15.

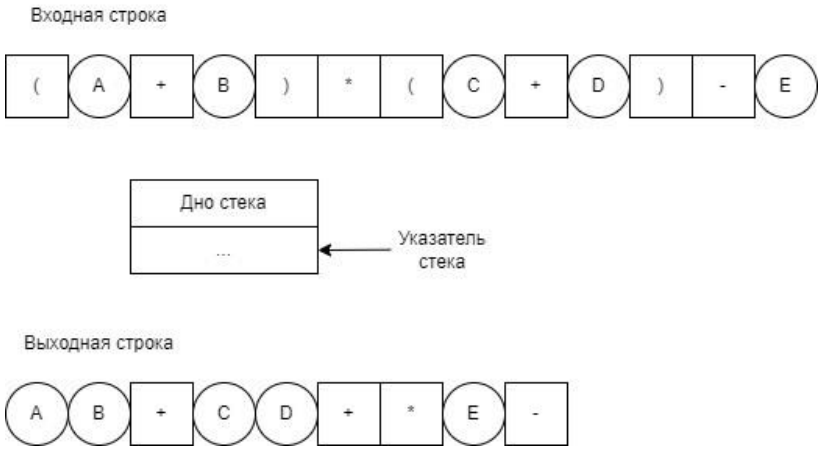


Рисунок 15. Итоговый результат перевода уравнения в ПОЛИЗ по алгоритму Дейкстры

Как можно заметить, выходная строка, полученная по алгоритму Дейкстры, равна той, что мы получили в прошлый раз, пользуясь алгоритмом перевода через ненаправленный граф.

1.5. Задания для самостоятельного выполнения

Варианты задач:

Вариант №1

Преобразовать в ПОЛИЗ арифметическое выражение Q=(A–B) *C/(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=2, B=4, C=1, D=2, E=3.

Вариант №2

Преобразовать в ПОЛИЗ арифметическое выражение Q=(A–B)*C/(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=3, B=5, C=3, D=4, E=1.

Вариант №3

Преобразовать в ПОЛИЗ арифметическое выражение Q=(A+B)*C/(D+E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=1, B=4, C=4, D=1, E=4.

Вариант №4

Преобразовать в ПОЛИЗ арифметическое выражение Q=A*(B+C)/(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=5, B=6, C=2, D=5, E=1.

Вариант №5

Преобразовать в ПОЛИЗ арифметическое выражение Q=A*(B+C)/(D+E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=6, B=5, C=3, D=1, E=3.

Вариант №6

Преобразовать в ПОЛИЗ арифметическое выражение Q=A*(B–C)/(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=7, B=4, C=2, D=5, E=3.

Вариант №7

Преобразовать в ПОЛИЗ арифметическое выражение Q=A*(B+C)*(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=4, B=2, C=3, D=8, E=4.

Вариант №8

Преобразовать в ПОЛИЗ арифметическое выражение Q=A/(B+C)*(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=8, B=2, C=2, D=5, E=1.

Вариант №9

Преобразовать в ПОЛИЗ арифметическое выражение Q=A*(B–C)/(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=9, B=1, C=5, D=5, E=4.

Вариант №10

Преобразовать в ПОЛИЗ арифметическое выражение Q=A/(B+C)/(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=12, B=1, C=3, D=5, E=2.

Вариант №11

Преобразовать в ПОЛИЗ арифметическое выражение Q=A*(B+C)*(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=2, B=3, C=5, D=1, E=4.

Вариант №12

Преобразовать в ПОЛИЗ арифметическое выражение Q=A*(B–C)/(D+E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=5, B=4, C=2, D=2, E=3.

Вариант №13

Преобразовать в ПОЛИЗ арифметическое выражение Q=A/(B–C)*(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=4, B=2, C=3, D=3, E=4.

Вариант №14

Преобразовать в ПОЛИЗ арифметическое выражение Q=(A+B)*C/(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=4, B=4, C=2, D=5, E=4.

Вариант №15

Преобразовать в ПОЛИЗ арифметическое выражение Q=A/(B–C)*(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=5, B=4, C=1, D=5, E=2.

Вариант №16

Преобразовать в ПОЛИЗ арифметическое выражение Q=A/(B+C)*(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=9, B=2, C=1, D=7, E=4.

Вариант №17

Преобразовать в ПОЛИЗ арифметическое выражение Q=A/(B–C)*(D+E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=2, B=7, C=3, D=5, E=4.

Вариант №18

Преобразовать в ПОЛИЗ арифметическое выражение Q=A*(B–C)*(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=2, B=4, C=3, D=7, E=4.

Вариант №19

Преобразовать в ПОЛИЗ арифметическое выражение Q=A/(B*C)+(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=12, B=4, C=3, D=5, E=4.

Вариант №20

Преобразовать в ПОЛИЗ арифметическое выражение Q=A– (B*C)*(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=2, B=7, C=2, D=5, E=4.

Вариант №21

Преобразовать в ПОЛИЗ арифметическое выражение Q=A/(B/C)+(D–E).

Изобразить на рисунке ход вычислительного процесса в стековой ВМ для данного примера, а также в виде таблицы последовательность выполнения арифметических операций при значениях A=4, B=4, C=2, D=5, E=2.

ПРАКТИЧЕСКАЯ РАБОТА № 2

2.1. D-триггер

D-триггер (триггер задержки, D-Tr) [5] — называют еще триггером задержки (рис. 16), поскольку он способен задерживать входной информационный сигнал на один такт и который функционирует по следующей формуле (4):

$$Q_{t+1} = D_t \ . \qquad (4)$$

D-триггеры имеет один информационный вход (D-вход) для установки в «1» или «0» и вход синхронизации C, его таблица истинности при условии, что C=1, представлена в табл. 3, а функциональная схема на рис. 17.

Время задержки определяется частотой импульсов синхронизации, если информационный сигнал изменится, то выходной повторит его изменение, но лишь после того, как тактовый сигнал примет значение, равное единице, пока его значение будет оставаться нулевым, сигнал на выходе меняться не будет.

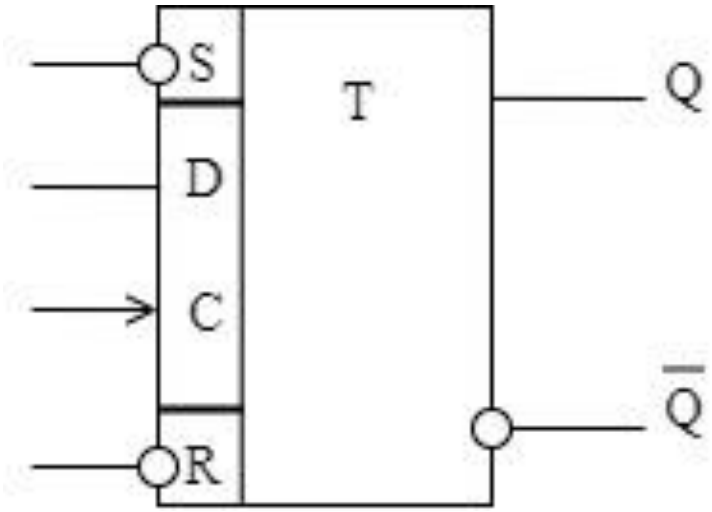


Рисунок 16. Структурное обозначение D-триггера

Таблица 3. Таблица истинности D-триггера при C = 1

D_t	Q_{T+1}	Примечание
0	0	Установка «0»
1	1	Установка «1»

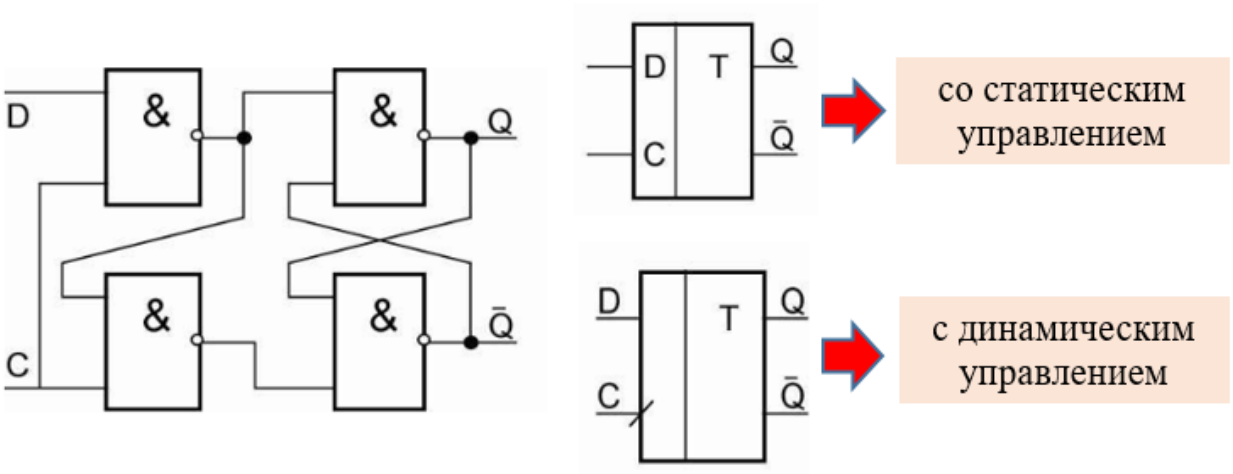


Рисунок 17. Функциональная схема D-триггера

Синхронный D-триггер можно также описать с использованием Verilog HDL (листинг 1).

Листинг 1. Модуль D-триггера, описанный на языке описания аппаратуры Verilog

```
module d_latch(
    input D, C,
    output Q, nQ
);

assign Q = ~( ~(D & C) & nQ);
assign nQ = ~( ~(~(D & C) & C) & Q);

endmodule
```

Кроме основных входов триггеры могут иметь дополнительные, например, **вход V** — разрешающий вход, входы начальной установки **S** и **R**. Структурное обозначение Dv-триггера представлено на рис. 18. Наличие V-входа расширяет функциональные возможности D-триггера, позволяя в нужный момент времени сохранять информацию на выходах в течение нужного числа тактов, соответственно таблица истинности указана в табл. 4 [6].

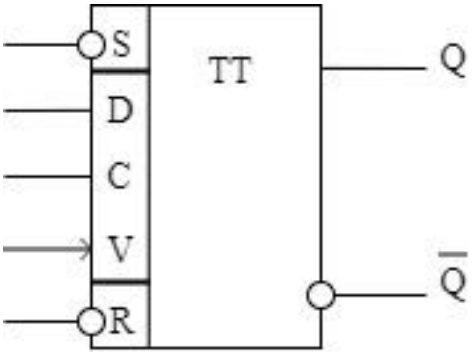


Рисунок 18. Структурное обозначение Dv-триггера

Таблица 4. Таблица истинности Dv-триггера, при C=1

D	V	Q_{T+1}	Примечание
0	0	Q_T	Хранение старого состояния
0	1	0	Установка «0»
1	0	Q_T	Хранение старого состояния
1	1	1	Установка «1»

Синхронный DV-триггер можно также описать с использованием Verilog HDL (листинг 2), временная диаграмма работы модуля представлена на рис. 19.

Листинг 2. Модуль Dv-триггера, описанный на языке описания аппаратуры Verilog

```
module dv_trig(
    input C, D, V, nS, nR,
    output reg Q,
    output nQ
);

reg pQ;
initial begin
    pQ = 0; Q = 0;
end
assign nQ = ~Q;
always@*
    case({nS,nR})
        2'b10:
            begin
                pQ <= 0; Q <= 0;
            end
        2'b01:
            begin
                pQ <= 1; Q <= 1;
            end
        2'b11:
            begin
                if(V)
                    if(~C)
                        pQ <= D;
                    else if (C)
                        Q <= pQ;
            end
    endcase
endmodule
```

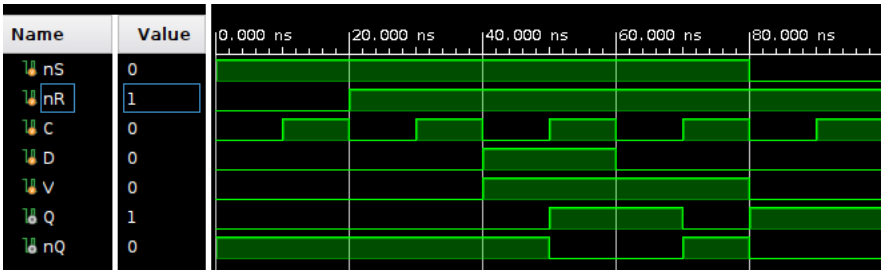


Рисунок 19. Временная диаграмма работы модуля двухступенчатого синхронного DV-триггера, описанного на поведенческом уровне

RS-триггер на основе Dv-триггера построить нельзя, так как функцию RS-триггера берет на себя D-триггер.

2.2. T-триггер

T- триггер (триггер со счетным входом) — это такой триггер, который имеет один информационный вход T (рис. 20). При подаче «1» на вход T триггер меняет свое состояние на противоположное; при подаче «0» на вход T триггер хранит старое состояние, его таблица истинности, при C=1 представлена в табл. 5.

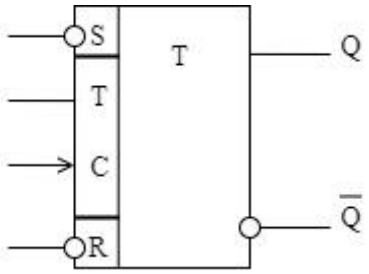


Рисунок 20. Структурное обозначение T-триггера

Таблица 5. Таблица истинности T-триггера при C=1

T	Q_{T+1}	Примечание
0	Q_T	Хранение старого состояния
1	$\overline{Q_T}$	Переброс

После поступления на вход T импульса, состояние триггера меняется на прямо противоположное. Счётным он называется потому, что T триггер как бы подсчитывает количество импульсов, поступивших на его вход. При поступлении второго импульса снова сбрасывается в исходное состояние.

Синхронный T-триггер можно также описать с использованием Verilog HDL (листинг 3), временная диаграмма работы модуля представлена на рис. 21.

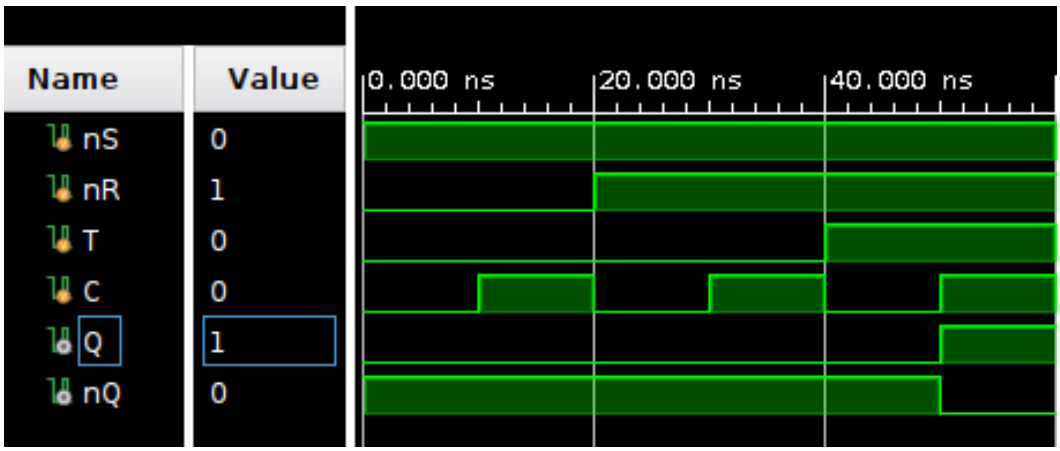


Рисунок 21. Временная диаграмма работы модуля двухступенчатого синхронного T-триггера, описанного на поведенческом уровне

Листинг 3. Модуль T-триггера, описанный на языке описания аппаратуры Verilog

```
module t_trig(
    input nS, T, C, nR,
    output reg Q,
    output nQ
);
initial Q <= 0;
assign nQ = ~Q;

reg pQ;
initial
begin
    pQ = 0; Q = 0;
end
assign nQ = ~Q;
always@*
    case({nS,nR})
        2'b10:
            begin
                pQ <= 0; Q <= 0;
            end
        2'b01:
            begin
                pQ <= 1; Q <= 1;
            end
        2'b11:
            begin
                if(~C & T)
                    pQ <= ~Q;
                else if (C)
                    Q <= pQ;
            end
    endcase
endmodule
```

2.3. JK-триггер

В машинах используются RS, T и D триггеры, а на универсальном JK-триггере можно построить любой из этих триггеров.

JK-триггер — это такой триггер, который имеет 2 информационных входа: J и K (рис. 22). При подаче на входы J и K «1» (J=K=1) триггер работает как T-триггер, т.е. меняет свое состояние на противоположное, в остальных случаях он работает как RS-триггер. Если вход J считать входом S, а вход K считать входом R, его таблица истинности представлена в табл. 6.

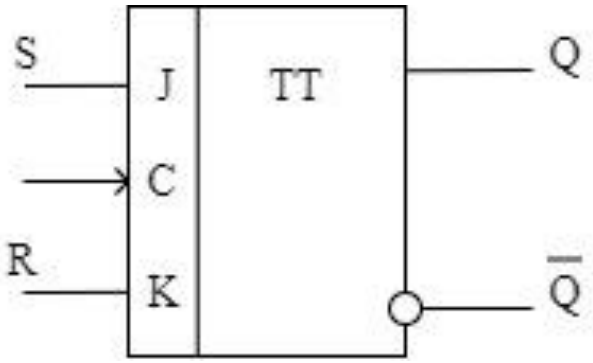


Рисунок 22. Структурное обозначение JK-триггера

Таблица 6. Таблица истинности JK-триггера при C = 1

J	K	Q_{T+1}	Примечание
0	0	Q_T	Хранение старого состояния
0	1	0	Установка «0»
1	0	1	Установка «1»
1	1	$\overline{Q_T}$	Переброс

Синхронный JK-триггер можно также описать с использованием Verilog HDL (листинг 4), временная диаграмма работы модуля представлена на рис. 23.

Листинг 4. Модуль синхронного JK-триггера, описанный на языке описания аппаратуры Verilog

```
module jk_trig(
    input nS, nR, C, J, K,
    output reg Q,
    output nQ
);
reg pQ;
initial
begin
    pQ = 0; Q = 0;
end
assign nQ = ~Q;
always@*
    case({nS,nR})
        2'b10:
            begin
                pQ <= 0; Q <= 0;
            end
        2'b01:
            begin
                pQ <= 1; Q <= 1;
            end
        2'b11:
            begin
                if(~C)
                    case({J, K})
                        2'b00: pQ <= pQ;
                        2'b01: pQ <= 0;
                        2'b10: pQ <= 1;
                        2'b11: pQ <= ~Q;
                    endcase
                else if (C)
                    Q <= pQ;
            end
    endcase
endmodule
```

Универсальные триггеры не имеют запрещенных комбинаций входных сигналов. Они просто переходят из одного режима триггера в другой.

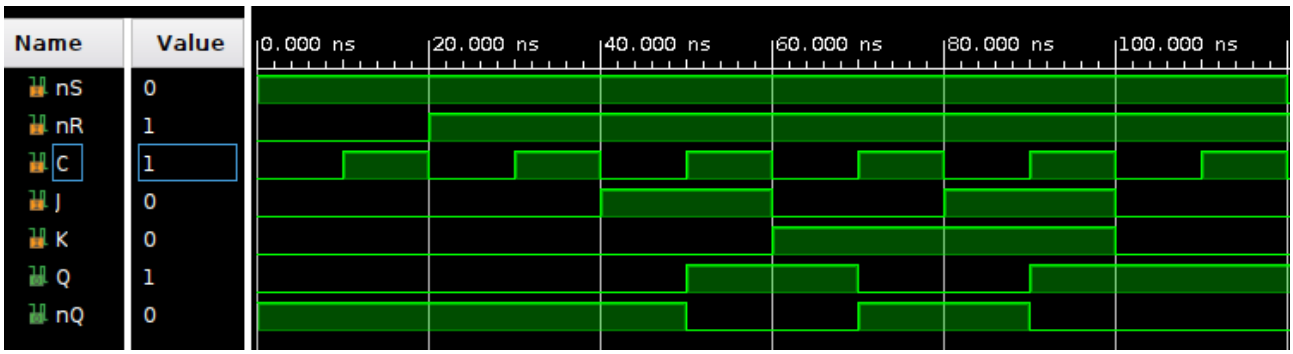


Рисунок 23. Временная диаграмма работы модуля двухступенчатого синхронного JK-триггера

Назначение входов J и K такое же, как и входов S и R, однако, универсальность J и K допускает возможность одновременной подачи сигналов на оба входа J = K = 1. В этом случае триггер переключается в противоположное предыдущему состояние. C — двоичный сигнал определяет, будут ли меняться выходные сигналы на основании действующего алгоритма или нет. Обычно срабатывание происходит при переходе от единичного к нулевому значению, а также срабатывание происходит при отрицательном фронте сигнала. Если на вход C поступает сигнал, равный нулю, то выходные значения будут сохранены. Как только он станет равным единице, формирование значения Q будет происходить по следующим правилам:

– если J=K=0, режим триггера не меняется;
– если J=K=1, происходит изменение состояния элемента на противоположное при поступлении каждого тактового импульса.

В этом случае триггер функционирует идентично делителю частоты. Такой режим работы называют счётным, поскольку его использование позволяет осуществлять подсчёт определённых событий:

– если K = 1, J = 0 выходное значение принимает нулевое значение;
– если K = 0, J = 1, тогда на выходе будет получена единица.

2.4. Счетчики

Счетчик — это узел машины, предназначенный для расчета входных сигналов, выполняется на T-триггерах или на JK-триггерах в режиме T-триггера [6].

Счетчики могут быть: двоичные с модулем (пере)счета ($M = 2^n$, $M \neq 2^n$); суммирующие; вычитающие; реверсивные; по организации цепей переноса (с последовательным переносом, со сквозным переносом, с параллельным переносом); кольцевые счетчики.

Все счетчики могут быть синхронными или асинхронными (в зависимости от триггеров, на которых они строятся).

Счетчики могут быть с ЕСТЕСТВЕННЫМ и ПРОИЗВОЛЬНЫМ порядком счета. При естественном порядке счета результат счета изменяется на единицу при поступлении на вход счетчика каждого сигнала (импульса). При произвольном порядке счета, в этом случае счетчики называют также пересчетными устройствами, при поступлении входных сигналов результат счета может изменяться произвольно в соответствии с заданным законом.

Счетчики используются в ЭВМ для осуществления последовательного выполнения команд программы, подсчета числа циклов выполненных операций, образования адресов при обращении к запоминающим устройствам, в качестве делителей частоты в цифровых электронных часах и частотомерах и др.

Основным параметром счетчиков является *модуль счета* (коэффициент пересчета, емкость счетчика) Ксч. Модулем счета называют число состояний, которые он приобретает под действием входных сигналов. Если счетчик начал считать с начального состояния, то через каждые Ксч входных сигналов он снова возвращается в начальное состояние, а на его выходе появляется сигнал Ксч-ичного переноса.

Виды счетчиков различают по следующим признакам:

По значению модуля счета: *двоичные* (Ксч $=2^n$, где n число разрядов); *двоично-кодированные* (имеют произвольный модуль счета, но все состояния кодируются двоичными кодами); счетчики *с одинарным кодированием* (состояние счетчика определяется местом расположения единственной единицы); счетчики *с унитарным кодированием* (состояние счетчика представлено количеством единиц) и др.

По направлению счета: *суммирующие* (прямого счета); вычитающие (обратного счета); *реверсивные*, т.е. способными работать как в режиме суммирования, так и в режиме вычитания.

По структурной организации: *последовательные* (асинхронные); *параллельные* (синхронные); *параллельно-последовательные*.

Помимо входа для приема подсчитываемых сигналов счетчик может иметь вход общего сброса R и входы данных D_i для параллельной загрузки произвольного кода. В последнем случае для осуществления загрузки предусматривается еще один вход — вход разрешения параллельной загрузки PL (parallel 1oad).

2.5. Пример построения счетчика на D-триггерах

В качестве примера рассмотрим решение следующей задачи: необходимо построить на базе D-триггера суммирующий счетчик по модулю 13 с шагом 10. Для решения данной задачи нужно выполнить несколько шагов:

1. Определить количество разрядов, необходимых для описания работы счетчика. Так как счетчик по модулю 13, следовательно, он должен включать в себя 13 элементов (от 0 до 12). Старшее значение равно 12. Для описания значения 12 требуется 4 разряда в двоичной системе, следовательно, потребуется 4 триггера (по одному на каждый разряд).
2. Построение таблицы истинности. Таблица истинности суммирующего счетчика по модулю 13 с шагом 10 представлена в табл. 7.

Таблица 7. Таблица истинности суммирующего счетчика по модулю 13 с шагом 10 на базе D-триггеров

Q3(t)	Q2(t)	Q1(t)	Q0(t)	Q3(t+1)	Q2(t+1)	Q1(t+1)	Q0(t+1)
0	0	0	0	1	0	1	0
0	0	0	1	1	0	1	1
0	0	1	0	1	1	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1
1	1	0	1	*	*	*	*
1	1	1	0	*	*	*	*
1	1	1	1	*	*	*	*

3. После построения таблицы истинности требуется провести минимизацию функций каждого разряда от Q3 до Q0. Минимизация функций при помощи карт Карно представлена на рис. 24-27.

Q3Q2 \ Q1Q0	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	1	X	X	X
10	0	0	1	0

Результат

$$F = Q3\,Q2 + Q3\,Q1\,Q0 + \overline{Q3}\,\overline{Q2}\,\overline{Q0} + \overline{Q3}\,\overline{Q2}\,\overline{Q1}$$

Рисунок 24. Минимизация разряда Q3(t+1) при помощи карты Карно

Q3Q2 \ Q1Q0	00	01	11	10
00	0	0	0	1
01	0	0	1	0
11	0	X	X	X
10	1	1	0	1

Результат

$$F = Q3\,\overline{Q1}\,Q0 + Q3\,\overline{Q2}\,\overline{Q0} + Q2\,Q1\,Q0 + \overline{Q2}\,Q1\,\overline{Q0}$$

Рисунок 25. Минимизация разряда Q2(t+1) при помощи карты Карно

Q3Q2 \ Q1Q0	00	01	11	10
00	1	1	0	0
01	0	1	0	1
11	0	X	X	X
10	0	1	0	1

Результат

$$F = \overline{Q1}\,Q0 + Q3\,Q1\,\overline{Q0} + Q2\,Q1\,\overline{Q0} + \overline{Q3}\,\overline{Q2}\,\overline{Q1}$$

Рисунок 26. Минимизация разряда Q1(t+1) при помощи карты Карно

Q3Q2 \ Q1Q0	00	01	11	10
00	0	1	0	0
01	1	0	0	1
11	1	X	X	X
10	1	0	0	1

Результат

$$F = Q3\,\overline{Q0} + Q2\,\overline{Q0} + \overline{Q3}\,\overline{Q2}\,\overline{Q1}\,Q0$$

Рисунок 27. Минимизация разряда Q0(t+1) при помощи карты Карно

4. В результате минимизации на базе полученных выражений нужно построить комбинационную схему в программной среде Logisim и проверить корректность ее работы. Результаты выражений по разрядам требуется соответственно подключить ко входу D D-триггера. К коммутатору Q выходов для индикации значений требуется подключить 16-сегментный индикатор. Реализованная комбинационная схема суммирующего счетчика по модулю 13 с шагом 10 на базе D-триггеров представлена на рис. 28.

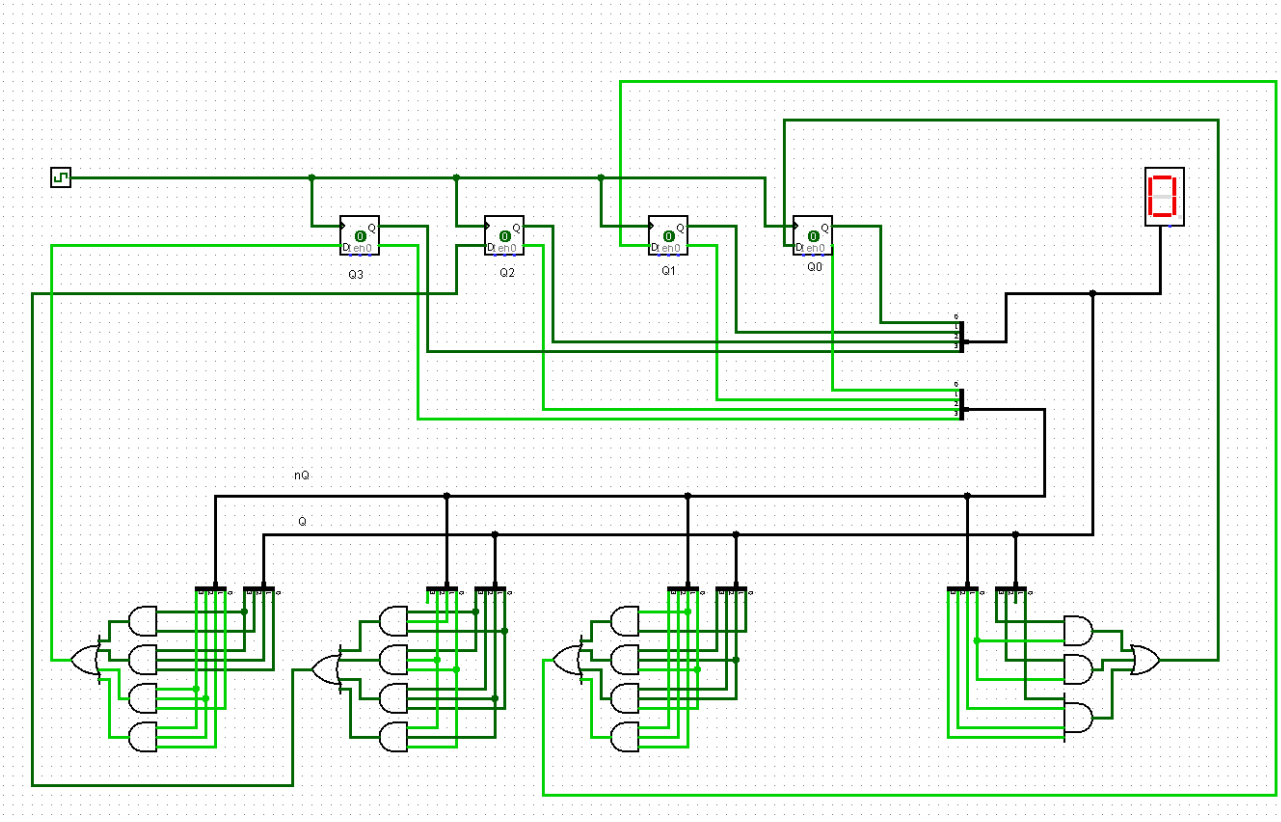


Рисунок 28. Комбинационная схема суммирующего счетчика по модулю 13 с шагом 10 на базе D-триггеров в программной среде Logisim

2.6. Пример построения счетчика на Т-триггерах

Для построения счетчика на Т-триггерах требуется использовать таблицу истинности счетчика на базе D-триггера, описанную в разделе 2.5, и Т-триггеры. Принцип построения новой таблицы истинности следующий: если в следующем такте идет изменение значения разряда, то тогда в таблицу от t+1 пишется 1, иначе 0. Так, например, в текущем примере старое состояние было 0001_2, а новое будет 1011_2, в таблице Т-триггера будет значение 1010, так как изменились значения только старшего 3 разряда и 1 разряда. Составленная таблица истинности в соответствии с описанным принципом представлена в табл. 8. Дальнейшая последовательность действий для построения комбинационной схемы счетчика на базе Т-триггеров абсолютно идентичная, что и на базе D-триггеров.

Таблица 8. Таблица истинности суммирующего счетчика по модулю 13 с шагом 10 на базе Т-триггеров

Q3(t)	Q2(t)	Q1(t)	Q0(t)	Q3(t+1)	Q2(t+1)	Q1(t+1)	Q0(t+1)
0	0	0	0	1	0	1	0
0	0	0	1	1	0	1	0
0	0	1	0	1	1	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	0	1	1
1	0	0	0	1	1	0	1
1	0	0	1	1	1	1	1
1	0	1	0	1	1	0	1
1	0	1	1	0	0	1	1
1	1	0	0	0	1	0	1
1	1	0	1	*	*	*	*
1	1	1	0	*	*	*	*
1	1	1	1	*	*	*	*

После построения таблицы истинности требуется провести минимизацию функций каждого разряда от Q3 до Q0. Минимизация функций при помощи карт Карно представлена на рис. 29-32.

Q3Q2 \ Q1Q0	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	X	X	X
10	1	1	0	1

Результат

$$F = \overline{Q2}\,\overline{Q0} + \overline{Q2}\,\overline{Q1}$$

Рисунок 29. Минимизация разряда Q3(t+1) при помощи карты Карно

Q3Q2 \ Q1Q0	00	01	11	10
00	0	0	0	1
01	1	1	0	1
11	1	X	X	X
10	1	1	0	1

Результат

$$F = Q3\,\overline{Q1} + Q2\,\overline{Q1} + Q1\,\overline{Q0}$$

Рисунок 30. Минимизация разряда Q2(t+1) при помощи карты Карно

Q1Q0 / Q3Q2	00	01	11	10
00	1	1	1	1
01	0	1	1	0
11	0	X	X	X
10	0	1	1	0

Результат

$$F = Q0 + \overline{Q3}\,\overline{Q2}$$

Рисунок 31. Минимизация разряда Q1(t+1) при помощи карты Карно

Q1Q0 / Q3Q2	00	01	11	10
00	0	0	1	0
01	1	1	1	1
11	1	X	X	X
10	1	1	1	1

Результат

$$F = Q3 + Q2 + Q1\,Q0$$

Рисунок 32. Минимизация разряда Q0(t+1) при помощи карты Карно

В результате минимизации на базе полученных выражений нужно построить комбинационную схему в программной среде Logisim и проверить корректность ее работы. Результаты выражений по разрядам требуется соответственно подключить ко входу T T-триггера. К коммутатору Q выходов для индикации значений требуется подключить 16-сегментный индикатор.

Реализованная комбинационная схема суммирующего счетчика по модулю 13 с шагом 10 на базе T-триггеров представлена на рис. 33.

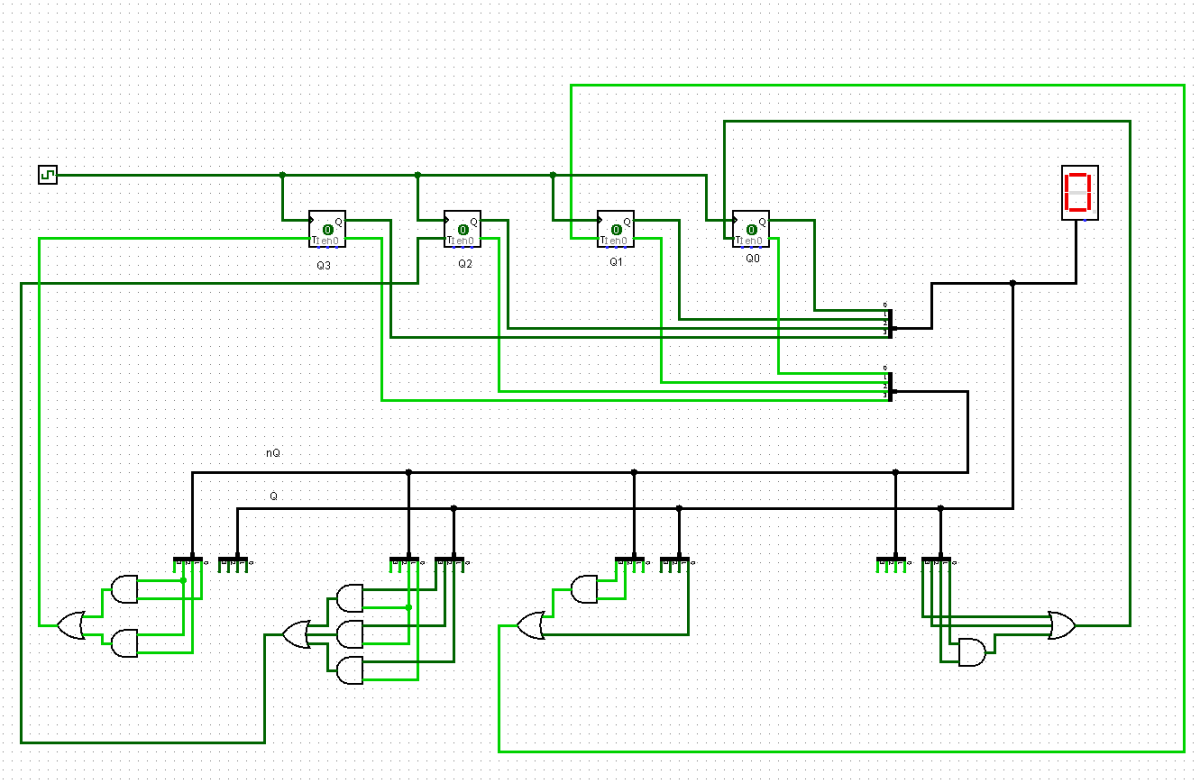


Рисунок 33. Комбинационная схема суммирующего счетчика по модулю 13 с шагом 10 на базе Т-триггеров в программной среде Logisim

2.7. Пример построения счетчика на JK-триггерах

Для построения счетчика на JK-триггерах требуется использовать таблицу истинности счетчика на базе D-триггера, описанную в разделе 2.5, и JK-триггеры. На основании таблицы истинности JK-триггера и принципов его работы уточним таблицу истинности (табл. 6) для определения следующего состояния Q, которая потребуется для перевода. Новая таблица истинности JK-триггера представлена в табл. 9.

Таблица 9. Таблица истинности JK-триггера для Q(t+1)

J	K	Q(t+1)
0	*	Хранение «0»
1	*	1
*	1	0
*	0	Хранение «1»

Для составления таблицы истинности счетчика на базе JK-триггеров требуется в таблице истинности D-триггера выписать текущее и следующее состояние каждого разряда и сделать запись в новой таблице истинности для JK. Так, например, если старое состояние было 0 и новое также останется 0, то в таблицу истинности счетчика на базе JK-триггеров будет занесено 0 и * для J и K соответственно.

Составленная в соответствии с этим принципом таблица истинности суммирующего счетчика по модулю 13 с шагом 10 на базе JK-триггера представлена в табл. 10.

Таблица 10. Таблица истинности суммирующего счетчика по модулю 13 с шагом 10 на базе JK-триггеров

J3	K3	J2	K2	J1	K1	J0	K0
1	*	0	*	1	*	0	*
1	*	0	*	1	*	*	0
1	*	1	*	*	1	0	*
0	*	0	*	*	1	*	1
0	*	*	1	0	*	1	*
0	*	*	1	1	*	*	1
0	*	*	1	*	0	1	*
0	*	*	0	*	1	*	1
*	1	1	*	0	*	1	*
*	1	1	*	1	*	*	1
*	1	1	*	*	0	1	*
*	0	0	*	*	1	*	1
*	0	*	1	0	*	1	*

После построения таблицы истинности требуется провести минимизацию функций каждого разряда от J3 и K3 до J0 и K0. Минимизация функций при помощи карт Карно представлена на рис. 34-41.



Рисунок 34. Минимизация разряда J3 при помощи карты Карно

Q3Q2 \ Q1Q0	00	01	11	10
00	X	X	X	X
01	X	X	X	X
11	0	X	X	X
10	1	1	0	1

Результат

$$F = \overline{Q1}\, Q0 + \overline{Q2}\, \overline{Q0}$$

Рисунок 35. Минимизация разряда K3 при помощи карты Карно

Q3Q2 \ Q1Q0	00	01	11	10
00	0	0	0	1
01	X	X	X	X
11	X	X	X	X
10	1	1	0	1

Результат

$$F = Q3\, \overline{Q1} + Q1\, \overline{Q0}$$

Рисунок 36. Минимизация разряда J2 при помощи карты Карно

Q1Q0 / Q3Q2	00	01	11	10
00	X	X	X	X
01	1	1	0	1
11	1	X	X	X
10	X	X	X	X

Результат

$$F = \overline{Q0} + \overline{Q1}$$

Рисунок 37. Минимизация разряда K2 при помощи карты Карно

Q1Q0 / Q3Q2	00	01	11	10
00	1	1	X	X
01	0	1	X	X
11	0	X	X	X
10	0	1	X	X

Результат

$$F = Q0 + \overline{Q3}\,\overline{Q2}$$

Рисунок 38. Минимизация разряда J1 при помощи карты Карно

Q3Q2 \ Q1Q0	00	01	11	10
00	X	X	1	1
01	X	X	1	0
11	X	X	X	X
10	X	X	1	0

Результат

$$F = Q0 + \overline{Q3}\,\overline{Q2}$$

Рисунок 39. Минимизация разряда K1 при помощи карты Карно

Q3Q2 \ Q1Q0	00	01	11	10
00	0	X	X	0
01	1	X	X	1
11	1	X	X	X
10	1	X	X	1

Результат

$$F = Q3 + Q2$$

Рисунок 40. Минимизация разряда J0 при помощи карты Карно

Q1Q0 / Q3Q2	00	01	11	10
00	X	0	1	X
01	X	1	1	X
11	X	X	X	X
10	X	1	1	X

Результат

F = Q3 + Q2 + Q1

Рисунок 41. Минимизация разряда K0 при помощи карты Карно

В результате минимизации на базе полученных выражений нужно построить комбинационную схему в программной среде Logisim и проверить корректность ее работы. Результаты выражений по разрядам требуется соответственно подключить ко входу J и K JK-триггера. К коммутатору Q выходов для индикации значений требуется подключить 16-сегментный индикатор.

Реализованная комбинационная схема суммирующего счетчика по модулю 13 с шагом 10 на базе JK-триггеров представлена на рис. 42.

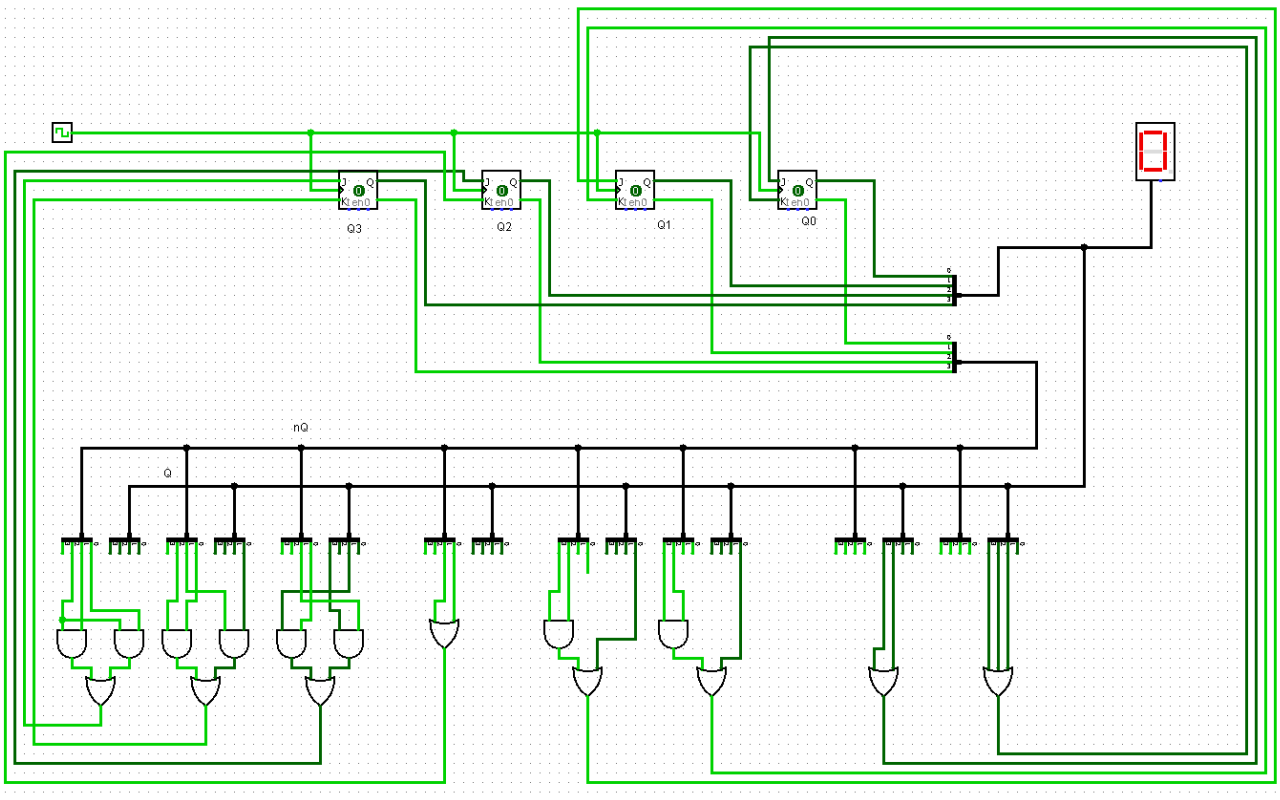


Рисунок 42. Комбинационная схема суммирующего счетчика по модулю 13 с шагом 10 на базе JK-триггеров в программной среде Logisim

2.8. Задания для самостоятельного решения

Задачи для закрепления данной темы и самостоятельного решения представлены в табл. 11. Требуется построить комбинационную схему заданного типа счетчика на базе заданного типа триггеров.

Таблица 11. Задачи для самостоятельного решения

№ варианта	Счетчик	Тип триггеров
1	Вычитающий по модулю 10 с шагом 4	T
2	Вычитающий по модулю 12 с шагом 3	JK
3	Суммирующий по модулю 17 с шагом 13	D
4	Суммирующий по модулю 22 с шагом 16	D
5	Суммирующий по модулю 14 с шагом 13	JK
6	Вычитающий по модулю 9 с шагом 5	T
7	Вычитающий по модулю 14 с шагом 6	T
8	Суммирующий по модулю 14 с шагом 13	T
9	Суммирующий по модулю 22 с шагом 11	D
10	Суммирующий по модулю 22 с шагом 12	D
11	Суммирующий по модулю 12 с шагом 10	JK
12	Суммирующий по модулю 11 с шагом 5	JK
13	Вычитающий по модулю 14 с шагом 4	T
14	Суммирующий по модулю 15 с шагом 4	JK
15	Суммирующий по модулю 20 с шагом 14	D
16	Вычитающий по модулю 13 с шагом 6	T
17	Вычитающий по модулю 9 с шагом 5	T
18	Суммирующий по модулю 13 с шагом 9	T
19	Вычитающий по модулю 13 с шагом 3	JK
20	Суммирующий по модулю 12 с шагом 5	T
21	Суммирующий по модулю 12 с шагом 7	T
22	Вычитающий по модулю 13 с шагом 8	JK
23	Вычитающий по модулю 19 с шагом 11	D
24	Вычитающий по модулю 18 с шагом 14	D
25	Вычитающий по модулю 13 с шагом 5	JK
26	Суммирующий по модулю 14 с шагом 9	T
27	Суммирующий по модулю 14 с шагом 5	T
28	Суммирующий по модулю 15 с шагом 5	T
29	Вычитающий по модулю 21 с шагом 7	D
30	Вычитающий по модулю 17 с шагом 3	D
31	Вычитающий по модулю 14 с шагом 10	JK
32	Вычитающий по модулю 10 с шагом 8	JK
33	Суммирующий по модулю 13 с шагом 9	T
34	Суммирующий по модулю 12 с шагом 5	JK
35	Вычитающий по модулю 20 с шагом 10	D

Окончание табл. 11

№ варианта	Счетчик	Тип триггеров
36	Суммирующий по модулю 15 с шагом 6	T
37	Суммирующий по модулю 11 с шагом 5	T
38	Суммирующий по модулю 11 с шагом 7	T
39	Суммирующий по модулю 15 с шагом 7	JK
40	Суммирующий по модулю 11 с шагом 3	T

ПРАКТИЧЕСКАЯ РАБОТА № 3

3.1. Учебное операционное устройство и его компоненты

Центральный процессор (ЦП) — главный элемент ВМ, который реализует выполнение команд. В состав ЦП входят регистры:

– PC — счетчик команд;
– IR — регистр команды;
– AC — аккумулятор;
– MAR — регистр адреса памяти;
– MDR — регистр данных памяти;
– InpR, OutR — регистры ввода и вывода данных, соответственно;
– АЛУ — арифметико-логическое устройство;
– M — память.

Все эти устройства входят в состав устройства управления (УУ) и операционного устройства (ОУ) ЦП. Рассмотрим операционное устройство по подробнее. В состав учебного ОУ, будут входить: регистры, дешифратор, мультиплексоры, АЛУ и ОЗУ. Коротко опишем каждое из них.

Регистр — это узел машины, выполненный на триггерах, и предназначенный для записи, хранения и выдачи информации. Как правило, регистры строятся на RS или D триггерах [7,8,9]

RS-триггер делится на два основных вида: синхронный и асинхронный. В регистрах, зачастую, используется первый из них. Синхронный RS-триггер, триггер, который при подаче «1» на вход S, устанавливается в «1». Если «1» подается на вход R, то триггер устанавливается в «0», и это при условии, что на вход синхронизации C подана «1». Структурная схема синхронного RS-триггера изображена на рис. 43, его таблица переходов — в табл. 12.

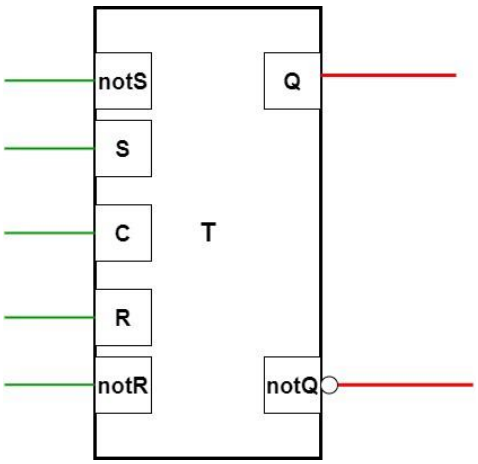


Рисунок 43. Структурная схема синхронного RS-триггера

Таблица 12. Таблица переходов синхронного RS-триггера при C = 1

S	R	Q_{T+1}	Примечание
0	0	Q_T	Хранение старого состояния
0	1	0	Установка «0»
1	0	1	Установка «1»
1	1	*	Запрет

D-триггер — это триггер задержки (от англ. «Delay» — задержка). У данного вида триггеров имеется один вход «D» и для установки такого триггера в «1», нужно подать 1 на вход, и вход синхросигнала «C», также установить в «1». Время задержки определяется частотой синхросигнала, если информация на входе «D» изменится, то выходной повторит его изменение, при условии наличия сигнала на входе для синхросигнала, в противном случае, сигнал останется неизменным. Структурная схема D-триггера изображена на рис. 44, его таблица истинности — в табл. 13.

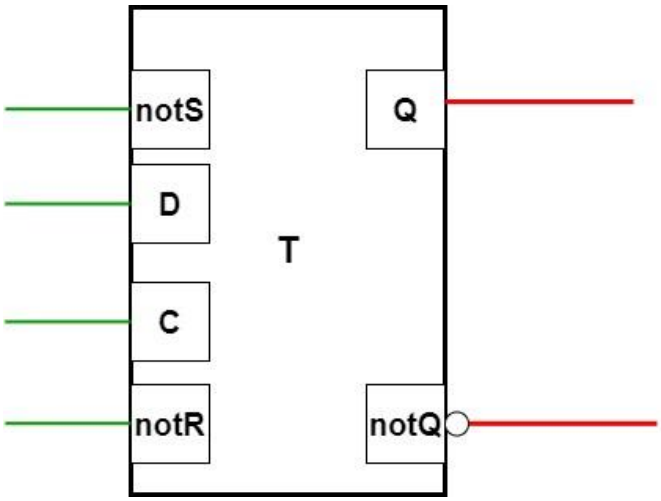


Рисунок 44. Структурная схема D-триггера

Таблица 13. Таблица истинности D-триггера при C = 1

D_t	Q_{T+1}	Примечание
0	0	Установка «0»
1	1	Установка «1»

Число триггеров, на основе которых строится тот или иной регистр определяет количество разрядов информации, которую он сможет хранить. Информация, представленная набором битов в регистре, обрабатывается как единое целое и именуется — машинное слово. Для нашего операционного устройства регистры имеют разный размер, 8, 12 или 16 разрядов, в зависимости от предназначения регистра в операционном устройстве.

Дешифратор (DC) — это комбинационная схема, которая имеет n входов и 2^n выходов. Структурная схема дешифратора представлена на рис. 45, его таблица истинности — в табл. 14.

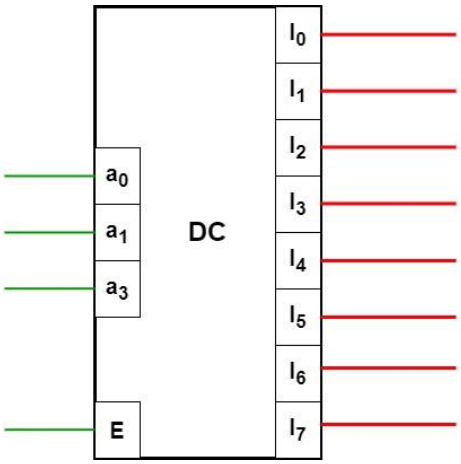


Рисунок 45. Структурная схема 3-х разрядного дешифратора

В зависимости от сигнала, поданного на вход дешифратора, формируется выходной сигнал «1» на заданном выходе, все остальные в этот момент времени равны «0». Вход «E» — это разрешающий вход, его задача предотвратить формирование сигнала на выходе, если на нем установлен «0». Стоит отметить, что для нашего ОУ выбран дешифратор с инверсным разрешающим сигналом.

Таблица 14. Таблица истинности дешифратора 3-8

Вход			Выход							
a2	a1	a0	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0
0	0	0	0	0	0	0	0	0	0	**1**
0	0	1	0	0	0	0	0	0	**1**	0
0	1	0	0	0	0	0	0	**1**	0	0
0	1	1	0	0	0	0	**1**	0	0	0
1	0	0	0	0	0	**1**	0	0	0	0
1	0	1	0	0	**1**	0	0	0	0	0
1	1	0	0	**1**	0	0	0	0	0	0
1	1	1	**1**	0	0	0	0	0	0	0

Следующее устройство — мультиплексор. Мультиплексор — это комбинационная схема, имеющая n адресных входов, 2^nинформационных входов и только один выход. В зависимости от комбинации, поданной на адресные входы, на выход подается входной цифровой сигнал только с одного информационного входа. Структурная схема изображена на рис. 46, его таблица истинности — в табл. 15. В различных источниках мультиплексор может именоваться как «MS» или «MX».

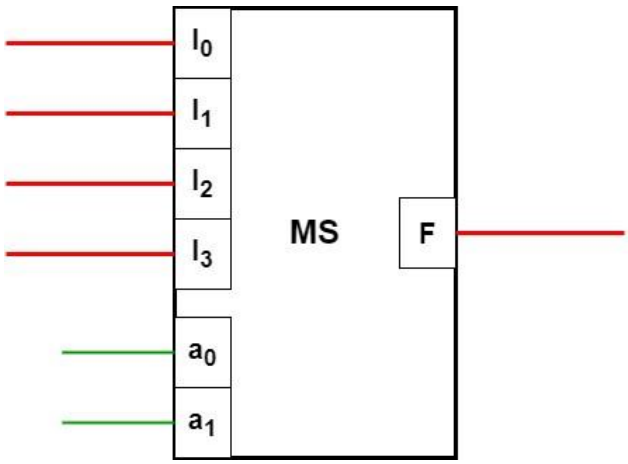


Рисунок 46. Структурная схема мультиплексора 4-1

Таблица 15. Таблица истинности мультиплексора 4-1

Вход						Выход
I_3	I_2	I_1	I_0	a_1	a_0	F
0	0	0	**1**	0	0	I_0
0	0	**1**	0	0	1	I_1
0	**1**	0	0	1	0	I_2
1	0	0	0	1	1	I_3

Арифметико-логическое устройство (АЛУ) — часть процессора, выполняющая арифметические, логические и иные операции над данными. Пример одноразрядного АЛУ приведен на рис. 47. Обобщенно, АЛУ состоит из 3 регистров, сумматора с соответствующими логическими схемами и элемента управления выполняемым процессом. Устройство работает в соответствии с сообщаемыми ему кодами операций, которые при пересылке данных нужно выполнить над переменными, помещаемыми в регистры.

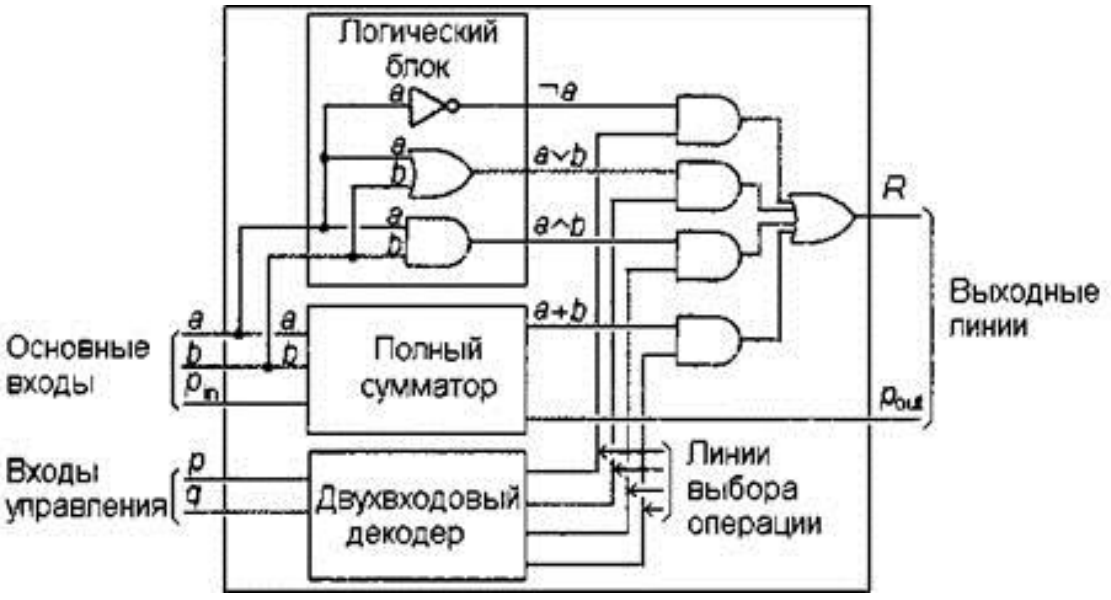


Рисунок 47. Упрощенная схема одноразрядного АЛУ

Пусть некоторая схема имеет два входа и один выход. Причем если на входе S имеется ноль, то на выходе тоже имеется ноль, если на входе S имеется 1, то выход схемы соответствует входу D.

S	D	Y
0	0	0
0	1	0
1	0	0
1	1	1

Фактически эта схема представляет собой известную схему «И». Она представляет собой цифровой вентиль, где один из входов (S) представляет собой разрешающий вход, а другой (D) — информационный (рис. 48).

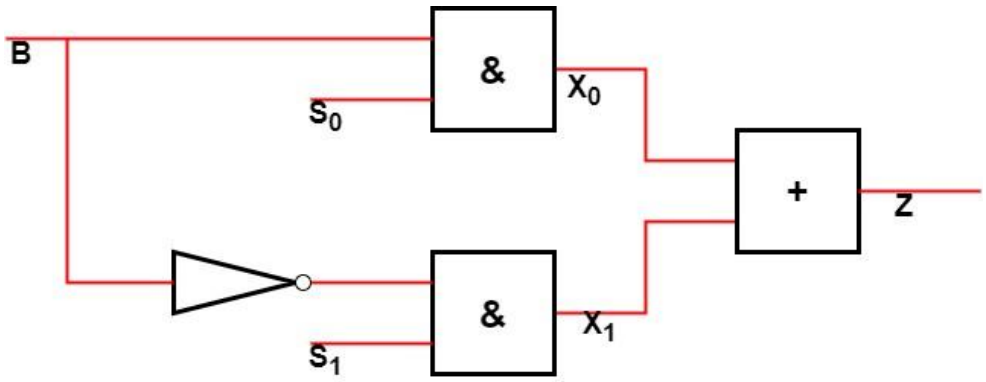


Рисунок 48. Цифровой вентиль

Таблицу состояний схемы сразу сокращаем, учитывая, что цифровые вентили пропускают на выход либо сигнал «B», либо его инверсию в зависимости от состояния управляющих входов S0 и S1.

S1	S0	X1	X2	Z
0	0	0	0	0
0	1	B	0	B
1	0	0	$\bar{B}$	$\bar{B}$
1	1	B	$\bar{B}$	1

При расположении такой схемы в каждом разряде мы имеем выполнение логических функций поразрядно. АЛУ должно обеспечивать выполнение как арифметических, так и логических функций. Условимся, что при значении 1 управляющего сигнала M будут выполняться логические функции, а при значении 0 — арифметические. В качестве логических функций будем рассматривать функции «ИЛИ», «исключающее ИЛИ», «И» и инверсию.

Для нашего ОУ АЛУ будет выполнять набор микроопераций, представленный в табл. 16.

Таблица 16. Таблица микроопераций учебной ОУ

M	S1	S0	C	Микрооперации	Наименование
0	0	0	0	A	Передача A
0	0	0	1	A+1	Инкремент A
0	0	1	0	A+B	Сумма
0	0	1	1	A+B+1	Сумма с инкрементом
0	1	0	0	A-B-1	Разность с декрементом
0	1	0	1	A-B	Разность
0	1	1	0	A-1	Декремент A
0	1	1	1	A	Передача A
1	0	0	X	A∨B	ИЛИ
1	0	1	X	$A \oplus B$	Искл. ИЛИ
1	1	0	X	A∧B	И
1	1	1	X	$\bar{A}$	Не A

Последним элементом нашего ОУ является оперативное запоминающее устройство (ОЗУ), упрощенная схема которого представлена на рис. 49.

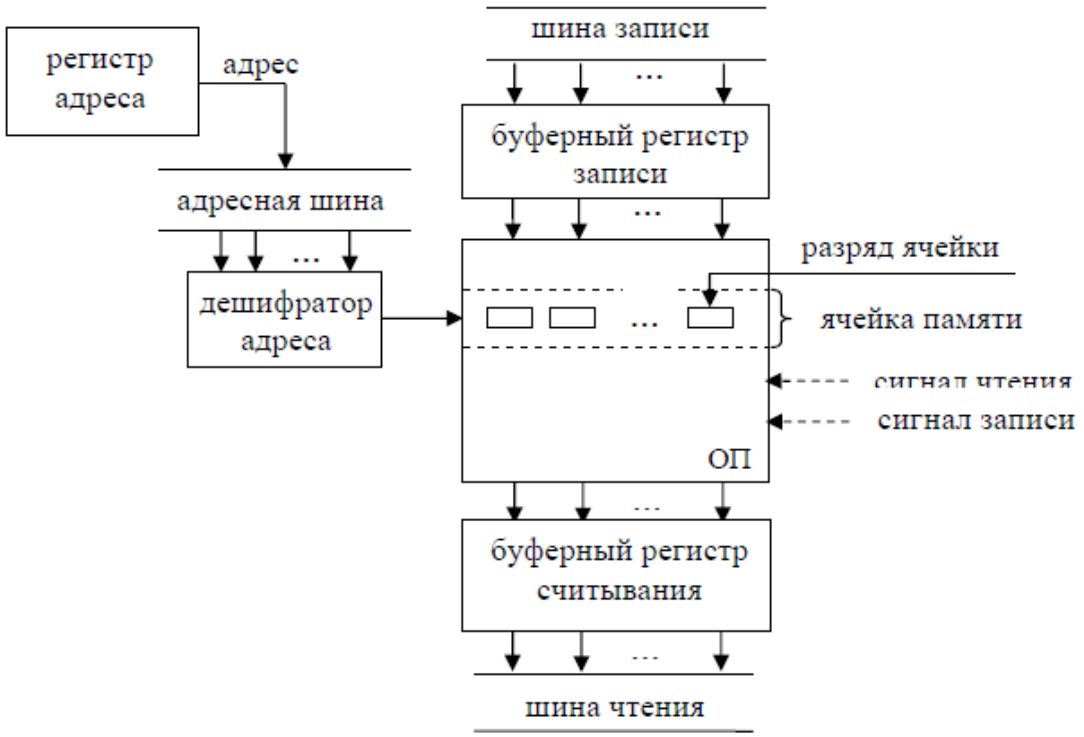


Рисунок 49. Упрощенная схема ОЗУ

Оперативное запоминающее устройство, или ОЗУ — это специальный тип памяти в электронных устройствах, предназначенный для быстрой обработки данных с помощью процессора. Физически современные образцы ОЗУ чаще всего представляет собой плату с банками памяти для хранения информации, подключающуюся к материнской плате персонального компьютера, или набор микросхем, подсоединенный к чипсету мобильного устройства. Для нашего

учебного ОУ ОЗУ будет хранить информацию о командах, а также операндах, которые АЛУ будет использовать в процессе своей работы.

Структурная схема учебного ОУ изображена на рис. 50.

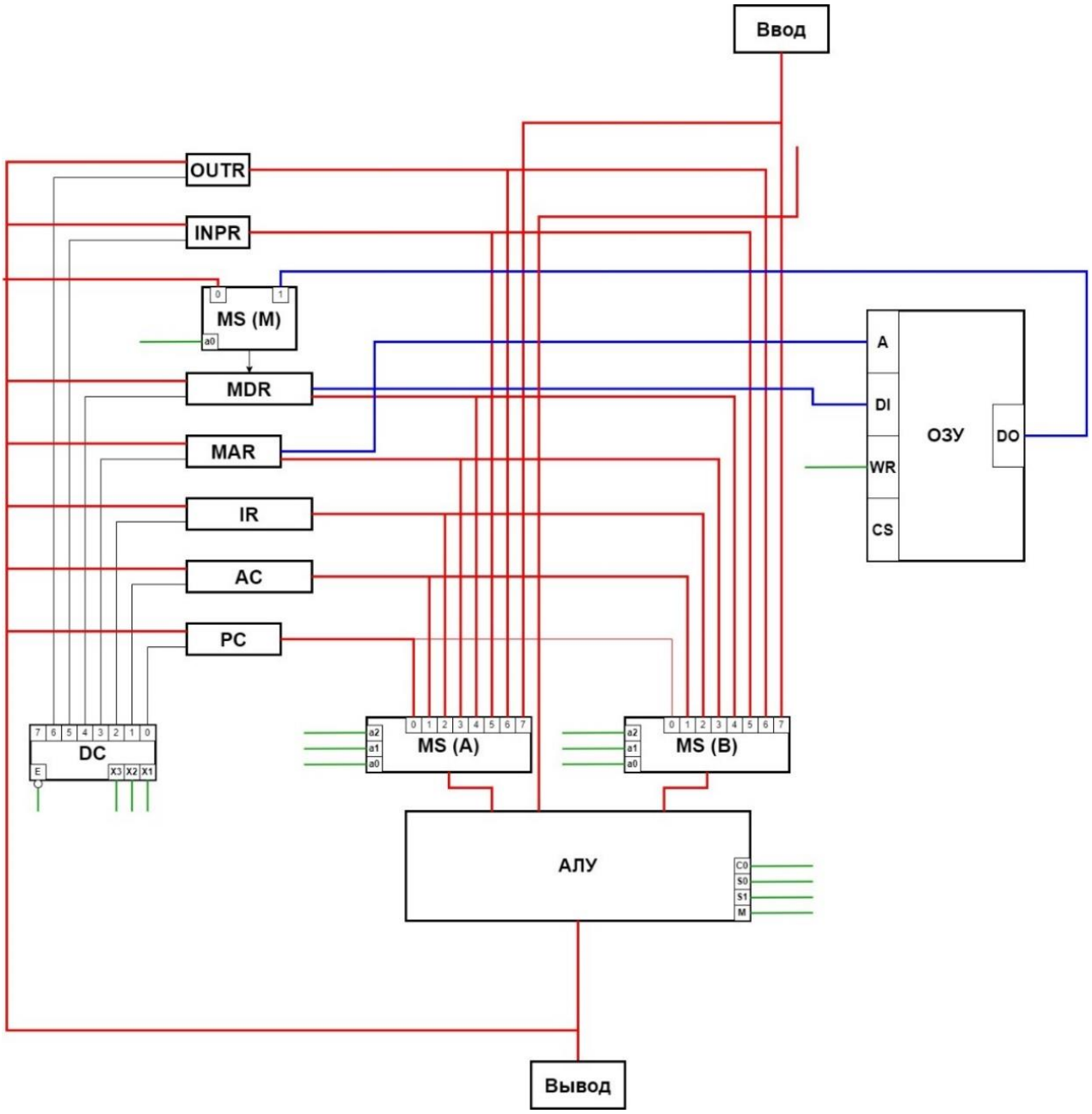


Рисунок 50. Структурная схема учебного ОУ

3.2. Запись управляющего слова ЦП

Для управления устройствами, входящими в состав учебного ОУ, используется управляющее слово, которое содержит 16 разрядов. В зависимости от значений разрядов управляющего слова ОУ выполнит ту или иную операцию над данными, которые хранятся в регистрах.

В представленном учебном ОУ (рис. 50), содержится 7 регистров, 3 мультиплексора, 1 дешифратор, 1 ОЗУ, 1 АЛУ. Каждый регистр отвечает за свою область, и хранит определенное значение, которое необходимо для работы всей схемы. Коротко о каждом из них:

- PC — Programm Counter — Счетчик команд — хранит значение номера текущей команды;
- AC — Accumulator — Аккумулятор — главный счетный регистр ЦП;
- IR — Instruction Register — регистр команд — хранит значение команды в зашифрованном виде;
- MAR — Memory Address Register — регистр адреса памяти — хранит значение адреса команды/операнда в памяти ОЗУ;
- MDR — Memory Data Register — регистр информации памяти — хранит значение прочитанное из ОЗУ, или значение, которое подлежит записи в ОЗУ;
- INPR — Input Register — регистр ввода — хранит значение, поступившее из внешней среды (например, было введено с клавиатуры);
- OUTR — Output Register — регистр вывода — хранит значение, которое должно быть отправлено во внешнюю среду (например, должно быть выведено на экран).

Для удобства использования управляющее слово ЦП запишем в виде таблицы (табл. 17), которая будет использована для выполнения демонстрационного задания.

Таблица 17. Управляющее слово ЦП

MS (A)			MS (B)			АЛУ				DC				MS (M)	WR
a2	a1	a0	a2	a1	a0	M	S1	S0	C0	E	x3	x2	x1	a0	wr

Пусть нам требуется выполнить на ОУ следующую операцию:

$$MAR \leftarrow PC + IR \quad (5)$$

Читается подобная запись следующим образом: «В регистр MAR записать сумму значений, находящихся в регистрах PC и IR, соответственно». Первое что необходимо сделать, это проанализировать записанную формулировку задания. Регистр MAR — это регистр, в который нужно записать результат. Регистр PC — в нашем случае хранит первое значение, для удобства назовем его «A», а регистр IR, хранит второе значение «B». Далее, нужно обратиться к структурной схеме ОУ (рис. 51), расположенной выше. На ней необходимо найти искомые регистры.

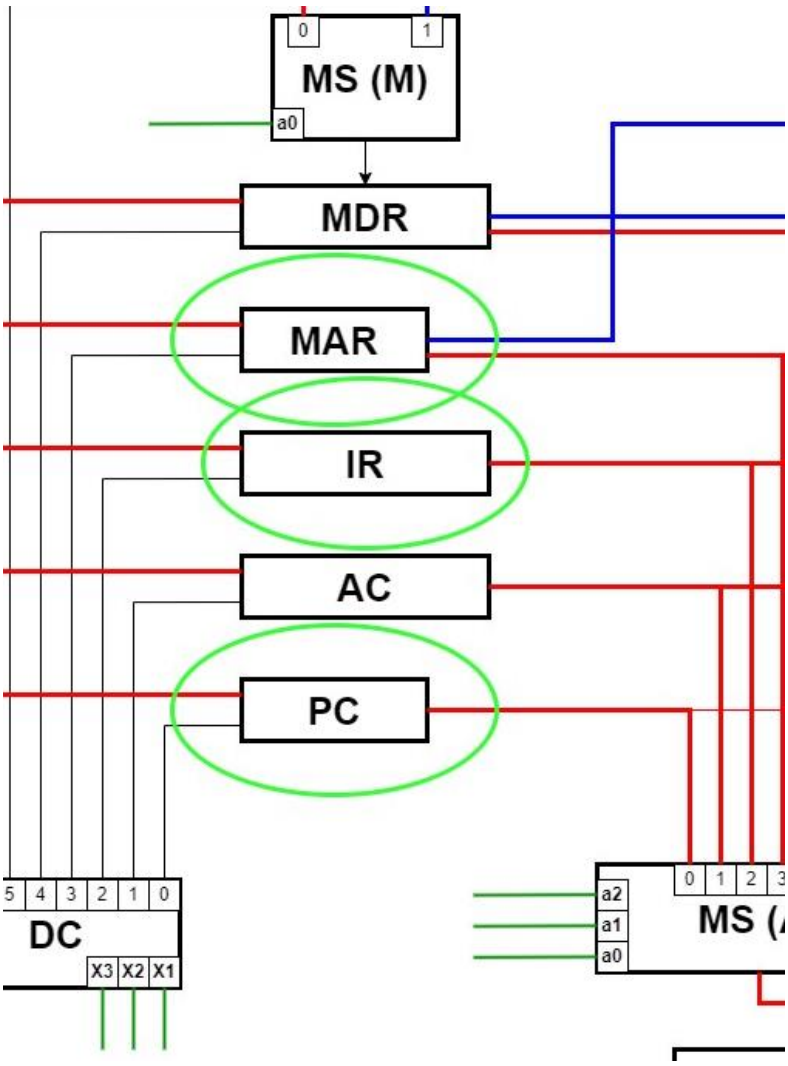


Рисунок 51. Регистры MAR, IR и PC на структурной схеме ОУ

Следующий этап — это анализ табл. 17. Первый столбец — это MS (A), он отвечает за биты, которые будут подаваться на адресные входы левого мультиплексора нашего ОУ. Через этот мультиплексор будет передаваться первый операнд для АЛУ, в нашем случае первое значение «А», хранится в регистре PC, а значит необходимо на управляющие входы мультиплексора подать «000» для того, чтобы он пропустил на выход значение с нулевого информационного входа. Второй столбец — это MS (B) — наш второй операнд, проходит на правый мультиплексор. На управляющие входы этого мультиплексора подаем значение «010», так как необходимо ко входу B АЛУ подключить второй информационный вход мультиплексора. Следующий столбец — это АЛУ (На рис. 52 показано расположение искомых элементов на схеме). АЛУ принимает 4 бита, и найти их можно в табл. 16, в которой записаны все микрооперации, которые способно выполнять АЛУ. Требуемая операция — сложение «+», следовательно, в столбец АЛУ заносим значение «0010».

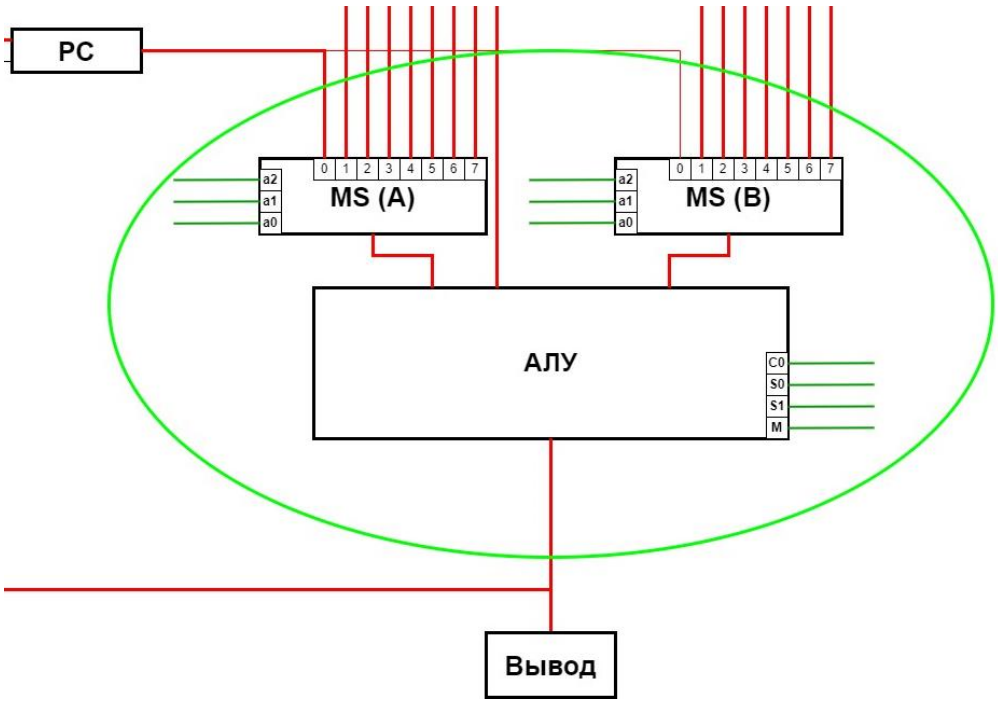


Рисунок 52. Расположение мультиплексоров и АЛУ на структурной схеме ОУ

На данном этапе АЛУ сформировало результат сложения на выходном регистре, и следующим шагом будет отправка результата в пункт назначения. Как видно из схемы (рис. 52), результат из АЛУ может подаваться как на вывод, например, на экран, так и перезаписываться в любой из доступных нам регистров. В нашем случае (операция вычисления формулы (5)) результат должен быть занесен в регистр MAR. В управляющем слове следующий незаполненный столбец — это DC. Задачей дешифратора является разрешение или запрет записи во внутренние регистры. Если от дешифратора к какому-то из регистров поступает значение «1», и дешифратору разрешена работа, то значение с выходного регистра АЛУ, попадет в целевой регистр. Наш регистр — MAR, его «адрес» в дешифраторе — «3», «3» в двоичной системе счисления «11», однако, дешифратор имеет разрядность 3-8, значит необходимо добавить незначащий ноль в старшем разряде, итоговый адрес будет «011». Необходимо также разрешить работу дешифратору, и подать на его разрешающий вход «0» для того, чтобы при поступлении на вход, поданное значение инвертировалось и получилась «1», которая позволит дешифратору выполнить свою работу. Итоговая запись в столбец дешифратора будет «0011».

Следующий столбец — это MS (M), это столбец, который отвечает за адресный вход, подаваемый на мультиплексор, расположенный над регистром MDR, и отвечающий за регулировку записываемых значений в этот регистр. Снова обратимся к схеме (рис. 53), у мультиплексора MS (M), один адресный вход «a0» и

два информационных входа «0» и «1», соответственно. Если адресный вход равен «0», то запись будет производиться из АЛУ, если «1», то запишется значение из ОЗУ. В рассматриваемом случае запись в регистр MDR запрещена (у дешифратора на выходе 4 установлено значение «0»), это означает, что вне зависимости от поданного значения на адресный вход мультиплексора, MDR свое значение не изменит. На основе этого делаем вывод, что значение этого бита управляющего слова нам не важно, записываем туда «x». Значение «x» в этом случае говорит о том, что там будет сохранено предыдущее значение бита.

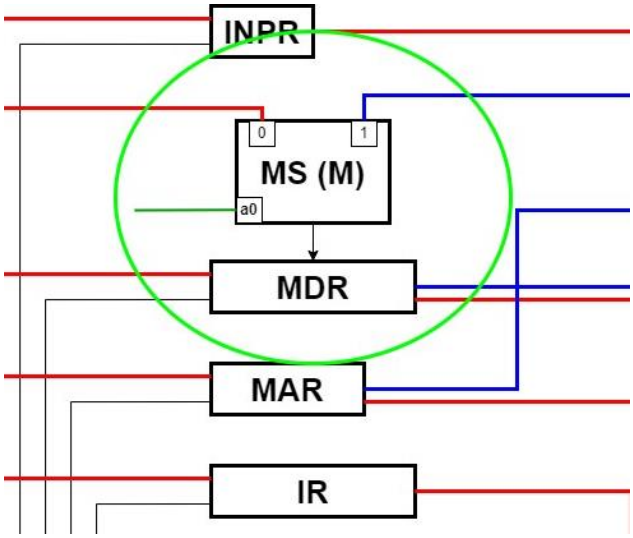


Рисунок 53. Подключение мультиплексора MS (M) к MDR

Последний столбец управляющего слова отвечает за бит записи/чтения значений в ОЗУ. Если бит равен «0», то ОЗУ разрешено чтение информации, то есть оно отправит информацию из некоторой ячейки памяти. Но так как запись в регистр MDR запрещена, а ни с каким другим регистром ОЗУ не связано, то значение «0» также отвечает и за хранение. Значение «1» — отвечает за запись в ячейку памяти. При «1», ОЗУ возьмет значение из регистра MAR в качестве адреса и значение MDR в качестве данных и запишет содержимое MDR в ячейку по адресу MAR. В случае вычисления формулы (5) работа с оперативной памятью не требуется, следовательно, значение бита WR будет «0».

Итоговое управляющее слово будет выглядеть следующим образом (табл. 18).

Таблица 18. Управляющее слово для демонстрационной задачи, формула (5)

MS (A)			MS (B)			АЛУ				DC				MS (M)	WR
a2	a1	a0	a2	a1	a0	M	S1	S0	C0	E	x3	x2	x1	a0	wr
0	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**1**	**1**	**x**	**0**

3.3. Задания для самостоятельного выполнения

Задачи для закрепления данной темы и самостоятельного выполнения представлены в табл. 19. Необходимо составить управляющее слово для действия, приведенного в таблице.

Таблица 19. Варианты практического задания для самостоятельного выполнения

№ варианта	Действие
1	S← INPR ∧ IR
2	INPR← IR ⊕ MAR
3	IR← MAR ∨ OUTR
4	MAR← OUTR – AC
5	OUTR←AC – MDR – 1
6	AC← MDR + PC + 1
7	MDR← PC + INPR
8	OUTR← $INPR$
9	S← INPR + 1
10	INPR← IR + MAR
11	IR← MAR + OUTR + 1
12	MAR← OUTR – AC – 1
13	OUTR← AC – MDR
14	AC← $IR \oplus MDR$
15	MDR← $IR \oplus AC$
16	PC← $MAR \wedge PC$
17	OUTR← IR – 1
18	IR← $MDR \vee IR$
19	PC← $INPR \oplus OUTR$
20	S← $MDR \wedge IR$
21	INPR← MAR + OUTR
22	MDR← OUTR + IR + 1
23	AC← IR – PC – 1
24	MAR← INPR – AC
25	PC← $MAR \oplus PC$
26	IR← OUTR + PC
27	OUTR← MDR + INPR + 1
28	S← INPR – AC – 1
29	MDR← AC – MAR
30	INPR← $IR \vee MDR$
31	AC← $INPR \oplus MDR$
32	MAR← $AC \wedge IR$
33	S← OUTR
34	S← AC+OUTR
35	S← IR+INPR+1

Окончание табл. 19

№ варианта	Действие
36	S←OUTR – MDR
37	S←INPR – MAR – 1
38	S←IR+1
39	S←OUTR – 1
40	S← AC ⊕ PC

ПРАКТИЧЕСКАЯ РАБОТА № 4

4.1. Устройство управления ЦП учебного ОУ

В предыдущей работе, при первом знакомстве с ОУ, было рассмотрено выполнение некоторой (абстрактной) задачи. Теперь рассмотрим более сложный, но, в то же время, более реальный способ работы схемы.

Для того, чтобы генерировались сигналы для учебного ОУ, необходимо устройство управления. Изобразим его на рис. 54.

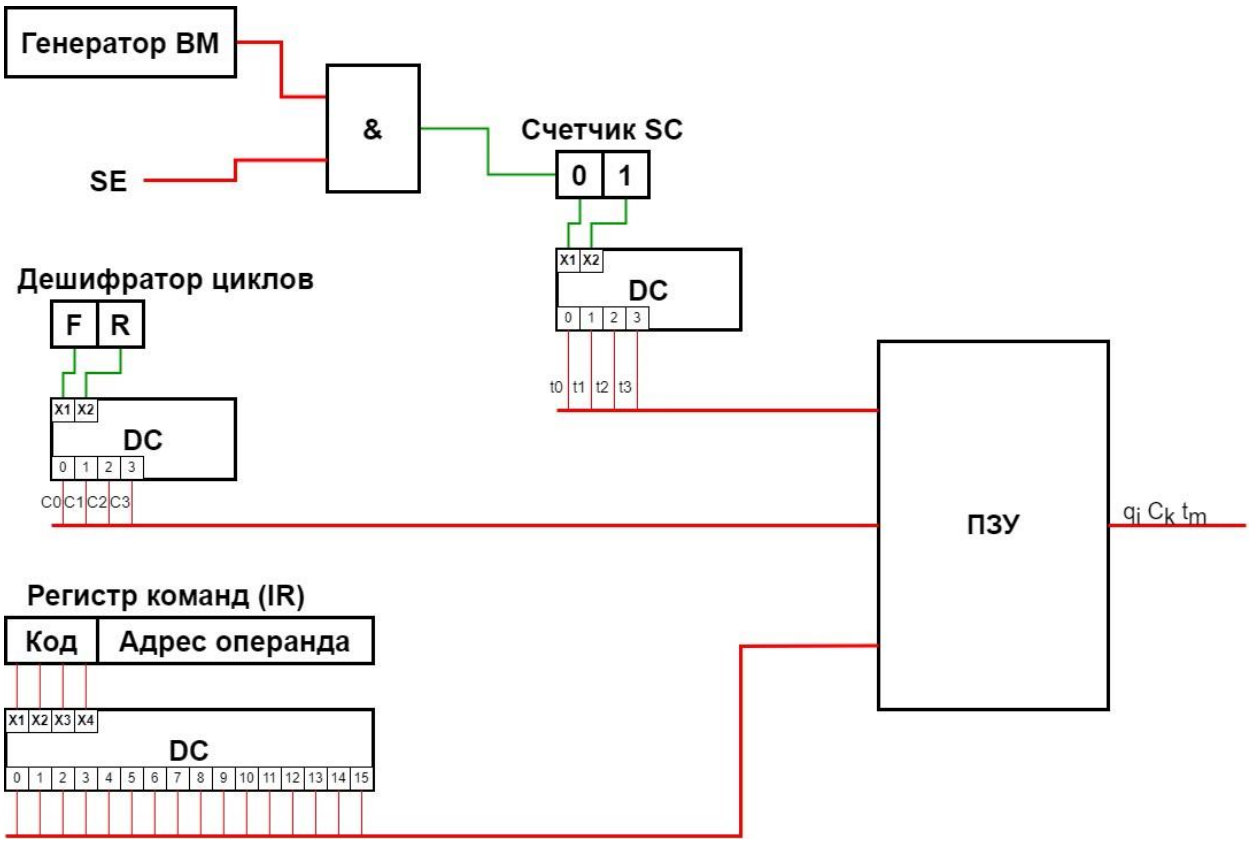


Рисунок 54. Схема устройства управления

В основе УУ имеется постоянное запоминающее устройство (ПЗУ). Сигналы, вырабатываемые на его выходе, управляют работой ОУ учебной ВМ. Генератор ВМ, вырабатывает синхросигнал. Сигнал SE представляет собой сигнал разрешения работы и останова. При SE=0 сигналы генератора ВМ не проходят через схему совпадения, и схема управления не работает — останов. При SE=1 сигналы генератора ВМ проходят через схему совпадения и поступают на вход двухразрядного счетчика, соединённого по выходу с дешифратором тактов, максимально в нашей системе может быть 4 такта. DC команд, работает по состоянию разрядов регистра команд, соответствующих полю кода операции. В нашем случае код имеет 4 разряда, что соответствует 16 выходам дешифратора команд. Составим таблицу команд нашей учебной ВМ (табл. 20).

Таблица 20. Список команд учебной ВМ

№ и обозначение команды	Команда	Код команды		Адрес	Расшифровка команды
		2	16		
0 q_0	LDA	0000	0	AD	AC←M[AD]
1 q_1	STA	0001	1	AD	M[AD] ← AC
2 q_2	ADD	0010	2	AD	AC← AC+ M[AD]
3 q_3	SUB	0011	3	AD	AC← AC- M[AD]
4 q_4	AND	0100	4	AD	AC← AC∧M[AD]
5 q_5	OR	0101	5	AD	AC← AC∨M[AD]
6 q_6	XOR	0110	6	AD	AC← AC$\overline{\oplus}$M[AD]
7 q_7	COM	0111	7	AD	AC← $\overline{M[AD]}$ (обратный код)
8 q_8	NEG	1000	8	AD	AC← $\overline{M[AD]}$ + 1(доп. код)
9 q_9	INC	1001	9	AD	AC←M[AD]+1
10 q_{10}	DEC	1010	A	AD	AC←M[AD]-1
11 q_{11}	JMP	1011	B	AD	PC←AD (переход)
12 q_{12}	CLEA	1100	C	FFF	AC←0
13 q_{13}	INCA	1101	D	FFF	AC← AC+1
14 q_{14}	INP	1110	E	FFF	AC← INPR (ввод)
15 q_{15}	OUT	1111	F	FFF	OUTR ← AC (вывод)

Значение AD в столбце адреса отражает адрес, по которому хранится второй операнд, который требуется для выполнения той или иной команды, сам адрес хранится в регистре адреса памяти, или в ОЗУ. В командах с 12 по 15 для их выполнения второй операнд не требуется, поэтому в качестве адреса указываются три 16-ричных символа F (то есть 12 двоичных единиц).

Наконец, DC циклов работает по выходам двух триггеров F и R. В зависимости от состояния этих триггеров формируется сигнал на соответствующем его выходе. Разрядность дешифратора 2-4, значит в нашей системе может быть 4 цикла: выборка команды (C0), дешифрация команды (C1), выполнение команды (C2) и прерывание (C3). В рамках нашей работы с учебной ВМ рассмотрим только два цикла: выборка и выполнение команды.

Для начала рассмотрим цикл выборки команды, так как прежде чем выполнить команду, надо выбрать её из памяти ВМ (рис. 55). Здесь рассматривается выполнение команды с прямой адресацией, когда в адресном поле команды находится адрес операнда.

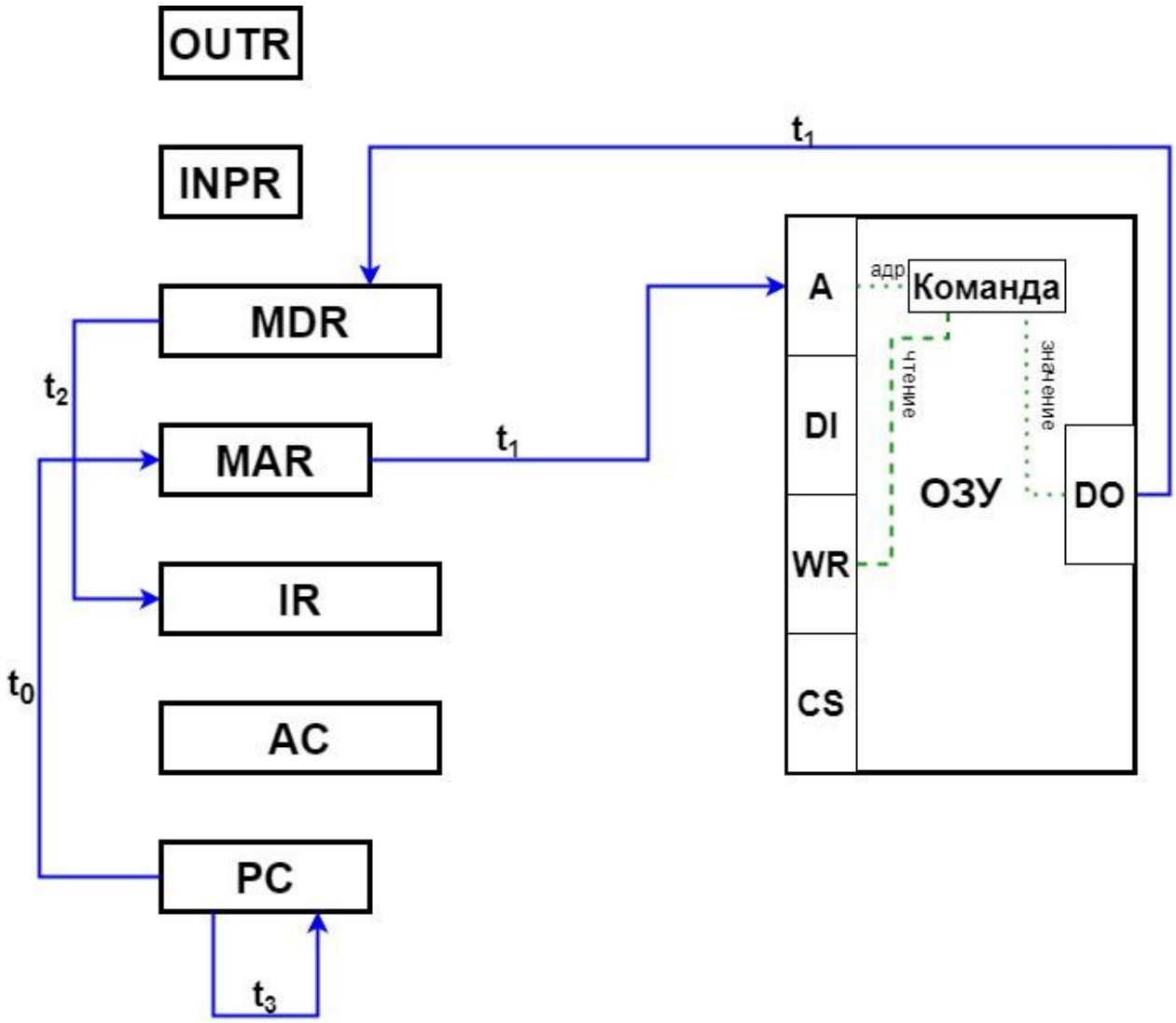


Рисунок 55. Цикл выборки команды из памяти

Для выборки команды из памяти ВМ надо, во-первых, поместить адрес команды из регистра адреса команды в регистр адреса памяти, во-вторых, прочитать команду из ячейки памяти в регистр данных памяти, в-третьих, переслать команду в регистр команды, в-четвертых, подготовить счетчик команд для выборки следующей команды — выполнить инкремент регистра PC.

Цикл дешифрации в ВМ необходим тогда, когда используются несколько методов адресации операндов. В нашей учебной ВМ используется только прямая адресация, когда в поле адреса команды содержится адрес команды или операнда. Поэтому мы не рассматриваем цикл дешифрации. После размещения команды в регистре команд IR она сразу готова к выполнению, причем в 4 старших разрядах регистра IR, в поле кода операции имеется номер команды в ПЗУ.

Рассмотрим цикл выполнения команды на примере операции сложения ADD (0010). Процесс выполнения команды изображен на рис. 56.

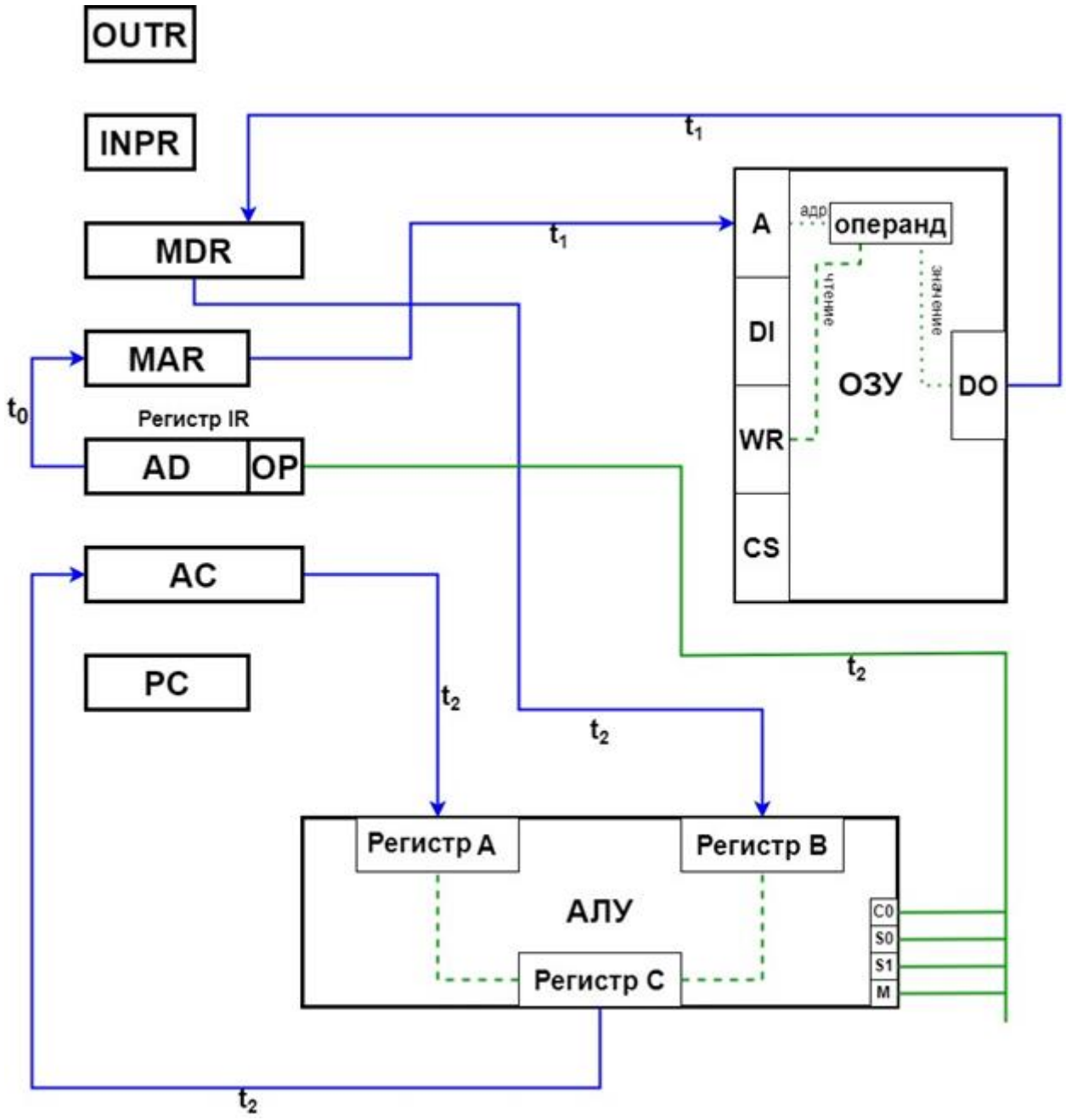


Рисунок 56. Цикл выполнения команды

Нулевой такт — пересылка адреса операнда из регистра команды в регистр адреса памяти. Первый такт — выборка из памяти операнда и размещение его в регистре данных памяти. Второй такт — собственно выполнение сложения в АЛУ с размещением результата в аккумуляторе. Четвертый такт пуст, поэтому состояние триггера F устанавливается в «0», что позволяет перевести ВМ обратно в цикл выборки.

4.2. Управляющие слова для выполнения команды ADD

Представим описанный выше процесс в виде таблицы (табл. 21), добавив к нему микрокоманды, которые составим, исходя из структуры ОУ.

Таблица 21. Микрокоманды выполнения команды ADD (0010)

Команда, цикл, такт (К,Ц,Т)	Микрокоманда
C_0t_0	MAR ← PC
C_0t_1	MDR ← M[MAR]
C_0t_2	IR ← MDR
C_0t_3	PC ←PC+1, F←1
$q_2C_2t_0$	MAR ← IR[AD]
$q_2C_2t_1$	MDR ← M[MAR]
$q_2C_2t_2$	AC ← AC+MDR
$q_2C_2t_3$	F ← 0

Рассмотрим введенные обозначения: C_0 — цикл выборки, C_2 — цикл выполнения, q_2 — обозначение команды, которая подлежит выполнению, в нашем случае команда ADD ее код соответствует 2 (в шестнадцатеричной системе счисления), t_0-t_3 — это такты внутри циклов. На основе составленных микрокоманд, расширим таблицу, добавив в нее коды управляющих слов. Полученная таблица будет выглядеть следующим образом (табл. 22).

Таблица 22. Управляющие слова для выполнения команды ADD (0010)

К,Ц,Т	Микрокоманда	MS (A)	MS (B)	АЛУ	DC	MS (M)	WR	FR
C_0t_0	MAR ← PC	000	xxx	0000	0011	x	0	00
C_0t_1	MDR ← M[MAR]	xxx	xxx	xxxx	0100	1	0	00
C_0t_2	IR ← MDR	100	xxx	0000	0010	x	0	00
C_0t_3	PC ←PC+1, F←1	000	xxx	0001	0000	x	0	10
$q_2C_2t_0$	MAR ← IR[AD]	010	xxx	0000	0011	x	0	10
$q_2C_2t_1$	MDR ← M[MAR]	xxx	xxx	xxxx	0100	1	0	10
$q_2C_2t_2$	AC ← AC+MDR	001	100	0010	0001	x	0	10
$q_2C_2t_3$	F ← 0	xxx	xxx	xxxx	1xxx	x	0	00

Значение «x», установленное в битах отдельных полей, означает, что значение бита данного поля не имеет значения в рамках выполнения конкретной микрокоманды. Новый столбец (FR) отражает какой цикл будет следующим относительно текущего такта. То есть, если текущий такт находится в цикле выборки команды, и в столбце FR установлено значение «00», значит следующий такт также будет находится в цикле выборки команды. Однако, если проанализировать четвертый такт цикла выборки команды, то в столбце установлено значение «10». Четвертый такт в цикле выборки — последний, значит необходимо перейти в новый цикл, по этой причине триггер F переводится в состояние «1», значение

«10» столбца FR означает переход в следующем такте к циклу выполнения команды.

Процесс выполнения команды ADD, описанный в табл. 22, почти полностью совпадает с выполнением команд 0-10, за исключением команд 1 и 8, которые требуют иного подхода в их выполнении. Команды от 11 до 15 — это исключения, для каждой из них требуется только 1 такт в цикле выполнения. Рассмотрим в качестве примера однотактовую команду номер 13 (1101) INCA — инкремент аккумулятора. Расшифровка команды: $AC \leftarrow AC + 1$. Как уже было сказано, команда выполняется в один такт, поскольку в выполнении данной команды отсутствует второй операнд. Благодаря этому, второе обращение к памяти для считывания операнда не требуется, следовательно, выполнение команды, сокращается на 2 такта, а так как оставлять первые два такта цикла выполнения, бездействовать нет смысла, перенесем выполнение команды из 2-ого такта в нулевой, и промежуточный результат будет выглядеть следующим образом (табл. 23).

Таблица 23. Промежуточный этап процесса выполнения команды INCA (1101)

К,Ц,Т	Микрокоманда	MS (A)	MS (B)	АЛУ	DC	MS (M)	WR	FR
C_0t_0	MAR ← PC	000	xxx	0000	0011	x	0	00
C_0t_1	MDR ← M[MAR]	xxx	xxx	xxxx	0100	1	0	00
C_0t_2	IR ← MDR	100	xxx	0000	0010	x	0	00
C_0t_3	PC ← PC+1, F←1	000	xxx	0001	0000	x	0	10
$q_2C_2t_0$	AC ← AC + 1	001	xxx	0001	0001	x	0	10
$q_2C_2t_1$								
$q_2C_2t_2$								
$q_2C_2t_3$	F ← 0	xxx	xxx	xxxx	1xxx	x	0	00

Однако, остальные такты, которые были убраны, нужно чем-то заполнить, для этого есть специальная микрокоманда, не указанная в табл. 16, которая отражает отсутствие действия «NOP» или «**No Op**eration». Дополним табл. 23 микрокомандами NOP, результат перепишем в табл. 24.

Таблица 24. Управляющие слова для выполнения команды INCA (1101)

К,Ц,Т	Микрокоманда	MS (A)	MS (B)	АЛУ	DC	MS (M)	WR	FR
C_0t_0	MAR ← PC	000	xxx	0000	0011	x	0	00
C_0t_1	MDR ← M[MAR]	xxx	xxx	xxxx	0100	1	0	00
C_0t_2	IR ← MDR	100	xxx	0000	0010	x	0	00

Окончание табл. 24

К,Ц,Т	Микрокоманда	MS (A)	MS (B)	АЛУ	DC	MS (M)	WR	FR
C_0t_3	PC ←PC+1, F←1	000	xxx	0001	0000	x	0	10
$q_2C_2t_0$	AC ← AC + 1	001	xxx	0001	0001	x	0	10
$q_2C_2t_1$	NOP	xxx	xxx	xxxx	1xxx	x	0	10
$q_2C_2t_2$	NOP	xxx	xxx	xxxx	1xxx	x	0	10
$q_2C_2t_3$	NOP, F ← 0	xxx	xxx	xxxx	1xxx	x	0	00

Как видно, более раннее завершение цикла — невозможно, поэтому ВМ бездействует 3 такта. По аналогии содержимому табл. 24 можно описать остальные однотактовые команды.

4.3. Изменения в регистрах при выполнении команды ADD

Рассмотрев процессы управления учебной ВМ, и выполнения команд, рассмотрим, как работает ВМ с точки зрения потоков данных в регистрах. За основу возьмем уже рассмотренную команду — ADD. Процесс выполнения остается прежним: цикл выборки, после которого идет цикл выполнения. Состав микрокоманд внутри циклов остается без изменений. Но, в новую таблицу (табл. 25) будем записывать не управляющие слова, а содержимое регистров. Вспомним назначение регистров: PC — счетчик команд, MAR — регистр адреса памяти, IR — регистр команд, MDR — регистр данных памяти, AC — аккумулятор, INPR/OUTR — регистры ввода/вывода.

Таблица 25. Алгоритм микрокоманд для выполнения команды ADD (0010)

К,Ц,Т	Микрокоманда	PC	MAR	IR	MDR	AC	INPR	OUTR
Исходное состояние								
C_0t_0	MAR ← PC							
C_0t_1	MDR ← M[MAR]							
C_0t_2	IR ← MDR							
C_0t_3	PC ←PC+1, F←1							
$q_2C_2t_0$	MAR ← IR[AD]							
$q_2C_2t_1$	MDR ← M[MAR]							
$q_2C_2t_2$	AC ← AC+MDR							
$q_2C_2t_3$	NOP, F ← 0							

Рассмотрим пример типовой задачи. Значения в регистрах будем указывать в 16-ричной системе счисления. Задача формулируется так:

По адресу 5A8 записана команда сложения ADD с адресом F12. По этому адресу записан операнд 731F. В регистре AC находится операнд 8721. Записать управляющие слова для каждого такта циклов выборки и выполнения команды и определить информацию, которая будет иметься в регистрах ВМ PC, MAR, MDR, IR, AC после выполнения данной команды.

В формулировке необходимо выделить основные значения, которые потребуются для записи в табл. 25. Первое значение 5A8 — это адрес, по которому в ОЗУ записана команда, помимо адреса, это значение выполняет функцию счетчика команд, следовательно, это значение будет храниться в счетчике команд (PC). Команда ADD — в таблице команд (табл. 20) имеет номер 2, а адрес F12 — это адрес второго операнда, с которым будет произведена операция сложения, также из расшифровки известно, что первое значение уже хранится в аккумуляторе. Следующее значение — 731F — это непосредственно значение второго операнда, с которым будет выполняться операция сложения. Наконец, последнее значение — 8721 — это значение первого операнда, который уже хранится в аккумуляторе.

Эта информация позволяет занести исходные значения в табл. 25, получим табл. 26.

Таблица 26. Процесс выполнения команды ADD (0010)

К,Ц,Т	Микрокоманда	PC	MAR	IR	MDR	AC	INPR	OUTR
Исходное состояние		5A8				8721		
C_0t_0	MAR ← PC							
C_0t_1	MDR ← M[MAR]							
C_0t_2	IR ← MDR							
C_0t_3	PC ←PC+1, F←1							
$q_2C_2t_0$	MAR ← IR[AD]							
$q_2C_2t_1$	MDR ← M[MAR]							
$q_2C_2t_2$	AC ← AC+MDR							
$q_2C_2t_3$	NOP, F ← 0							

Имея начальные значения, будем следовать заданной последовательности микрокоманд. Нулевой такт в цикле выборки — перенос значения из счетчика команд в регистр адреса. Первый такт — считывание значения из ОЗУ по адресу (5A8), который был передан в предыдущем такте. Значение, которое будет получено, это 2F12, команда ВМ в зашифрованном виде, запишется оно в регистр

MDR, так как только он связан с выходным сигналом ОЗУ. Следующий такт — перенос значения из регистра MDR в регистр команд (IR), для последующей дешифрации. Последний такт в цикле выборки — это инкремент счетчика (PC), что одновременно означает вычисление адреса новой команды в памяти.

Нулевой такт цикла выполнения каманды — это передача адреса операнда в регистр адреса памяти, для чтения. Регистр команд (IR) имеет 16 размер бит, а регистр адреса памяти (MAR) — только 12 бит, следовательно, при передаче значения запишутся только младшие 12 бит, которые как раз отвечают за адрес операнда в памяти, а старшие 4 разряда, которые отвечают за код команды, не запишутся. Первый такт цикла выполнения полностью совпадает с первым тактом цикла выборки — чтение из памяти (ОЗУ). К началу второго такта цикла выполнения, имеется все необходимое для непосредственного выполнения команды: первый операнд в регистре AC, второй операнд в регистре MDR, следовательно, во втором такте происходит выполнение сложения и запись результата в регистр AC. Последний (третий) такт цикла выполнения — не имеет операции, поэтому значения регистров на этом этапе не изменяются. Перенесем результаты описанного алгоритма в итоговую табл. 27.

Таблица 27. Процесс выполнения команды ADD (0010)

К,Ц,Т	Микрокоманда	PC	MAR	IR	MDR	AC	INPR	OUTR
Исходное состояние		5A8				8721		
C_0t_0	MAR ← PC		5A8					
C_0t_1	MDR ← M[MAR]				2F12			
C_0t_2	IR ← MDR			2F12				
C_0t_3	PC ←PC+1, F←1	5A9						
$q_2C_2t_0$	MAR ← IR[AD]		F12					
$q_2C_2t_1$	MDR ← M[MAR]				731F			
$q_2C_2t_2$	AC ← AC+MDR					FA40		
$q_2C_2t_3$	NOP, F ← 0							
Итоговое состояние		5A9	F12	2F12	731F	FA40		

Итоговое состояние регистров после выполнения текущей команды, будет использоваться в качестве исходного состояния для выполнения следующей команды.

4.4. Задания для самостоятельного выполнения

Для выполнения практической работы потребуется выполнить два пункта: первый — записать управляющие слова для заданной команды, второй — записать выполнение команды, расписав движение информации в регистрах.

Вариант №1

1. Записать микрооперации цикла выборки и выполнения команды LDA.

2. По адресу 139 записана команда LDA с адресом B05. По этому адресу команды записан операнд 1729. В регистре-аккумуляторе — операнд 3891.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 2

1. Записать микрооперации цикла выборки и выполнения команды STA.

2. По адресу 650 записана команда STA с адресом 4C2. По этому адресу команды записан операнд B0F5. В регистре-аккумуляторе — операнд BD02.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 3

1. Записать микрооперации цикла выборки и выполнения команды ADD.

2. По адресу 259 записана команда ADD с адресом 50A. По этому адресу команды записан операнд 841F. В регистре-аккумуляторе — операнд 53F3.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 4

1. Записать микрооперации цикла выборки и выполнения команды SUB.

2. По адресу 3B9 записана команда SUB с адресом 420. По этому адресу команды записан операнд 8F22. В регистре-аккумуляторе — операнд FF32.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 5

1. Записать микрооперации цикла выборки и выполнения команды AND.

2. По адресу 301 записана команда AND с адресом 5C2. По этому адресу команды записан операнд ABF1. В регистре-аккумуляторе — операнд 7048.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 6

1. Записать микрооперации цикла выборки и выполнения команды OR.

2. По адресу 601 записана команда OR с адресом 3AD. По этому адресу команды записан операнд C15D. В регистре-аккумуляторе — операнд 2F74.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 7

1. Записать микрооперации цикла выборки и выполнения команды XOR.

2. По адресу 37A записана команда XOR с адресом 40B. По этому адресу команды записан операнд BA12. В регистре-аккумуляторе — операнд CD3A.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 8

1. Записать микрооперации цикла выборки и выполнения команды COM.

2. По адресу 74C записана команда COM с адресом 508. По этому адресу команды записан операнд 52D3. В регистре-аккумуляторе — операнд 2A7E.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 9

1. Записать микрооперации цикла выборки и выполнения команды JMP.

2. По адресу E73 записана команда JMP с адресом 8BF. По этому адресу команды записан операнд E257. В регистре-аккумуляторе — операнд B2A3.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 10

1. Записать микрооперации цикла выборки и выполнения команды DEC.

2. По адресу 2E5 записана команда DEC с адресом 379. По этому адресу команды записан операнд 6A4B. В регистре-аккумуляторе — операнд 56DC.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 11

1. Записать микрооперации цикла выборки и выполнения команды INC.

2. По адресу 785 записана команда INC с адресом D19. По этому адресу команды записан операнд 5246. В регистре-аккумуляторе находится операнд A7E6.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 12

1. Записать микрооперации цикла выборки и выполнения команды NEG.

2. По адресу 38E записана команда NEG с адресом 407. По этому адресу команды записан операнд AD42. В регистре-аккумуляторе — операнд 2B0C.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 13

1. Записать микрооперации цикла выборки и выполнения команды CLEA.

2. По адресу 1F3 записана команда CLEA с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе — операнд F658.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 14

1. Записать микрооперации цикла выборки и выполнения команды INCA.

2. По адресу 2A7 записана команда INCA с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе — операнд F23E.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 15

1. Записать микрооперации цикла выборки и выполнения команды INP.

2. По адресу 83C записана команда INP с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе — операнд 5A4B.Регистры INPR=AB, OUTR=DE.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC, INPR, OUTR после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 16

1. Записать микрооперации цикла выборки и выполнения команды OUT.

2. По адресу DF1 записана команда OUT с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе — операнд AB12. Регистры INPR=BC, OUTR=AE.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC, INPR, OUTR после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант №17

1. Записать микрооперации цикла выборки и выполнения команды LDA.

2. По адресу 27A записана команда LDA с адресом 505. По этому адресу команды записан операнд C34A. В регистре-аккумуляторе — операнд 5541.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 18

1. Записать микрооперации цикла выборки и выполнения команды STA.

2. По адресу 342 записана команда STA с адресом 15E. По этому адресу команды записан операнд 1F15. В регистре-аккумуляторе — операнд CE13.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 19

1. Записать микрооперации цикла выборки и выполнения команды AND.

2. По адресу 34A записана команда ADD с адресом 14B. По этому адресу команды записан операнд 477A. В регистре-аккумуляторе — операнд 226F.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 20

1. Записать микрооперации цикла выборки и выполнения команды SUB.

2. По адресу 12F записана команда SUB с адресом 234. По этому адресу команды записан операнд 2AB5. В регистре-аккумуляторе — операнд FF45.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 21

1. Записать микрооперации цикла выборки и выполнения команды AND.

2. По адресу 25C записана команда AND с адресом 67B. По этому адресу команды записан операнд 377D. В регистре-аккумуляторе — операнд 557A.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 22

1. Записать микрооперации цикла выборки и выполнения команды OR.

2. По адресу 28F записана команда OR с адресом 2CC. По этому адресу команды записан операнд CCFF. В регистре-аккумуляторе — операнд FAC1.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 23

1. Записать микрооперации цикла выборки и выполнения команды XOR.

2. По адресу 45F записана команда XOR с адресом 50C. По этому адресу команды записан операнд AB45. В регистре-аккумуляторе — операнд FF34.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 24

1. Записать микрооперации цикла выборки и выполнения команды COM.

2. По адресу 88A записана команда COM с адресом 609. По этому адресу команды записан операнд 54FF. В регистре-аккумуляторе — операнд 3CCF.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 25

1. Записать микрооперации цикла выборки и выполнения команды JMP.

2. По адресу A17 записана команда JMP с адресом 99B. По этому адресу команды записан операнд CF12. В регистре-аккумуляторе — операнд AB55.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 26

1. Записать микрооперации цикла выборки и выполнения команды DEC.

2. По адресу 33D записана команда DEC с адресом 457. По этому адресу команды записан операнд 7B11. В регистре-аккумуляторе — операнд 543F.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 27

1. Записать микрооперации цикла выборки и выполнения команды INC.

2. По адресу 567 записана команда INC с адресом A12. По этому адресу команды записан операнд B456. В регистре-аккумуляторе — операнд A113.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 28

1. Записать микрооперации цикла выборки и выполнения команды NEG.

2. По адресу 41F записана команда NEG с адресом 609. По этому адресу команды записан операнд CB64. В регистре-аккумуляторе — операнд CCA9.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 29

1. Записать микрооперации цикла выборки и выполнения команды CLEA.

2. По адресу 123 записана команда CLEA с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе — операнд 458C.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 30

1. Записать микрооперации цикла выборки и выполнения команды INCA.

2. По адресу 30F записана команда INCA с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе — операнд AB1F.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 31

1. Записать микрооперации цикла выборки и выполнения команды INP.

2. По адресу 39F записана команда INP с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе — операнд 172F. Регистры INPR=FE, OUTR=F6.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC, INPR, OUTR после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

Вариант № 32

1. Записать микрооперации цикла выборки и выполнения команды OUT.

2. По адресу F33 записана команда OUT с адресом FFF. По этому адресу команды записан операнд FFFF. В регистре-аккумуляторе — операнд CD1E. Регистры INPR=D1, OUTR=7A.

Определить информацию, которая будет записана в регистрах PC, MAR, MDR, IP, AC, INPR, OUTR после выполнения этой команды.

Показать процесс выполнения команды. Записать ответ в строчку.

ПРАКТИЧЕСКАЯ РАБОТА №5

5.1. Описание 8-разрядного процессора типа КР580ВМ80

Микросхема КР580ВМ80А — функционально законченный однокристальный параллельный 8-разрядный микропроцессор с фиксированной системой команд, применяется в качестве центрального процессора в устройствах обработки данных и управления. Микропроцессор имеет раздельные 16-разрядный канал адреса и 8-разрядный канал данных. Канал адреса обеспечивает прямую адресацию внешней памяти объемом до 65536 байт, 256 устройств ввода и 256 устройств вывода [10].

Восьмиразрядное арифметико-логическое устройство микропроцессора обеспечивает выполнение арифметических и логических операций над двоичными данными, представленными в дополнительном коде, а также обработку двоично-десятичных упакованных чисел.

В состав блока регистров входят: 16-разрядный регистр адреса команды (IP), 16-разрядный регистр указателя стека (SP), 16-разрядный регистр временного хранения (WZ), 16-разрядная схема инкремента-декремента и шесть 8-разрядных регистров общего назначения (B, C, D, E, H, L), которые могут использоваться и как три 16-разрядных регистра (BC, DE, HL) [11].

Микропроцессор выполняет команды по машинным циклам. Число циклов, необходимое для выполнения команды, зависит от ее типа и может быть от одного до пяти. Машинные циклы выполняются по машинным тактам. Число тактов в цикле определяется кодом выполняемой команды и может быть от трех до пяти. Длительность такта равна периоду тактовой частоты и при частоте 2,0 МГц составляет 500 нс. Микропроцессор представлен на рис. 57.



Рисунок 57. Микропроцессор КР580ВМ80

5.2. Описание эмулятора CPU580

Эмулятор 8-разрядного процессора реализован на ЭВМ типа IBM — PCAT. При входе в эмулятор открывается окно ПМК КР580ВМ80, в котором изображена структура «стенда». В верхней части окна (рис. 58) указываются режимы работы: «файл», «структурная схема», «система команд», «помощь».

Строкой ниже указывается адрес (состояние шины адреса в шестнадцатеричной системе счисления), данные (состояние шины данных в шестнадцатеричной системе счисления) [12].

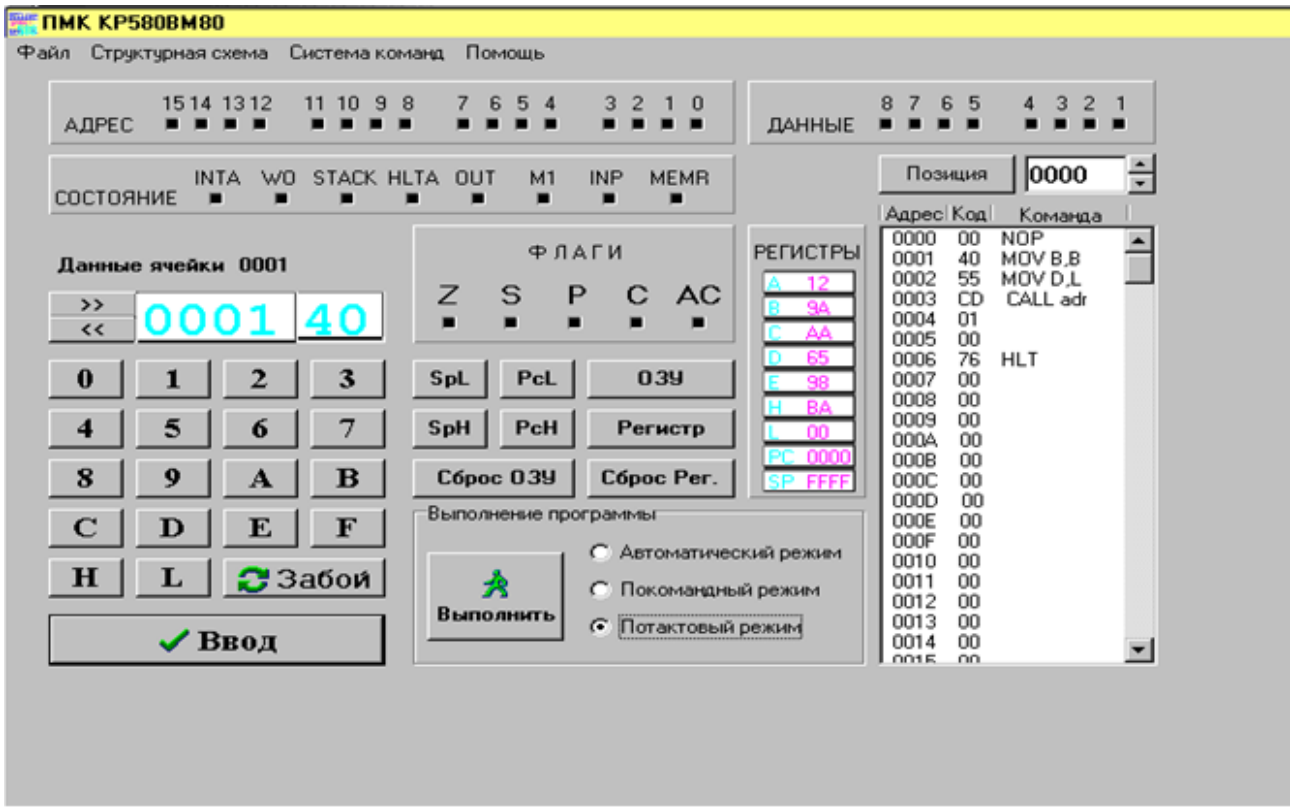


Рисунок 58. Стенд эмулятора микропроцессора КР580ВМ80

Далее показано окно, в котором указывается адрес ОЗУ и его содержимое, или номер регистра и его содержимое. В окне флагов записывается состояние флагов Z, S, P, C, AC на данный момент выполнения команды процессором, где:

- Z — признак нулевого результата;
- S — знак;
- P — признак четного количества единиц результата;
- C — признак переноса из старшего разряда;
- AC — признак дополнительного переноса из старшего разряда младшей тетрады байта.

Для ввода информации используется клавиатура, которая подразделяется на цифровую от 0 до F и режимную SpL, SpH, PcL, PcH, ОЗУ, Сброс ОЗУ, Регистр, Сброс рег.

Для ввода информации ОЗУ необходимо нажать кнопку «ОЗУ», затем кнопку «Ввод». После этого ввести в данную ячейку код команды или данные.

Для ввода информации в регистры требуется нажать клавишу «Регистр», клавишу «Ввод», а затем на цифровой клавиатуре произвести ввод данных.

Клавиши SpL, SpH и PcL, PcH позволяют записывать адреса в указатель стека и счетчик команд. При вводе любой информации необходимо выполнить нажатия «мышкой» на клавишу «ввод».

В первой части «окна» расположены регистры процессора:

– A — аккумулятор;
– B,C,D,E,H,L — общие регистры RON, PC — программный счетчик;
– SP — указатель стека — указывает вершину стека FFFF (перевернутый стек).

В области окна «выполнение программы» задается режим:

– автоматический режим — при нажатии кнопки «автоматический режим» выполняется программа до команды HLT;
– покомандный режим — при нажатии кнопки «покомандный режим» реализуется весь цикл исполнения команды;
– потактный режим — при каждом нажатии кнопки «выполнить» осуществляется выполнение одного такта работы процессора.

В правой части окна экрана расположено «окно», в котором указана «Позиция» адресного пространства ОЗУ, где производится набор программы: адрес команды, КОП, ассемблерный код команды. «Позиция» указывает область адресного пространства ОЗУ, в котором прокруткой «Δ» или «∇» можно указывать начало требуемого адресного пространства.

При указании «мышью» позиции «структурная схема» раскрывается следующее окно, в котором показана структурная схема процессора (рис. 59).

Структурная схема 8-разрядного процессора включает АЛУ с входными регистрами («Аккумулятор», «Буф.Регистр 1», «Буф.Регистр 2»), «Рег.Признаков», «Схема десятичной коррекции»; «Регистр команд», «Деш. Команд», «Блок синхронизации и управления», «Регистры временного хранения W и Z», «регистры общего назначения B, C, D, E, H, L», указатель стека, счетчик команд, схемы инкремента и декремента .

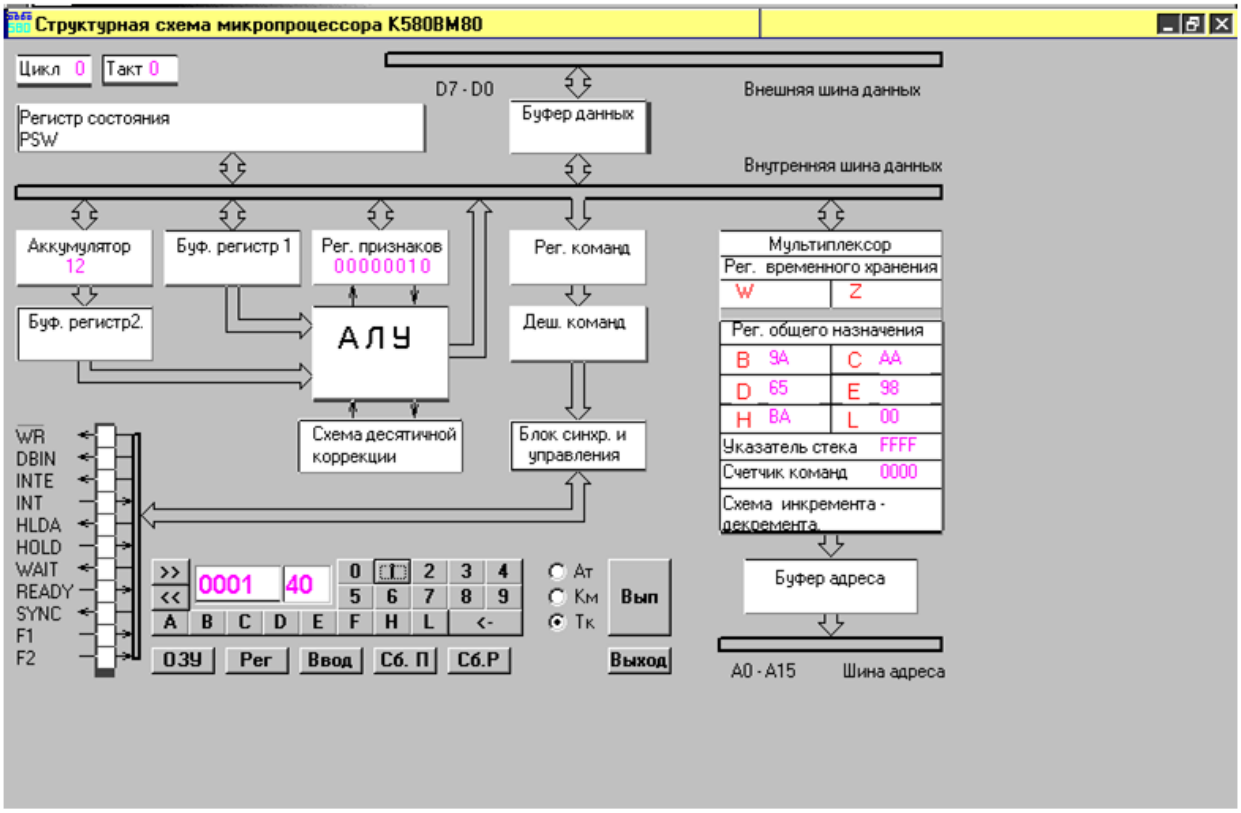


Рисунок 59. Стенд структурной схемы эмулятора микропроцессора КР580ВМ80

Указанные схемы подключены к внешним шинам адреса и данных через «Буферные регистры» адреса и данных.

Для организации режимов работы, как и в предыдущем окне, в данном расположена клавиатура ввода информации, аналогичная клавиатуре предыдущего окна, и соответствующие режимные клавиши, а также клавиши режима работы:

– автоматический режим — АТ;
– покомандный режим — КМ;
– потактный режим. При потактном режиме можно исследовать выполнение любой команды.

При каждом нажатии кнопки «Вып.» реализуется очередной такт работы процессора. При этом фиксируется на экране все состояния его регистров, номер такта, номер цикла, состояния управляющих сигналов, регистр состояния и PSW.

Режим работы с клавиатурой соответствует предыдущему окну. При указании «мышью» в окне «ПМК КР580ВМ80» на позицию «Система команд» раскрывается окно «Команды», в котором предоставлена таблица (рис. 60) системы команд. Код команды определяется по матрице в 16-ричной системе счисления по столбцу и строке таблицы, например, MOV B, C — 41 или HLT — 76.

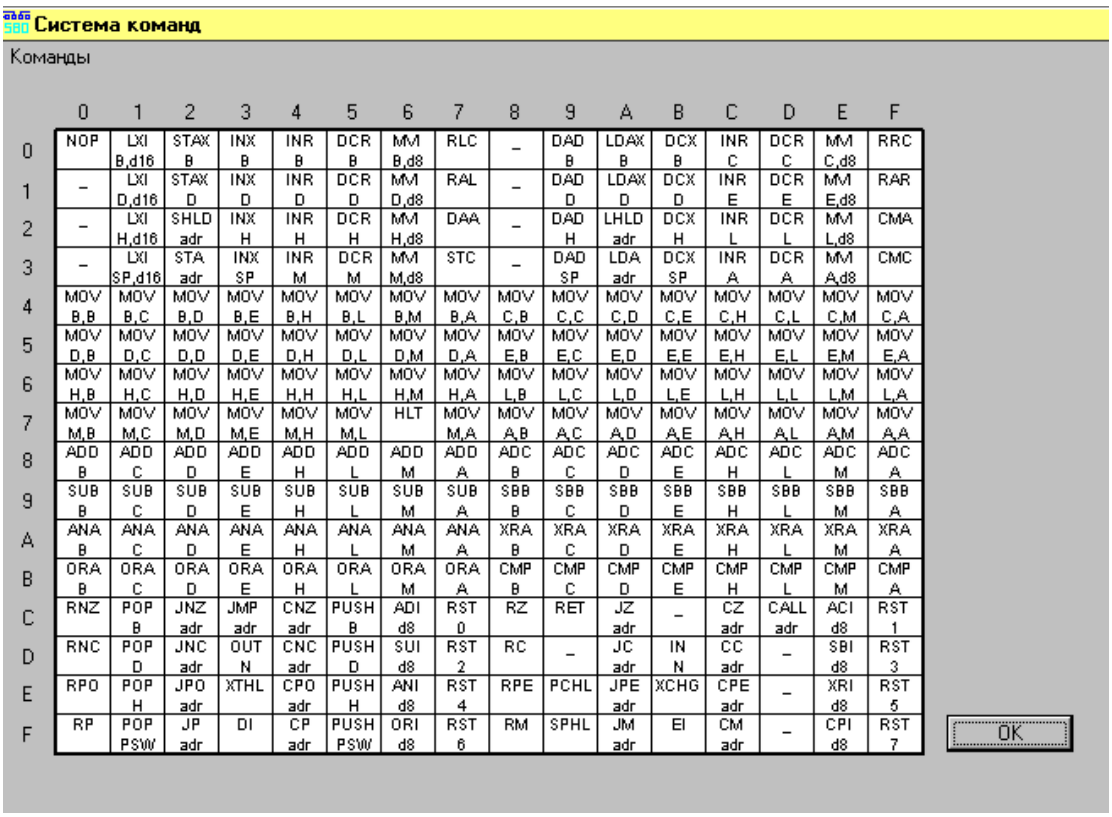


Рисунок 60. Система команд эмулятора микропроцессора КР580ВМ80

Команды процессора представлены следующими форматами: однобайтным, двухбайтным и трехбайтным. За одно обращение к памяти выбирается один байт команды и размещается в регистре команд. Если команда имеет 2 или 3 байта, то второй и третий байты записываются в программно недоступные регистры W и Z, которые называются регистрами временного хранения.

В процессоре применяются четыре следующие способа адресации:

– прямая адресация (относится к командам обращение к памяти, причем во втором и третьем байтах команды записывается адрес ОЗУ);
– регистровая адресация (используется для обращения к внутренним регистрам процессора);
– непосредственная адресация (в поле команды размещается операнд);
– регистровая косвенная адресация (в поле команды размещается адрес регистровой пары, в которой записан адрес ОЗУ); в качестве регистровых пар могут быть использованы пары регистров B-C, D-E, H-L, причем последняя пара H-L — адресует ячейку памяти M, которая называется псевдорегистром.

Система команд процессора включает 5 групп команд:

1) команды пересылок;
2) команды арифметических операций;
3) команды логических операций;

4) команды передачи управления;
5) команды обращения к стеку, ввода-вывода и управление МП-системой.

5.3. Пример решения задачи

Задача: требуется найти сумму натурального ряда первых 50 целочисленных элементов начиная с «1». Решение представлено в табл. 28.

Таблица 28. Пример решения задачи по поиску суммы первых 50 элементов

Ячейка ОЗУ	Код команды	Расшифровка команды	Комментарий
0000	1E	MVI E, d8	загружаем в регистр E число 32_{16}
0001	32		операнд $50_{10} = 32_{16}$
0002	19	DAD D	Сложение конкатенаций пар регистров DE и HL
0003	1D	DCR E	уменьшить содержимое регистра E на 1
0004	C2	JNZ adr	перейти к началу цикла, если содержимое регистра E не 0
0005	02		младший байт адреса перехода
0006	00		старший байт адреса перехода
0007	76	HLT	Остановка

5.4. Задания для самостоятельного выполнения

Постройте сумму натурального ряда первых n значений в соответствии с вариантом, представленным в табл. 29.

Таблица 29. Задания для самостоятельной работы

Вариант	n
1	10
2	20
3	35
4	45
5	60
6	70
7	80
8	85
9	90
10	95
11	100
12	105
13	110
14	115
15	120

ПРАКТИЧЕСКАЯ РАБОТА № 6

6.1. Понятие конвейера

Конвейеризация или конвейерная обработка — это совмещение операций, основанное на разбиении выполняемых функций на более мелкие части, назовем их ступени, и выделении для каждой из них отдельного аппаратурного блока. При конвейерной обработке используется совмещение различных этапов выполнения последовательности команд. Производительность при этом увеличивается благодаря одновременному выполнению нескольких команд на различных ступенях конвейера. Подобная конвейерная обработка широко используется всеми современными быстродействующими процессорами.

Различают синхронные и асинхронные конвейеры, их отличие заключается в признаке использования синхронизации работы ступеней. Традиционные вычислительные машины основаны на применении синхронных конвейеров. Из-за того, что ступени конвейера размещены компактно, то тракты распространения сигналов синхронизации являются крайне короткими и фактор перекоса этих сигналов становится не существенным. По уровням конвейеризации отличают следующие виды конвейеров:

- макроконвейер — конвейеризация на уровне процессоров;
- конвейер команд — конвейеризация команд процессора;
- конвейер арифметический — конвейеризация на уровне выполнения каждой команды процессора.

При изучении процесса конвейерной обработки мы будем использовать архитектуру ВМ, включающую целочисленные регистры общего назначения (R0,...,R31), регистры с плавающей точкой (F0,...,F31) и счетчик команд PC. В системе команд процессора предусмотрены типичные арифметические и логические операции, операции с плавающей точкой, операции передачи данных, операции управления потоком команд и системные операции. Для доступа к памяти используются специальные операции загрузки регистров и записи данных в память. Каждая команда выполняется на пяти этапах:

- выборка команды;
- декодирование команды/извлечение операндов из регистров;
- выполнение операции/вычисление адреса памяти;
- обращение к памяти;
- сохранение результата.

Для конвейеризации выполнения команд каждый этап разбивается на отдельные такты синхронизации, что позволяет начинать выполнение новой команды на каждом такте. Промежуточные результаты хранятся в промежуточных

регистрах или в буферной памяти для передачи данных и управляющих сигналов между этапами конвейера. Общее время выполнения одной команды в конвейере составляет пять тактов, но аппаратура способна одновременно выполнять пять различных команд в каждом такте. Работу конвейера обычно иллюстрируют временными диаграммами, на которых отображаются выполняемые команды, номера тактов и этапы выполнения. Запишем процесс конвейерной обработки 5 команд, состоящих из 5 тактов в табл. 30, как наиболее удобный вид изображения конвейерной обработки.

Таблица 30. Конвейерная обработка

Номер команды	Номер такта								
	1	2	3	4	5	6	7	8	9
Команда 1	T_1	T_2	T_3	T_4	T_5				
Команда 2		T_1	T_2	T_3	T_4	T_5			
Команда 3			T_1	T_2	T_3	T_4	T_5		
Команда 4				T_1	T_2	T_3	T_4	T_5	
Команда 5					T_1	T_2	T_3	T_4	T_5

Теперь представим последовательное выполнение тех же 5 команд. Новая команда не может начаться, пока не завершилась предыдущая команда, следовательно, 5 команд умножаем на 5 тактов — получаем число 25. Двадцать пять тактов необходимо для последовательного выполнения 5 команд, в то время как при конвейерной обработке этот же массив команд обрабатывается за 9 тактов.

Однако, конвейеризация, хоть и увеличивает пропускную способность процессора (количество команд, завершающихся в единицу времени), время выполнения каждой отдельной команды не сокращается. В действительности конвейеризация даже несколько увеличивает время выполнения каждой команды из-за присутствия накладных расходов, связанных с необходимостью промежуточного хранения данных между ступенями конвейера и управления конвейером. Но увеличение пропускной способности означает, что программа будет выполняться быстрее по сравнению с простой неконвейерной схемой.

При работе конвейера команд могут возникнуть три типа конфликтов: структурный, конфликт по данным, конфликт по управлению.

Структурный конфликт связан с обращением различных ступеней конвейера к одному и тому же устройству. Его еще называют конфликтом по ресурсам.

Конфликт по данным заключается в том, что следующая команда может требовать операнд, который еще не вычислен предыдущей командой.

Конфликт по управлению возникает, когда на вход конвейера загружается какая-либо из команд перехода и следующей исполняемой командой должна быть команда, определяемая адресом перехода. Это изменяет последовательность загрузки команд на вход конвейера.

При появлении любого типа конфликта в конвейере появляется «пузырь», который связан с холостой работой некоторых ступеней. Этот «пузырь» проходит по всем ступеням конвейера. В зависимости от типа конфликта «пузырь» может иметь длительность от 1 и более тактов, что приводит к общей потере производительности конвейера.

6.2. RISC-конвейер

Прежде всего, напомним основные особенности процессоров с ограниченным набором команд:

- команды подразделяются на 2 группы; к первой группе отнесены команды обработки данных, ко второй — все остальные команды;
- команды обработки данных имеют одинаковую длину;
- команды первой группы выполняют только короткие операции; команды длинных операций выносятся на программный уровень;
- в командах первой группы используются самые простые способы адресации регистровой памяти (РОН).

Чаще всего используется трехадресный формат (рис. 61).

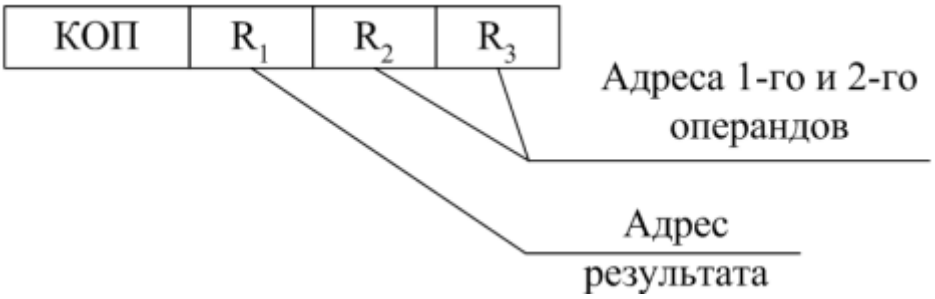


Рисунок 61. Трехадресный формат команды обработки данных

Такой формат позволяет на цикле исполнения команды иметь большую экономию времени по обращению за операндами, поскольку время обращения к регистровой памяти много меньше времени обращения к ОЗУ и соизмеримо с длительностью такта работы процессора.

Быстрые синхронные конвейеры можно строить в случаях, когда команды имеют одинаковую длину, простейшие способы адресации, реализуют только короткие операции и могут выполняться за один и тот же интервал времени.

Ниже представлена временная диаграмма такого конвейера (рис. 62).

За ступенями конвейера закреплены следующие функции:

– 1-я ступень выбирает команды из ОЗУ (или из КЭШ-памяти);
– 2-я — дешифрирует код операции;
– 3-я — осуществляет одновременное обращение за операндами к РОН;
– 4-я ступень выполняет процедуры, указанные в КОП;
– 5-я — записывает результат в РОН.

Функции ступени	Т / ступ	1	2	3	4	5	
Выбор команды	1	$К_1$	$К_2$	$К_3$	$К_4$	$К_5$	Обращение к ОЗУ
ДС КОП	2		$К_1$	$К_2$	$К_3$	$К_4$	
Выбор 1-го и 2-го операнда	3			$К_1$	$К_2$	$К_3$	Обращение к РФ
Выполнение операции	4				$К_1$	$К_2$	Выполнение опер. АЛУ
Запись результата	5					$К_1$	Обращение к РФ

Рисунок 62. Временная диаграмма работы конвейера

6.3. Описание эмулятора RISC-конвейера

В состав эмулятора (рис. 63) входят:

– ОЗУ — оперативное запоминающее устройство;
– СЧАК — счетчик адреса команд;
– РК — регистр команд;
– DC — дешифратор;
– РОН — блок регистров общего назначения;
– АЛУ — арифметико-логическое устройство;
– Р1, Р2 — буферные регистры АЛУ;
– РР — регистр результата АЛУ;
– КЭШ — кэш предсказания переходов.

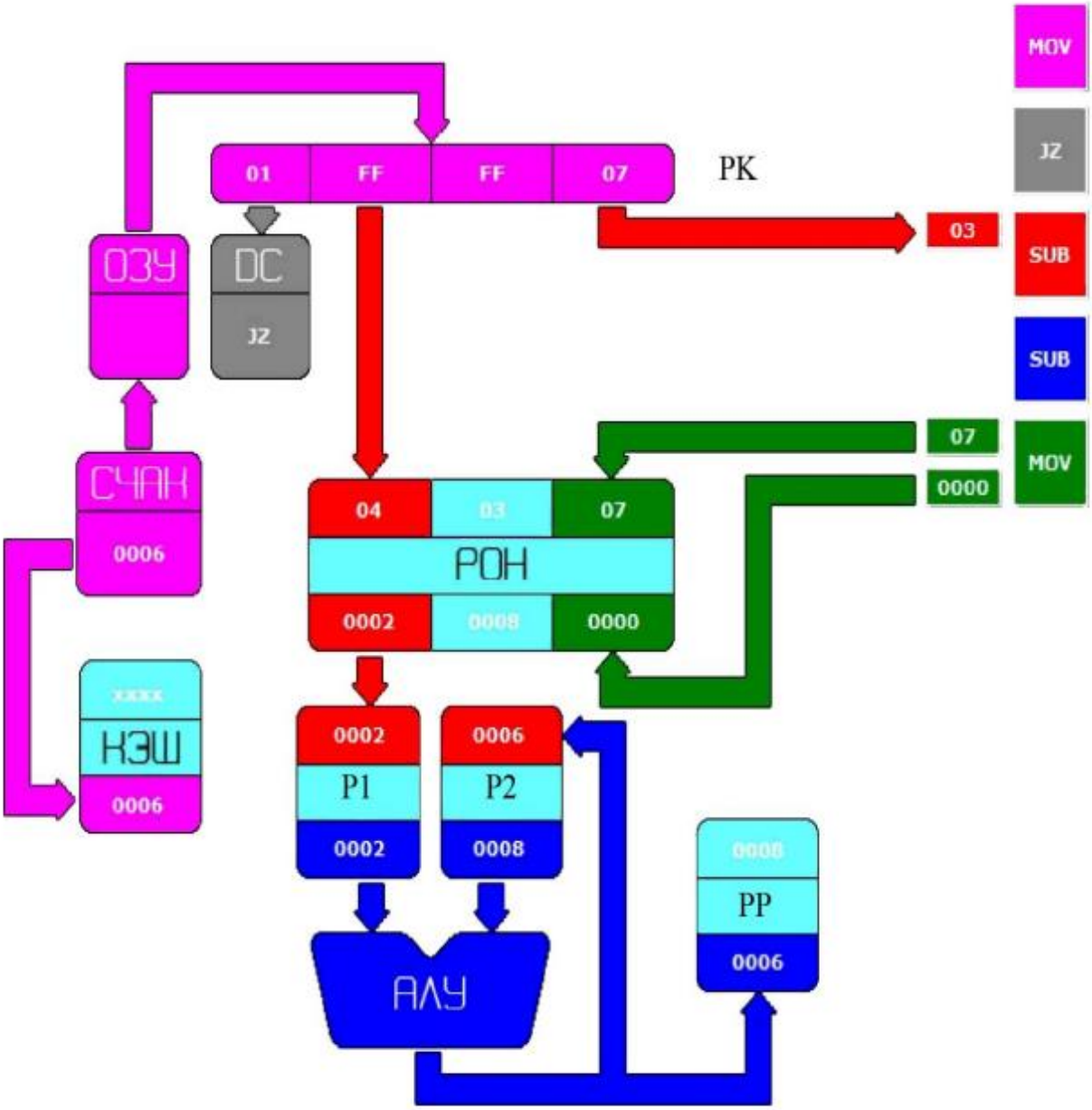


Рисунок 63. Состав эмулятора RISC-процессора

Эмулятор реализует следующий набор команд:

– команды загрузки в РОН операнда из адресного поля (MOV);
– команды арифметических и логических операций (ADD, SUB, INCR, DECR, OR, AND);
– команды безусловного и условных переходов (JUMP, JZ, JC, JP);
– пустую команду (NOP).

Интерфейс пользователя (рис. 64) состоит из левой части окна, где показано содержимое ОЗУ. Команды, находящиеся на ступенях конвейера, окрашиваются

в соответствующий цвет в окне ОЗУ. В правой части окна показывается содержимое РОН и КЭШ предсказания переходов. Для удобства восприятия отличные от нуля значения регистров блока РОН подсвечиваются полужирным шрифтом.

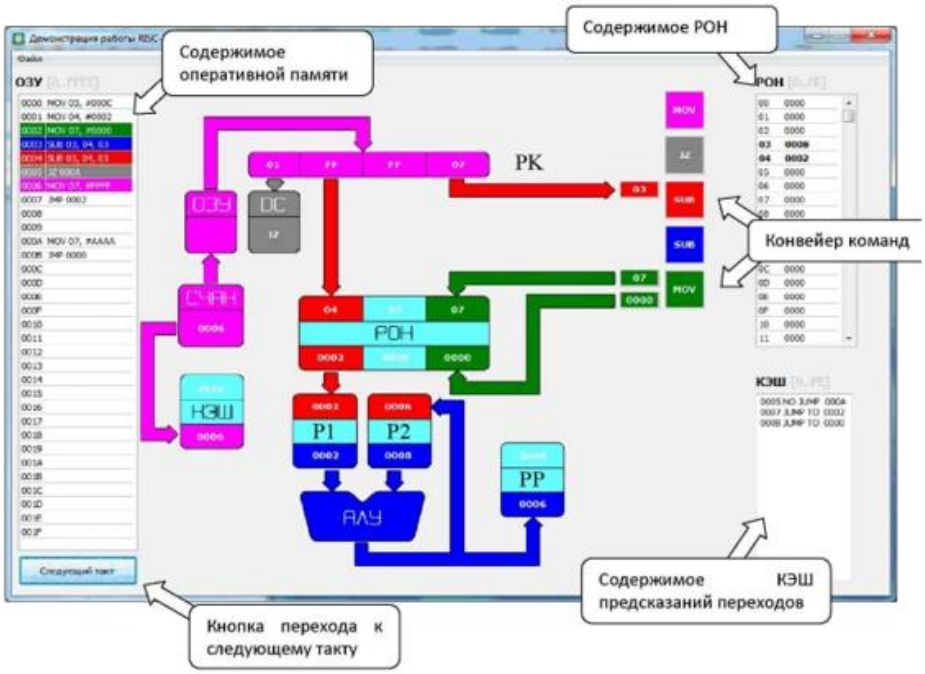


Рисунок 64. Главное окно эмулятора

Для предоставления пользователю возможности редактировать имеющиеся примеры программ и создавать свои собственные программы, предназначено отдельное «окно» (рис. 65).

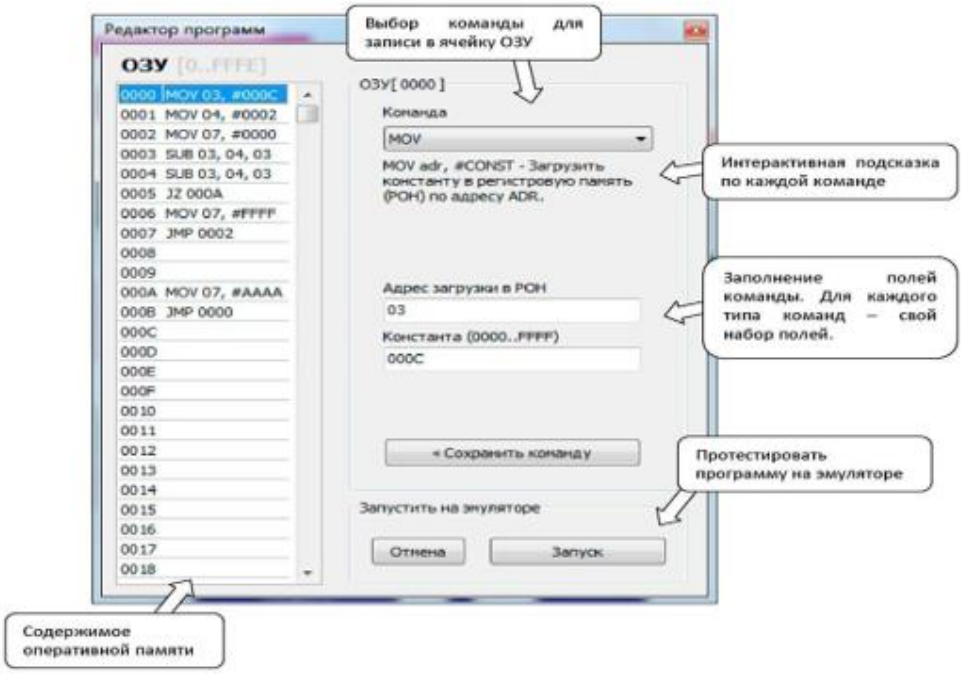


Рисунок 65. Редактирование программы

6.4. Примеры конфликтов в эмуляторе RISC-конвейера

В программе эмулятора представлены 2 примера программ, где происходят конфликты. Рассмотрим подробно каждый из них. Первый из примеров представлен в табл. 31, другой в табл. 32 и на рис. 66 и 67, соответственно.

Таблица 31. Пример 1 задачи

	Выбор команды	DC КОП	Выбор операндов	Выполнение операции	Результат
1	MOV(03) 0011				
2	MOV(07) 0022	MOV(03)			
3	ADD	MOV(07)	MOV(03)		
4	SUB(09)	ADD	MOV(07)	MOV(03)	
5	SUB(09)	ADD		MOV(07)	MOV(03)
6	SUB(09)	ADD			MOV(07)
7	SUB(0A)	SUB(09)	ADD		
8	JMP	SUB(0A)	SUB(09)	ADD	
9	MOV(07) FFFF	JMP	SUB(0A)	SUB(09)	ADD
10		MOV(07) FFFF	JMP	SUB(0A)	SUB(09)
11			MOV(07) FFFF	JMP	SUB(0A)
12				MOV(07) FFFF	JMP
13					MOV(07) FFFF

Конфликт по данным возникает на 5 такте, так как команда ADD запрашивает значения из РОН по адресу 3 и 7, однако на тот момент их там нет.

Таблица 32. Пример 2 задачи

	Выбор команды	DC КОП	Выбор операндов	Выполнение операции	Результат
1	MOV(03)				
2	MOV(04)	MOV(03)			
3	MOV(07)	MOV(04)	MOV(03)		
4	SUB	MOV(07)	MOV(04)	MOV(03)	
5	SUB	SUB	MOV(07)	MOV(04)	MOV(03)
6	SUB	SUB		MOV(07)	MOV(04)
7	JC	SUB	SUB		MOV(07)
8	MOV(07) AAAA	JC	SUB	SUB	
9	JMP 0000	MOV(07) AAAA	JC	SUB	SUB
10				JC (0005) no	SUB

Окончание табл. 32

	Выбор команды	DC КОП	Выбор операндов	Выполнение операции	Результат
11	MOV(07) FFFF				JC
12	JMP (0002)	MOV(07) FFFF			
13	MOV(07) 0000	JMP	MOV(07) FFFF		
14	SUB	MOV(07) 0000	JMP	MOV(07) FFFF	
15	SUB	SUB	MOV(07) 0000	JMP	MOV(07) FFFF
16	JC	SUB	SUB	MOV(07) 0000	JMP
17	MOV(07) FFFF	JC	SUB	SUB	MOV(07) 0000
18	JMP (0002)	MOV(07) FFFF	JC	SUB	SUB

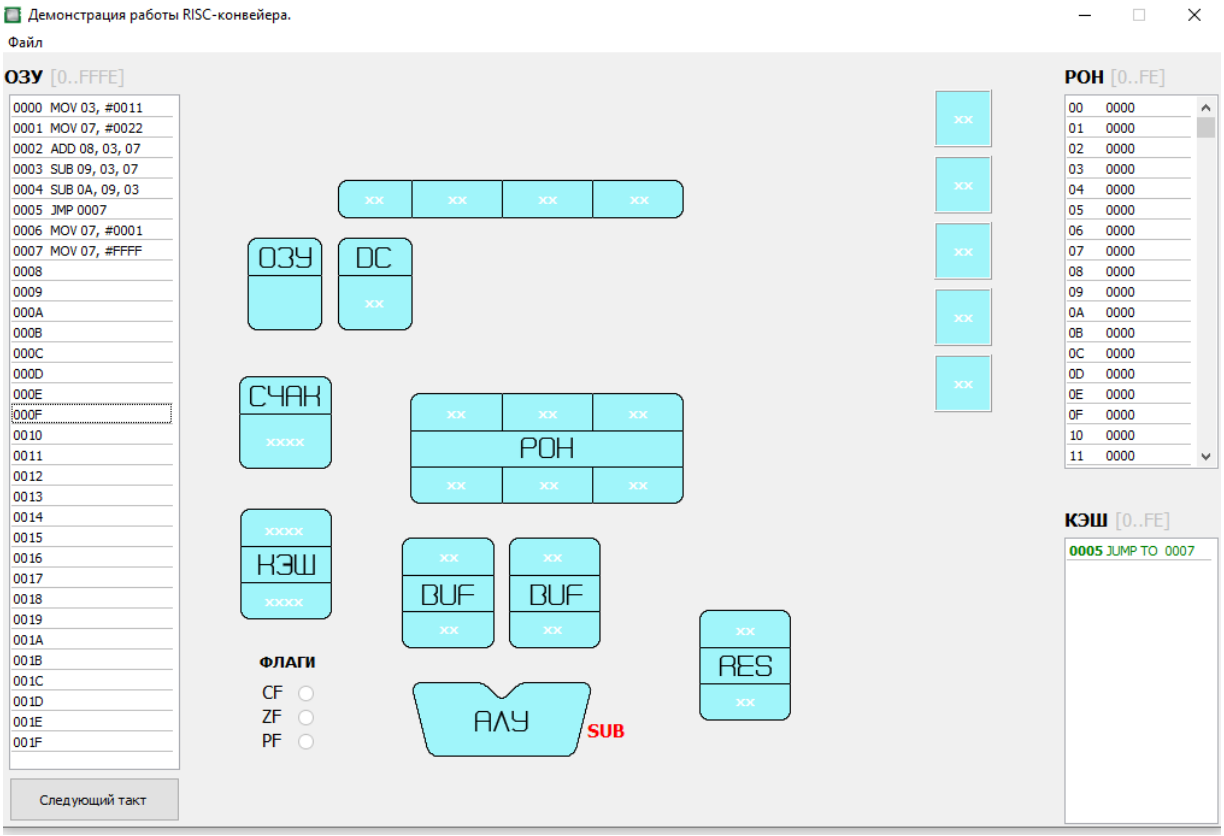


Рисунок 66. Пример 1 задачи

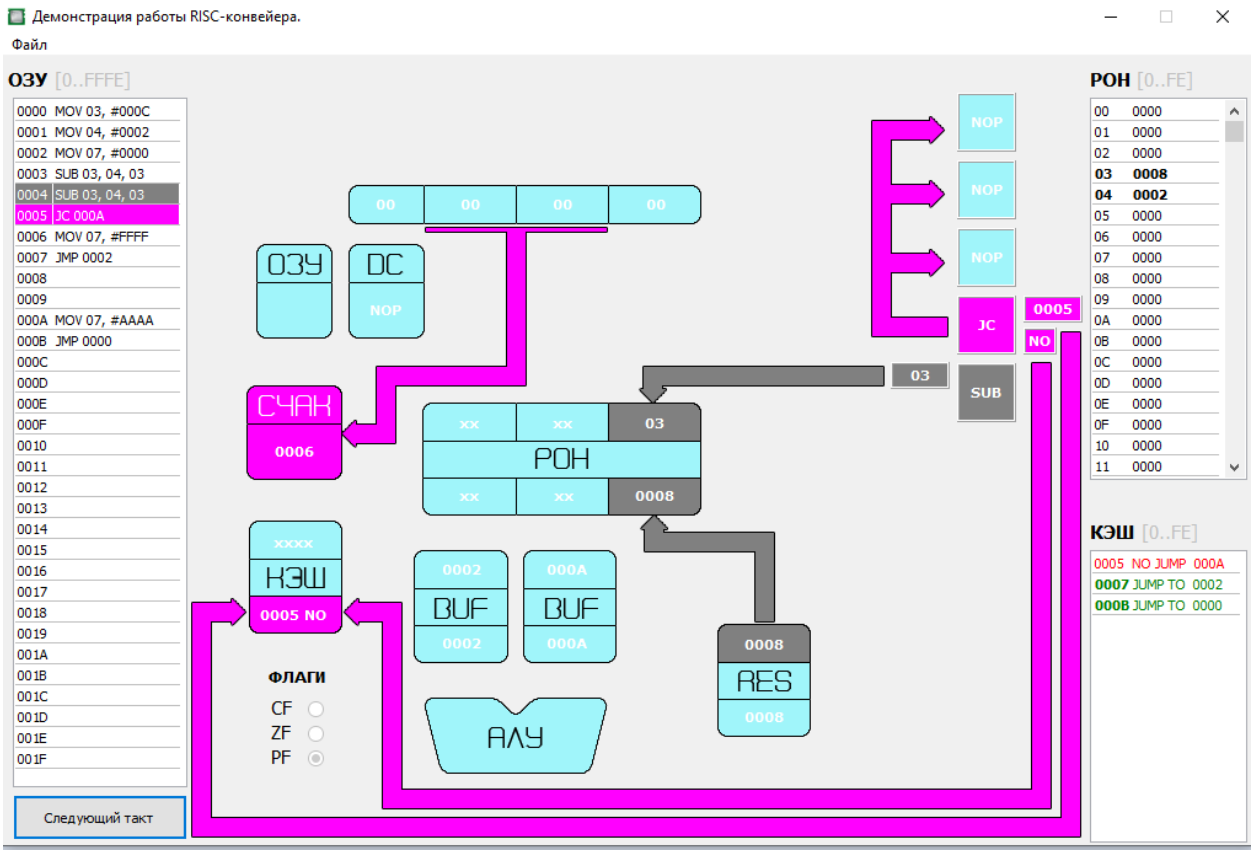


Рисунок 67. Пример 2 задачи

6.5. Варианты для самостоятельной работы

Разработать программу для организации инкремента содержимого регистра РОН от 0 до n. Пояснить типы возникающих конфликтов. Варианты задач представлены в табл. 33.

Таблица 33. Варианты для самостоятельной работы

№ варианта	1	2	3	4	5	6	7	8	9	10	11	12
n	2	3	4	5	6	7	8	9	10	11	12	13

ПРАКТИЧЕСКАЯ РАБОТА № 7

7.1. Параллельные вычисления

Современные требования к повышению производительности вычислительных машин и систем требуют использования параллельных вычислений как в рамках одной машины, так и путем создания многопроцессорных систем и сетей, объединяющих большое количество отдельных процессоров или вычислительных устройств. Для данного подхода рекомендуется использовать термин «вычислительная система» вместо «вычислительная машина». Основной особенностью вычислительных систем является возможность параллельной обработки благодаря введению параллельных ветвей в вычислениях, что не предусмотрено классической архитектурой вычислительных машин.

Из-за специфики параллельных вычислений используются специальные метрики для оценки их эффективности. Наиболее распространенными из них являются:

- профиль параллелизма программы;
- индекс параллелизма;
- ускорение;
- эффективность;
- утилизация;
- избыточность;
- сжатие;
- качество.

Для выведения соотношений, определяющих значение этих метрик, используются следующие характеристики:

- n — количество процессоров в вычислительной системе;
- O(n) — общее количество операций, выполненных на n-процессорной системе;
- T(n) — время выполнения O(n) операций на n-процессорной системе в виде числа временных квантов.

Принимая во внимание дискретность времени и выполнение операций процессором за один временной квант, можно установить следующие соотношения для времени и объема вычислений: $T(1) = O(1)$, $T(n) \le O(n)$, $T(1) \le T(n) \cdot n$. Второе соотношение показывает, что время вычислений можно сократить за счет распределения объема вычислений по нескольким процессорам.

Важно отметить, что объем вычислений также увеличивается при увеличении числа используемых процессоров из-за дополнительных операций, необходимых для параллельного выполнения программы. Кроме того, не весь объем

программы может быть выполнен параллельно, что ограничивает возможность сокращения времени выполнения за счет увеличения числа процессоров. Эффективность сокращения времени можно оценить по нескольким метрикам.

7.2. Метрики эффективности параллельных вычислений

Профиль параллелизма программы определяется функцией p(t), которая указывает количество процессоров, участвующих в выполнении программы в каждый момент времени t. Изменения в загрузке процессоров зависят от различных факторов. Типичный профиль параллелизма для алгоритма декомпозиции показан на рис. 68. В дальнейшем мы будем исходить из предположения, что система состоит из n гомогенных процессоров, максимальный параллелизм в профиле равен τ и стремится к n. Производительность одиночного процессора системы выражается в единицах скорости вычислений и не учитывает издержек, связанных с обращением к памяти и передачей данных. Если i процессоров загружены за определенный период времени, то p=i.

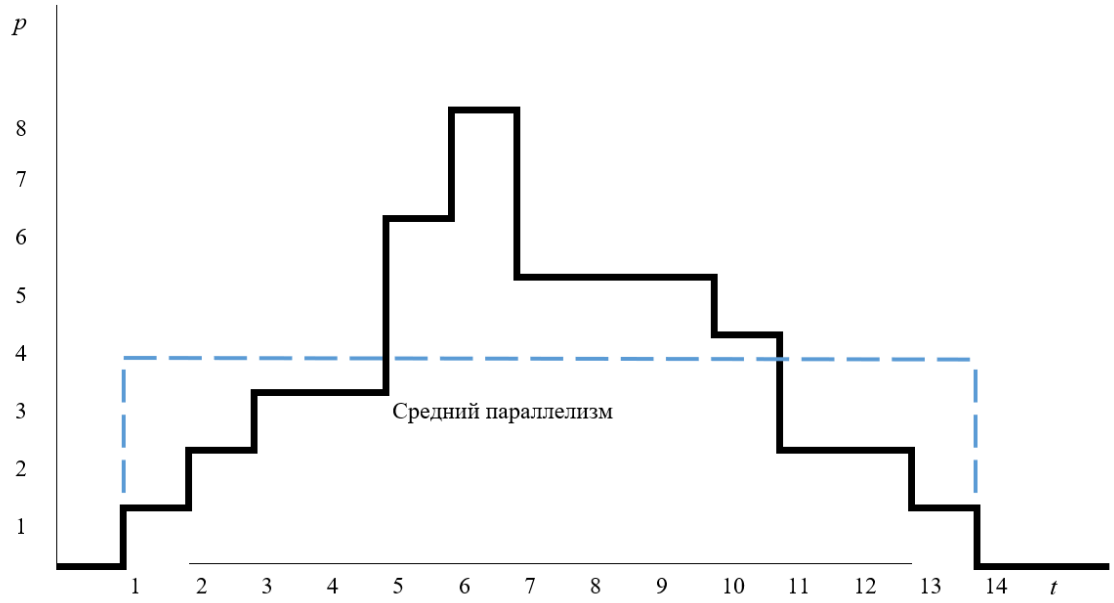


Рисунок 68. Профиль параллелизма

Общий объем вычислений O(n) за период от первого кванта времени t_1 до последнего кванта времени $t_к$ пропорционален площади под кривой профиля параллелизма:

$$O(n) = \sum_{i=1}^{k} p_i, \qquad (6)$$

где p_i — дискретное представление степени параллелизма p(t), показывающее количество задействованных процессоров для каждого кванта времени;

k — номер последнего кванта времени выполнения программы.

Средний параллелизм определяется как O(n)/k.

К метрикам характеризую щим скорость вычислений, относятся **индекс параллелизма** PI(n) и **ускорение** S(n).

Индекс параллелизма характеризует среднюю скорость параллельных вычислений через количество выполненных операций:

$$PI(n) = O(n)/T(n). \quad (7)$$

Ускорение за счет параллельного выполнения программы служит показателем эффективной скорости вычислений. Ускорение определяется как отношение времени, затрачиваемого на проведение вычислений на однопроцессорной машине (в варианте наилучшего последовательного алгоритма), к времени решения той же задачи на параллельной n-процессорной ВС (при использовании наилучшего параллельного алгоритма):

$$S(n) = T(1)/T(n). \quad (8)$$

Примечания по поводу алгоритмов отражают тот факт, что время выполнения вычислений существенно зависит от алгоритмов, реализуемых программой, и ускорение вычисляется при условии реализации наилучших алгоритмов.

К метрикам, характеризующим эффективность вычислений, относятся **эффективность** E(n) и **утилизация** U(n).

Эффективность характеризует целесообразность наращивания числа процессоров через ту долю ускорения, достигнутого за счет параллельных вычислений, которая приходится на один процессор:

$$E(n) = \frac{S(n)}{n} = \frac{T(1)}{n \cdot T(n)}. \quad (9)$$

Утилизация учитывает вклад каждого процессора при параллельных вычислениях, но в виде количества операций, приходящихся на один процессор в единицу времени:

$$U(n) = \frac{O(n)}{n \cdot T(n)}. \quad (10)$$

К метрикам, характеризующим эффективность вычислений по объему, относятся **избыточность** R(n) и **сжатие** C(n). Данные метрики сравнивают объем вычислений при параллельном и последовательном объеме программ и обратны друг другу.

Избыточность — это отношение объема параллельных вычислений к объему эквивалентных последовательных вычислений:

$$R(n) = \frac{O(n)}{O(1)}. \quad (11)$$

Данная метрика основана на абсолютных показателях, базирующихся на объеме вычислительной работы. Избыточность характеризует степень соответствия между программным и аппаратным параллелизмом. Избыточность больше

1, так как при распараллеливании необходимо выполнить дополнительный объем операций.

Кстати, поскольку T(1)=O(1) и с учетом выражений (9-11), утилизация может быть определена через метрики избыточности и эффективности:

$$U(n) = R(n) \cdot E(n).$$

Сжатие вычисляется как величина, обратная избыточности:

$$C(n) = R(n)^{-1} = \frac{O(1)}{O(n)}. \quad (12)$$

Метрика качества Q(n). Эта метрика увязывает между собой метрики ускорения, эффективности и сжатия, является более объективным показателем улучшения производительности за счет параллельных вычислений:

$$Q(n) = \frac{T(1)^3}{n \cdot T(n)^2 \cdot O(n)} = S(n) \cdot E(n) \cdot C(n). \quad (13)$$

Поскольку эффективность E(n) и сжатие C(n) меньше 1, то Q(n) ≤ S(n). То есть качество Q(n) при любых условиях ограничено сверху величиной ускорения S(n).

7.3. Закономерности параллельных вычислений

В идеальном случае, если бы система состояла из n процессоров, вычисления могли бы быть ускорены в n раз. Однако в реальности такое идеальное ускорение часто не достигается по ряду причин. Основная из них заключается в том, что невозможно полностью распараллелить ни одну из задач. Обычно в каждой программе есть участок кода, который должен выполняться последовательно только одним процессором, например, часть программы, отвечающая за запуск задачи или операции ввода/вывода.

Известны также проблемы с теми частями задачи, которые поддаются параллелизации. Идеальным сценарием было бы, если бы параллельные ветви программы постоянно загружали все процессоры системы равномерно. Но на практике оба эти условия сложно реализуемы. Поэтому при ориентации на параллельные вычисления важно понимать, что пропорционального увеличения производительности с увеличением числа процессоров достичь не всегда возможно, и встает вопрос о достижимом реальном ускорении.

Реальное ускорение зависит от способа использования вычислительных ресурсов при увеличении числа процессоров. Существуют три типичных сценария:

- если цель — сократить время вычислений без изменения объема вычислений, то ускорение определяется законом Амдала;
- если время вычислений остается неизменным при расширении системы, но увеличивается объем задачи, то применяется закон Густафсона;

– в случае, когда объем задачи ограничен доступной памятью, ускорение может быть определено по закону Сана-Ная.

7.3.1. Закон Амдала

Джин Амдал (Gene Amdahl) — один из разработчиков всемирно известной системы IBM 360, в своей работе, опубликованной в 1967 году, предложил формулу, отражающую зависимость ускорения вычислений, достигаемого на многопроцессорной вычислительной системе, от числа процессоров и соотношения между последовательной и параллельной частями программы. Показателем сокращения времени вычислений служит такая метрика, как «ускорение», определяемое по формуле (8).

Оговорки относительно алгоритмов решения задачи сделаны, чтобы подчеркнуть тот факт, что для последовательного и параллельного решения лучшими могут оказаться разные реализации, а при оценке ускорения необходимо исходить именно из наилучших алгоритмов.

Проблема рассматривалась Амдалом в следующей постановке. Прежде всего, объем решаемой задачи с изменением числа процессоров, участвующих в ее решении, остается неизменным. Программный код решаемой задачи состоит из двух частей: последовательной и распараллеливаемой. Обозначим долю операций, которые должны выполняться последовательно одним из процессоров, через f, где $0<f<1$ (здесь доля понимается не по числу строк кода, а по числу реально выполняемых операций). Отсюда доля, приходящаяся на распараллеливаемую часть программы, составит $1–f$. Крайние случаи в значениях f соответствуют полностью параллельным ($f=0$) и полностью последовательным ($f=1$) программам. Распараллеливаемая часть программы равномерно распределяется по всем процессорам.

С учетом приведенной формулировки имеем:

$$T(n) = f \cdot T(1) + \frac{(1-f)\cdot T(1)}{n}. \quad (14)$$

Подставив (7.9) в выражение для ускорения (8) и проведя небольшие преобразования, получаем следующую расчетную формулу:

$$\mathrm{S_Amd}(n) = \frac{n}{1+(n-1)\cdot f}. \quad (15)$$

Формула выражает простую и обладающую большой общностью зависимость. Если устремить число процессоров n к бесконечности, то в пределе получаем:

$$\lim_{n\to\infty} S(n) = \frac{1}{f}. \quad (16)$$

Это означает, что если в программе 10% последовательных операций (то есть f=0,1), то, сколько бы процессоров ни использовалось, ускорения работы программы более чем в десять раз никак ни получить, да и то 10 — это теоретическая верхняя оценка лучшего случая, когда никаких других отрицательных факторов нет.

Следует отметить, что распараллеливание ведет к определенным издержкам, которых нет при последовательном выполнении программы. К ним, например, относятся:

Программные издержки. Даже если последовательные и параллельные алгоритмы выполняют одни и те же вычисления, параллельным алгоритмам присущи добавочные программные издержки — дополнительные индексные вычисления, неизбежно возникающие из-за декомпозиции данных и распределения их по процессорам; различные виды учетных операций, требуемые в параллельных алгоритмах, но отсутствующие в последовательных алгоритмах.

Издержки из-за дисбаланса загрузки процессоров. Между точками синхронизации каждый из процессоров должен быть загружен одинаковым объемом работы, иначе часть процессоров будет ожидать, пока остальные завершат свои операции. Эта ситуация известна как дисбаланс загрузки. Таким образом, ускорение ограничивается наиболее загруженным из процессоров.

Коммуникационные издержки. Если принять, что обмен информацией и вычисления могут перекрываться, то любые коммуникации между процессорами снижают ускорение. В плане коммуникационных затрат важен уровень гранулярности, определяющий объем вычислительной работы, выполняемой между коммуникационными фазами алгоритма. Для уменьшения коммуникационных издержек выгоднее, чтобы вычислительные гранулы были достаточно крупными, и доля коммуникаций была меньше.

7.3.2. Закон Густафсона

Решая на вычислительной системе из 1024 процессоров три больших задачи, для которых доля последовательного кода f лежала в пределах от 0,4 до 0,8%, Джон Густафсон из NASA Ames Research получил значения ускорения по сравнению с однопроцессорным вариантом, равные соответственно 1021,1020 и 1016. Согласно закону Амдала для данного числа процессоров и диапазона f, ускорение не должно было превысить величины порядка 201.

Пытаясь объяснить это явление, Густафсон пришел к выводу, что причина кроется в исходной предпосылке, лежащей в основе закона Амдала: увеличение числа процессоров не сопровождается увеличением объема решаемой задачи.

Реальное же поведение пользователей существенно отличается от такого представления.

Обычно, получая в свое распоряжение более мощную систему, пользователь не стремится сократить время вычислений, а, сохраняя его практически неизменным, старается пропорционально увеличению мощности ВС увеличить объем решаемой задачи. И тут оказывается, что наращивание общего объема программы касается главным образом распараллеливаемой части программы. Это ведет к сокращению значения f. Примером может служить решение дифференциального уравнения в частных производных. Если доля последовательного кода составляет 10% для 1000 узловых точек, то для 100 000 точек доля последовательного кода снизится до 0,1%. Фактически, оставаясь практически неизменной, последовательная часть в общем объеме увеличенной программы имеет уже меньший удельный вес.

Было отмечено, что в первом приближении объем работы, которая может быть произведена параллельно, возрастает линейно с ростом числа процессоров в системе. Для того чтобы оценить возможность ускорения вычислений, когда объем последних увеличивается с ростом количества процессоров в системе (при постоянстве общего времени вычислений), Густафсон рекомендует использовать выражение, предложенное Е. Барсисом (E. Barsis):

$$\mathrm{S_Gus}(n) = n + (1 - n) \cdot f. \qquad (17)$$

Данное выражение известно как закон масштабируемого ускорения или закон Густафсона (иногда его называют также законом Густафсона-Барсиса). В заключение отметим, что закон Густафсона не противоречит закону Амдала. Различие состоит лишь в форме утилизации дополнительной мощности ВС, возникающей при увеличении числа процессоров.

7.3.3. Закон Сана-Ная

В многопроцессорной параллельной ВС каждый процессор обычно имеет независимую локальную память сравнительно небольшой емкости. Общая память ВС образуется объединением локальной памяти каждого процессора ВС. При решении задача разделяется на подзадачи и распределяется по множеству процессоров. Подзадача размещается в локальной памяти процессора. Как и в постановке Густафсона, увеличение числа процессоров сопровождается возрастанием размера решаемой задачи, но до предела, обусловленного объемом доступной памяти. Иными словами, объем задачи увеличивается так, чтобы каждая подзадача полностью занимала локальную память процессора. Такая постановка задачи лежит в основе закона, сформулированного Ксианом Хе Саном и Лайонелом Наем и носит название закона ускорения, ограниченного памятью.

Если M — емкость локальной памяти одного процессора, то суммарная память n процессоров будет составлять $n \cdot M$. В формулировке проблемы с ограничением, обусловленным доступной памятью, предполагается, что память каждого процессора задействуется полностью, а рабочая нагрузка на один процессор равна O(1), где $O(1) = f \cdot O(1) + (1 - f)O(1)$. Положим, что при использовании всех n процессоров распараллеливаемая часть задачи может масштабироваться в G(n) раз. В этом случае масштабируемая рабочая нагрузка может быть описана выражением $O^* = f \cdot O(1) + (1 - f)G(n) \cdot f \cdot O(1)$. Здесь параметр G(n) отражает возрастание рабочей нагрузки с увеличением числа процессоров, а значит, и емкости памяти в n раз. В указанных рамках ускорение описывается выражением:

$$S_SN(n) = \frac{f + (1-f) \cdot G(n)}{f + (1-f) \cdot \frac{G(n)}{n}}. \quad (18)$$

Нетрудно показать, что полученное выражение представляет собой обобщение законов Амдала и Густафсона.

При $G(n) = 1$ размер задачи фиксирован. Это соответствует постановке Амдала. Если $G(n) = 1$ подставить в (18), то выражение выродится в закон Амдала (15).

Вариант G(n) = n соответствует случаю, когда с увеличением емкости памяти в n раз рабочая нагрузка возрастает тоже в n раз. Это идентично постановке Густафсона. Если $G(n) = n$ подставить в (18), то выражение выродится в закон Густафсона (17).

В случае, когда вычислительная нагрузка возрастает быстрее, чем требования к памяти ($G(n) > n$) модель с ограничением памяти дает более оптимистическую оценку ускорения.

7.4. Пример решения задачи

Пусть использование процессора в 16-процессорной ВС определяется следующей функцией:

$$P_f = \{1\ 5\ 6\ 8\ 10\ 12\ 14\ 16\ 16\ 16\ 14\ 16\ 14\ 10\ 9\ 8\ 6\ 4\ 3\ 1\}. \quad (19)$$

1. Определите профиль параллелизма, средний параллелизм, ускорение, эффективность, утилизацию, сжатие и качество параллельных вычислений, если избыточность $R(n)$=1,35.
2. Рассчитайте ускорение вычислений S(n) на n-процессорной ВС (n=2, 8, 32, 128, 512) по сравнению с однопроцессорной ВС в соответствии с закономерностями Амдала, Густафсона и Сана-Ная при доле последовательной части программы f=0,1 и G(n)=0,1n. Постройте графики зависимости S(n) от n.

Рассмотрим пример решения задачи. Изобразим в графическом виде профиль параллелизма, заданный функцией (19), рис. 69.

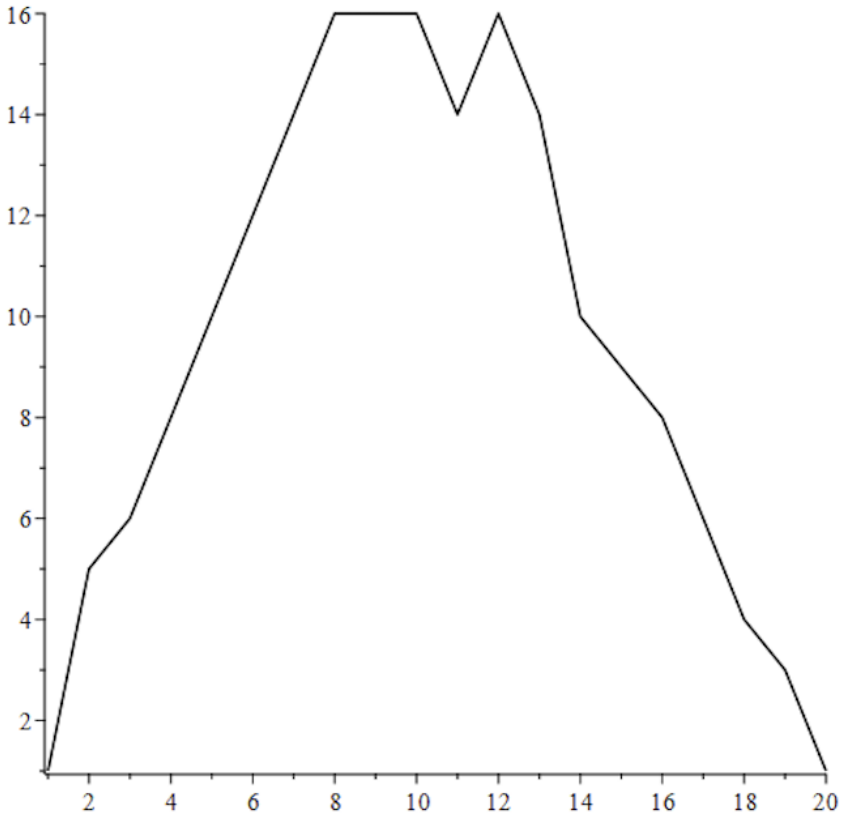


Рисунок 69. Графическое отображение профиля параллелизма

Первый этап. Рассчитаем метрики параллелизма. Максимальное количество процессоров, данных в функции (19) равно: n = 16. По формуле (6) получим значение общего объема вычислений O(n) = 189.

Исходя из формулы (11) и заданного значения избыточности $R(n)$=1,35, подсчитаем значение объема вычислений для однопроцессорной системы O(1) = 140. Общее время для однопроцессорной системы, равно значению объема вычислений, значит T(1) = 140.

Общее время для 16-ядерной системы T(n) = 20. Средний параллелизм, он же индекс параллелизма, вычисляем по формуле (7), получаем P = O(n)/T(n) = 9.45.

Ускорение по формуле (8) составляет S(n)=7.

Следующим шагом будет вычисление эффективности, по формуле (9), E(n) = 0,438. Далее рассчитаем утилизацию, по формуле (10), U(n) = 0,591, и сжатие, применив формулу (12), C(n) = 0,741.

Метрика качества будет равна Q(n) = 2,27, используя формулу (13).

Второй этап. Применяя формулы (15), (17) и (18) законов Амдала, Густафсона и Сана-Ная, соответственно, рассчитаем значения ускорения для количества процессоров 2, 8, 32, 128, 512 и построим графики. Результаты приведены графически на рис. 70 и 71.

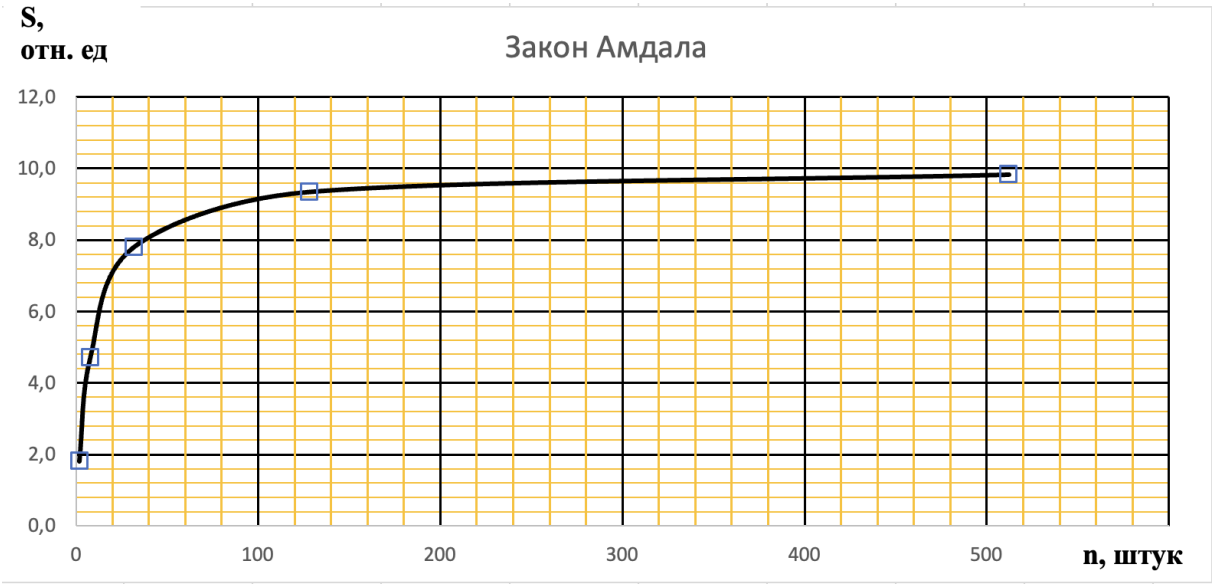


Рисунок 70. Зависимость ускорения от количества ядер по закону Амдала

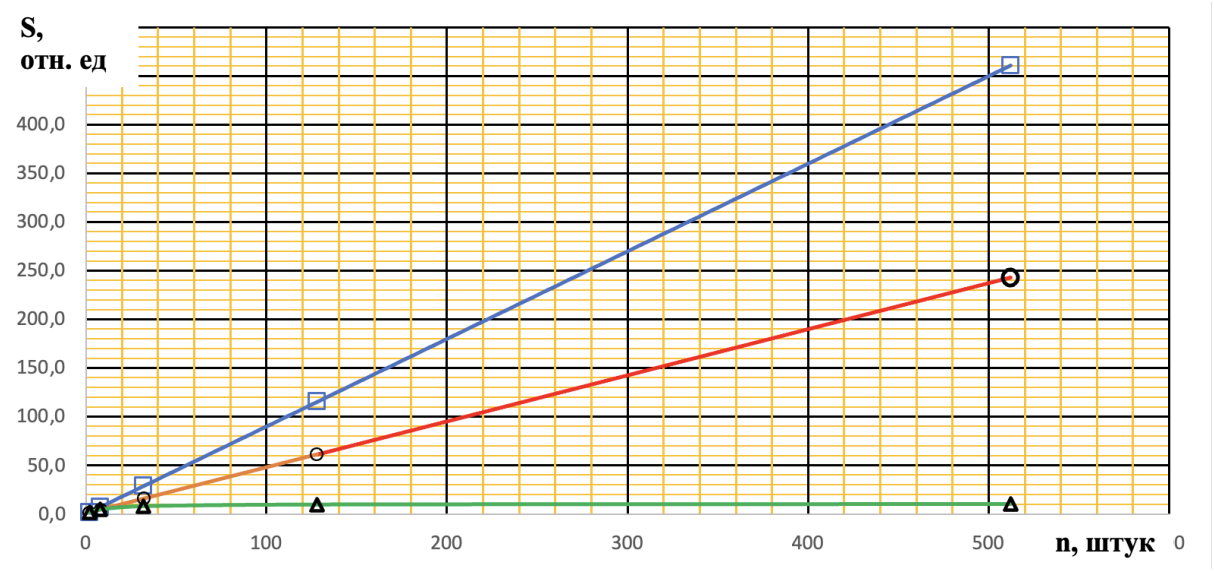


Рисунок 71. Зависимость ускорения от количества ядер по трем законам: Амдала (зеленый), Густафсона (синий), Сана-Ная (красный).

7.5. Задания для самостоятельного выполнения

1. Определите профиль параллелизма, средний параллелизм, ускорение, эффективность, утилизацию, сжатие и качество параллельных вычислений, при заданном значении избыточности $R(n)$ (табл. 34) и использовании процессоров в 16-процессорной ВС, заданном функцией (19).
2. Рассчитайте ускорение вычислений S(n) на n-процессорной ВС (n=2, 8, 32, 128, 512) по сравнению с однопроцессорной ВС в соответствии с закономерностями Амдала, Густафсона и Сана-Ная при заданных значениях доли последовательной части программы f и G(n). Постройте графики зависимости S(n) от n. Представить результаты графически для данных варианта, заданного преподавателем (табл. 34).

Таблица 34. Список вариантов для самостоятельного решения

Вариант	Избыточность R(n)	Доля последовательной части программы f	Коэффициент G(n)
1	1,1	0,2	0,1n
2	1,25	0,2	0,2n
3	1,3	0,2	0,5n
4	1,4	0,2	0,8n
5	1,5	0,1	0,1n
6	1,6	0,1	0,2n
7	1,15	0,1	0,5n
8	1,2	0,1	0,8n
9	1,35	0,05	0,1n
10	1,45	0,05	0,2n
11	1,55	0,05	0,5n
12	1,65	0,05	0,8n
13	1,7	0,01	0,1n
14	1,75	0,01	0,2n
15	1,8	0,01	0,5n
16	1,85	0,01	0,8n
17	1,9	0,005	0,1n
18	1,95	0,005	0,2n
19	1.05	0,005	0,5n
20	1.15	0,005	0,8n

ПРАКТИЧЕСКАЯ РАБОТА № 8

8.1. Общие сведения

Стек протоколов TCP/IP предназначен для соединения отдельных подсетей, построенных по разным технологиям канального и физического уровней (Ethernet, Token Ring, FDDI, ATM, X.25 и т. д.) в единую составную сеть. Каждая из технологий нижнего уровня предполагает свою схему адресации. Поэтому на межсетевом уровне требуется единый способ адресации, позволяющий уникально идентифицировать каждый узел, входящий в составную сеть. Таким способом в TCP/IP-сетях является IP-адресация.

Узел составной сети, имеющий IP-адрес, называется хост (host). Хороший пример, иллюстрирующий составную сеть, — международная почтовая система адресации. Информация сетевого уровня — это индекс страны, добавленный к адресу письма, написанному на одном из тысяч языков земного шара, например, на китайском. И даже если это письмо должно пройти через множество стран, почтовые работники которых не знают китайского, понятный им индекс страны-адресата подскажет, через какие промежуточные страны лучше передать письмо, чтобы оно кратчайшим путем попало в Китай. А уже там работники местных почтовых отделений смогут прочитать точный адрес, указывающий город, улицу, дом и человека, и доставить письмо адресату, так как адрес написан на языке и в форме, принятой в данной стране.

8.2. Типы адресов стека TCP/IP

В стеке TCP/IP используются три типа адресов:

- локальные (другое название — аппаратные);
- IP-адреса (сетевые адреса);
- символьные доменные имена.

Локальный адрес — это адрес, присвоенный узлу в соответствии с технологией подсети, входящей в составную сеть. Если подсетью является локальная сеть Ethernet, Token Ring или FDDI, то локальный адрес — это MAC-адрес.

MAC-адреса (MAC address — Media Access Control address) назначаются сетевым адаптерам и портам маршрутизаторов производителями оборудования и являются уникальными, так как распределяются централизованно. MAC-адрес имеет размер 6 байт и записывается в шестнадцатеричном виде, например, 00-08-A0-12-5F-72.

IP-адреса (IP address) представляют собой основной тип адресов, на основании которых сетевой уровень передает сообщения, называемые IP-пакетами.

Эти адреса состоят из 4 байт, записанных в десятичном виде и разделенных точками, например, 117.52.9.44.

Номер узла в протоколе IP назначается независимо от локального адреса узла. Маршрутизатор по определению входит сразу в несколько сетей. Поэтому каждый порт маршрутизатора имеет собственный IP-адрес. Конечный узел также может входить в несколько IP-сетей. В этом случае компьютер должен иметь несколько IP-адресов, по числу сетевых адаптеров. Поэтому IP-адрес характеризует не отдельный компьютер или маршрутизатор, а одно сетевое соединение.

Символьные доменные имена (domain name) служат для удобства представления IP-адресов. Человеку неудобно запоминать числовые IP-адреса, поэтому была организована специальная служба — DNS (Domain Name System), которая устанавливает соответствие между IP-адресами и символьными доменными именами. Пример символьного доменного имени — www.rambler.ru.

Структура IP-адреса. IP-адрес представляет собой 32-разрядное двоичное число, разделенное на группы по 8 бит, называемых октетами.

Пример IP-адреса: 00010001 11101111 00101111 01011110. Обычно IP-адреса записываются в виде четырех десятичных октетов и разделяются точками. Таким образом, приведенный выше IP-адрес можно записать в следующей форме: 17.239.47.94.

Следует заметить, что максимальное значение октета равно 11111111_2 (двоичная система счисления), что соответствует в десятичной системе 255_{10}. Поэтому IP-адреса, в которых хотя бы один октет превышает это число, являются недействительными. Пример: 172.16.123.1 — действительный адрес, 172.16.123.256 — несуществующий адрес, поскольку 256 выходит за пределы допустимого диапазона.

IP-адрес состоит из двух логических частей — номера подсети (ID подсети) и номера узла (ID хоста) в этой подсети. При передаче пакета из одной подсети в другую используется ID подсети. Когда пакет попал в подсеть назначения, ID хоста указывает на конкретный узел в рамках этой подсети.

Чтобы записать ID подсети, в поле номера узла в IP-адресе ставят нули. Чтобы записать ID хоста, в поле номера подсети ставят нули. Например, если в IP-адресе 172.16.123.1 первые два байта отводятся под номер подсети, остальные два байта — под номер узла, то номера записываются следующим образом: ID подсети: 172.16.0.0. ID хоста: 0.0.123.1.

По числу разрядов, отводимых для представления номера узла (или номера подсети), можно определить общее количество узлов (или подсетей) по простому правилу: если число разрядов для представления номера узла равно N, то общее

количество узлов равно $2^N - 2$.

Два узла вычитаются вследствие того, что адреса со всеми разрядами, равными нулям или единицам, являются особыми и используются в специальных целях. Например, если под номер узла в некоторой подсети отводится два байта (16 бит), то общее количество узлов в такой подсети равно $2^{16} - 2 = 65534$ узла.

Для определения того, какая часть IP-адреса отвечает за ID подсети, а какая за ID хоста, применяются два способа: с помощью классов и с помощью масок. Общее правило: под ID подсети отводятся первые несколько бит IP- адреса, оставшиеся биты обозначают ID хоста.

8.3. Классы IP-адресов

Существует пять классов IP-адресов: A, B, C, D и E.

За принадлежность к тому или иному классу отвечают первые биты IP- адреса. Деление сетей на классы описано в RFC 791 (документ описания протокола IP). Целью такого деления являлось создание малого числа больших сетей (класса A), умеренного числа средних сетей (класс B) и большого числа малых сетей (класс C).

Если адрес начинается с 0, то сеть относят к классу A и номер сети занимает один байт, остальные 3 байта интерпретируются как номер узла в сети.

Сети класса A имеют номера в диапазоне от 1 до 126. Сетей класса A немного, зато количество узлов в них может достигать $2^{24} - 2$, то есть 16 777 214 узлов.

Если первые два бита адреса равны 10, то сеть относится к классу B. В сетях класса B под номер сети и под номер узла отводится по 16 бит, то есть по 2 байта. Таким образом, сеть класса B является сетью средних размеров с максимальным числом узлов $2^{16} - 2$, что составляет 65 534 узлов.

Если адрес начинается с последовательности 110, то это сеть класса C. В этом случае под номер сети отводится 24 бита, а под номер узла — 8 бит. Сети этого класса наиболее распространены, число узлов в них ограничено $2^8 - 2$, то есть 254 узлами.

Адрес, начинающийся с 1110, обозначает особый, групповой адрес (multicast). Пакет с таким адресом направляется всем узлам, которым присвоен данный адрес.

Адреса класса E в настоящее время не используются (зарезервированы для будущих применений).

Применение классов удовлетворительно решало задачу деления на подсети в начале развития Интернета. В 90-е годы с увеличением числа подсетей стал ощущаться дефицит IP-адресов. Это связано с неэффективностью распределения

при классовой схеме адресации. Например, если организации требуется тысяча IP-адресов, ей выделяется сеть класса B, при этом 64534 адреса не будут использоваться.

Существует два основных способа решения этой проблемы:

1) более эффективная схема деления на подсети с использованием масок (RFC 950);
2) применение протокола IP версии 6 (IPv6).

8.4. Использование масок

Маска подсети (subnet mask) — это число, которое используется в паре с IP-адресом; двоичная запись маски содержит единицы в тех разрядах, которые должны в IP-адресе интерпретироваться как номер сети. Для стандартных клас сов сетей маски имеют следующие значения:

– класс A — 11111111. 00000000. 00000000. 00000000 (255.0.0.0);
– класс B — 11111111. 11111111. 00000000. 00000000 (255.255.0.0);
– класс C — 11111111. 11111111. 11111111. 00000000 (255.255.255.0).

Маска подсети записывается либо в виде, аналогичном записи IP-адреса, например 255.255.255.0, либо совместно с IP-адресом с помощью указания числа единичных разрядов в записи маски, например 192.168.1.1/24, т. е. в маске содержится 24 единицы (255.255.255.0).

При использовании масок можно вообще отказаться от понятия классов.

Пример. Пусть задан IP-адрес 17.239.47.94, маска подсети 255.255.0.0 (другая форма записи: 17.239.47.94/16). Требуется определить ID подсети и ID хоста в обеих схемах адресации.

Адресация с использованием классов. Двоичная запись IP-адреса имеет вид: 00010001. 11101111. 00101111. 01011110 Так как первый бит равен нулю, адрес относится к классу A. Следовательно, первый байт отвечает за ID подсети, остальные три байта — за ID хоста: ID подсети: 17.0.0.0. ID хоста: 0.239.47.94.

Адресация с использованием масок. Запишем IP-адрес и маску подсети в двоичном виде. IP-адрес: 17.239.47.94 = 00010001. 11101111. 00101111. 01011110. Маска подсети: 255.255.0.0 = 11111111. 11111111. 00000000. 00000000.

Вспомним определение маски подсети: интерпретируем как номер подсети те биты, которые в маске равны 1, т. е. первые два байта. Оставшаяся часть IP-адреса будет номером узла в данной подсети. Следовательно, ID подсети: 17.239.0.0. ID хоста: 0.0.47.94.

Номер подсети можно получить другим способом, применив к IP-адресу и маске операцию логического умножения AND: В масках количество единиц в последовательности, определяющей границу номера сети, не обязательно должно быть кратным 8.

Пример

Задан IP-адрес 192.168.89.16, маска подсети 255.255.192.0 (другая форма записи: 192.168.89.16/18). Требуется определить ID подсети и ID хоста. Вос пользуемся операцией AND: чтобы получить номер узла, нужно в битах, отвечающих за номер подсети, поставить нули: Host ID: для масок существует важное правило: разрывы в последовательности единиц или нулей недопустимы. Например, не существует маски подсети, имеющей следующий вид: 11111111. 11110111. 00000000. 00001000 (255.247.0.8), так как последовательности единиц и нулей не являются непрерывными.

Особые IP-адреса. Некоторые IP-адреса являются особыми, они не должны применяться для идентификации обычных сетей.

1. Если первый октет ID сети начинается со 127, такой адрес считается адресом машины-источника пакета. В этом случае пакет не выходит в сеть, а возвращается на компьютер-отправитель. Такие адреса называются loopback («петля», «замыкание на себя») и используются для проверки функционирования стека TCP/IP.
2. Если все биты IP-адреса равны нулю, адрес обозначает узел-отправитель и используется в некоторых сообщениях ICMP.
3. Если все биты ID сети равны 1, адрес называется ограниченным широковещательным (limited broadcast), пакеты, направленные по такому адресу, рассылаются всем узлам той подсети, в которой находится отправитель пакета.
4. Если все биты ID хоста равны 1, адрес называется широковещательным (broadcast), пакеты, имеющие широковещательный адрес, доставляются всем узлам подсети назначения.
5. Если все биты ID хоста равны 0, адрес считается идентификатором подсети (subnet ID). Наличие особых IP-адресов объясняет, почему из диапазона доступных адресов исключаются два адреса — это случаи, когда все биты ID хоста равны 1 или 0. Например, в сети класса С не 256, а 254 узла.

С помощью масок администратор (провайдер сети) может структурировать свою сеть путем выделения нескольких подсетей с меньшим количеством узлов.

Например, в сети 160.240.128.0/17 требуется выделить 32 подсети меньшего объема. Для этого администратор сети выделяет требуемое количество бит ($32=2^5$, то есть 5 разрядов) в старших разрядах выделенного ему адресного пространства для обозначения номера подсети (от 0 до 31). Тогда для нумерации узла в подсети остается 32-17-5=10 двоичных разрядов, что соответствует максимальному количеству узлов в каждой подсети 2^{10}-2=1022. В этом случае IP-адрес 160.240.192.177/17 будет принадлежать 177-му узлу в 16-й подсети данного провайдера (сети 160.240.128.0/17).

Маршрутизаторы во внешней сети (Интернете) ничего «не знают» о делении сети 160.240.128.0/17 на подсети, все пакеты направляются на маршрутизатор организации, который переправляет их в требуемую внутреннюю подсеть.

8.5. Протокол IPv6

Использование масок является временным решением проблемы дефицита IP-адресов, так как адресное пространство протокола IP не увеличивается, а количество хостов в Интернете растет с каждым днем. Для принципиального решения проблемы требуется существенное увеличение количества IP-адресов. Используемый в настоящее время и рассматриваемый в данном курсе протокол IP называется IPv4 — протокол IP 4-й версии. Для преодоления ограниченийIPv4 был разработан протокол IP 6-й версии — IPv6 (RFC 2373, 2460).

Протокол IPv6 имеет следующие основные особенности:

- длина адреса 128 бит — такая длина обеспечивает адресное пространство 2128, или примерно 3.4·1038 адресов. Такое количество адресов позволит присваивать в обозримом будущем уникальные IP-адреса любым устройствам;
- автоматическая конфигурация — протокол IPv6 предоставляет средства автоматической настройки IP-адреса и других сетевых параметров даже при отсутствии таких служб, как DHCP;
- встроенная безопасность — для передачи данных является обязательным использование протокола защищенной передачи IPsec. Протокол IPv4 также может использовать IPsec, но не обязан этого делать.

В настоящее время многие производители сетевого оборудования включают поддержку протокола IPv6 в свои продукты, однако преобладающим остается протокол IPv4. Связано это с тем, что IPv6 обратно несовместим с IPv4 и процесс перехода сопряжен с определенными трудностями.

8.6. Протокол ARP

Протокол IP действует на сетевом уровне модели OSI, поэтому IP-адреса

называются сетевыми. Они предназначены для передачи сообщений в составных сетях, связывающих подсети, построенные на различных локальных или глобальных сетевых технологиях, например, Ethernet или ATM. Однако для непосредственной передачи сообщения в рамках одной подсети вместо IP-адреса нужно использовать локальный (аппаратный) адрес технологии канального уровня, чаще всего MAC-адрес. При этом к IP-пакету добавляются заголовок и концевик кадра канального уровня, в заголовке указываются MAC-адреса источника и приемника кадра.

При формировании кадра канального уровня возникает проблема: каким образом по известному IP-адресу определить соответствующий MAC-адрес? Указанная проблема решается при помощи протокола ARP (Address Resolution Protocol — протокол разрешения адресов).

Протокол ARP определяет MAC-адреса следующим образом. Осуществляется рассылка всем узлам сети специального кадра, который называется ARP-запрос (ARP Request). В этом кадре содержится IP-адрес компьютера, у которого требуется узнать MAC-адрес. Каждый узел сети принимает ARP-запрос и сравнивает IP-адрес из запроса со своим IP-адресом. Если адреса совпадают, узел высылает ARP-ответ (ARP Reply), содержащий требуемый MAC-адрес. Результаты своей работы протокол ARP сохраняет в специальной таблице, хранящейся в оперативной памяти, которая называется ARP-кэш. При необходимости разрешения IP-адреса, протокол ARP сначала ищет IP-адрес в ARP-кэше и только в случае отсутствия нужной записи производит рассылку ARP- запроса.

ARP-кэш имеет следующий вид: Записи в ARP-кэше могут быть двух типов: статические и динамические. Статические записи заносятся в кэш администратором при помощи утилиты arp с ключом /s. Динамические записи помещаются в кэш после полученного ARP-ответа и по истечении двух минут удаляются.

Удаление происходит для того, чтобы при перемещении в другую подсеть компьютера с MAC-адресом, занесенным в таблицу, кадры не отправлялись бесполезно в сеть. Иногда требуется по известному MAC-адресу найти IP-адрес (например, при начале работы компьютеров без жесткого диска, у которых есть MAC-адрес сетевого адаптера и им нужно определить свой IP-адрес). В этом случае используется реверсивный протокол RARP (Reverse ARP).

8.7. Пример задания

В качестве примера практического задания рассмотрим разбор IP-адреса.

Дан IP-адрес хоста, например, 113.217.23.89/18 для 8 подсетей. Требуется найти:

1. IP-адрес хоста в двоичном коде.

2. Маску в двоичном коде.
3. Маску в десятичном коде.
4. IP-адрес провайдера в двоичном коде.
5. IP-адрес провайдера в десятичном коде.
6. № текущей подсети.
7. № текущего хоста.
8. Максимальное количество хостов в одной подсети.

Решение:

1. Для нахождения IP-адреса хоста в двоичном коде требуется представить заданный IP-адрес (в текущем случае 113.217.23.89) в двоичном представлении. Согласно правилам написания IPv4, в каждом значении, разделенным точкой при двоичном представлении должны быть обязательно задействованы и отражены все 8 разрядов (даже если первые могут быть нулями). Так, переводя 113.217.23.89 в двоичный код получаем: 01110001.11011001.00010111.01011001.
2. Маска в двоичном коде определяется по количеству бит, которые занял под свои задачи провайдер. Это число пишется после знака «/» (в данном случае это 18). Следовательно, первые 18 бит занял провайдер и маска будет состоять в двоичном представлении из первых 18 единиц: 11111111.11111111.11000000.00000000.
3. Маска в десятичном коде находится в результате перевода, полученного в п.2 числа в десятичный формат: 255.255.192.0.
4. IP-адрес провайдера находится в результате побитового логического умножения двоичных представлений IP-адреса и маски (п.1 и п.2). Результатом будет: 01110001.11011001.00000000.00000000.
5. IP-адрес провайдера в десятичном коде находится в результате перевода, полученного в п.4 числа в десятичный формат: 113.217.0.0.
6. Для определения номера текущей подсети требуется сначала определить необходимое количество бит в двоичном коде. Это определяется по формуле log2 (число подсетей). В данном примере будет требоваться 3 бита для описания 8 подсетей. Биты подсетей (выделены желтым) в исходном IP-адресе идут сразу после битов провайдера (выделены зеленым): 01110001.11011001.00010111.01011001. № текущей подсети хоста определяется двоичным числом, находящимся среди битов подсети. В данном случает № текущей подсети будет равен 2.
7. Для определения номера текущего хоста требуется перевести число, представленное в двоичном коде и составленное по оставшимся битам,

которые не задействованы были у провайдера или в подсетях. В данном случае число 111010110012 = 188110.

8. Для определения максимального количество хостов в одной подсети требуется возвести 2 в степень количества бит, используемых хостом (в данном случае 11) и вычесть 2 номера, так как они являются широковещательными (номер со всеми нулями и всеми единицами). В результате получится: 211-2=2046 хостов может быть в одной подсети.

8.8. Задания для самостоятельного решения

Формулировка задания изложена в подразделе 8.7. Переменные данные для вариантов заданий для закрепления данной темы и самостоятельного решения представлены в табл.35.

Таблица 35. Список вариантов для самостоятельного решения

№ варианта	IP адрес	Кол-во подсетей
1	114.107.30.99/17	32
2	135.26.49.103/18	16
3	111.210.37.56/20	32
4	71.47.77.254/19	4
5	132.255.45.83/13	128
6	27.244.92.72/20	8
7	61.40.143.98/21	64
8	11.204.52.89/19	16
9	240.96.219.33/15	32
10	204.208.144.9/21	4
11	176.254.20.218/12	256
12	82.18.76.156/18	8
13	151.26.196.195/15	16
14	66.98.135.13/14	64
15	8.52.217.4/15	32
16	36.138.197.239/20	8
17	243.54.2.37/19	4
18	147.99.96.16/18	2
19	41.92.199.215/22	4
20	47.113.213.152/17	64
21	31.40.163.180/17	32
22	26.38.32.100/16	8
23	117.47.119.174/13	128
24	212.19.133.125/17	16
25	78.166.217.22/15	64
26	153.83.88.30/18	32
27	231.162.157.61/19	8

Окончание табл. 35

№ варианта	IP адрес	Кол-во подсетей
28	29.57.167.223/13	128
29	146.191.99.111/17	16
30	169.12.101.152/19	4
31	211.40.236.146/18	16

ПРАКТИЧЕСКАЯ РАБОТА №9

9.1. Виртуальная и физическая память

По способу использования можно разделить память на виртуальную, физическую, страничную и другие виды. Проблема заключается в том, что оперативной памяти (ОП) часто не хватает для выполнения процессов на виртуальных машинах. Один из способов решения этой проблемы - использование оверлеев, где программист самостоятельно дробит программу на части, которые загружаются в ОП по очереди. Это помогает частично справиться с нехваткой оперативной памяти, но создает сложности для разработчиков.

Другим решением стало использование виртуальной памяти с делегированием операционной системе задач по загрузке необходимых для работы процессов частей виртуальной памяти с диска в ОП. Это позволило обеспечить прозрачность работы с памятью для программистов. Диспетчер памяти операционной системы берет на себя всю работу по загрузке и выгрузке частей процесса с диска в ОП, что делает этот процесс невидимым для программистов.

Виртуальная память — это совокупность программно-аппаратных средств, позволяющих пользователю писать и выполнять на ВМ программы, объём которых превосходит имеющуюся в ВМ оперативную (физическую) память. Для этого делают следующее:

- в исходном состоянии все данные и программный код размещают на диске;
- ОС при выполнении процесса перемещает данные и программный код между диском и ОП частями таким образом, чтобы пользователь этого не замечал;
- пользователь все время работает как бы в виртуальной памяти большого объема.

Рассмотрим, как это работает на практике. Для этого существует три метода управления памятью: страничное распределение, сегментное распределение и сегментно-страничное.

На рис. 72 приведена схема взаимодействия виртуальной и физической памяти. В диске данные хранятся на дорожках секторами, размер которых равен 512 байт. Поэтому размер страницы выбирают кратным этому значению: 1К, 2К, 4К, 8К.

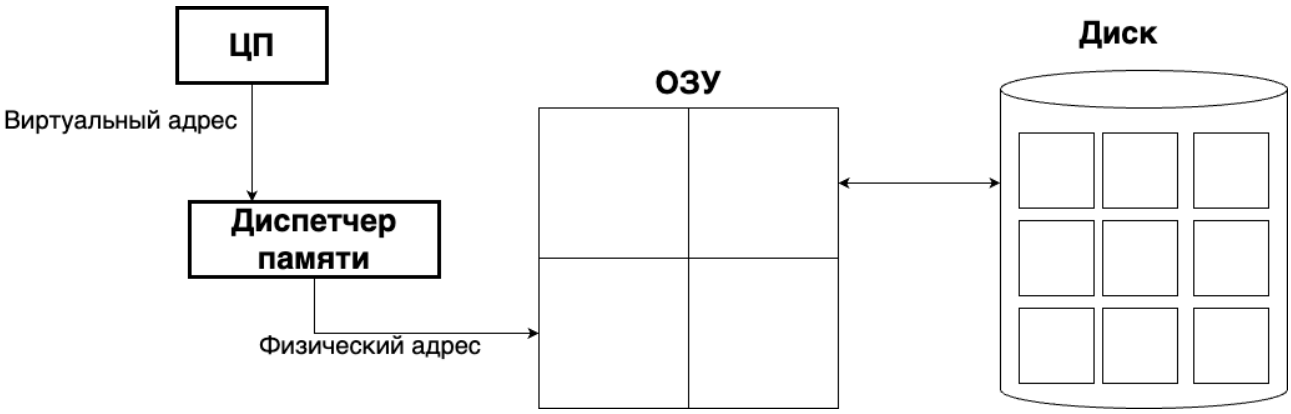


Рисунок 72. Взаимодействие виртуальной и физической памяти

9.2. Страничное распределение памяти

Представим страничное распределение в виде схемы на рис. 73.

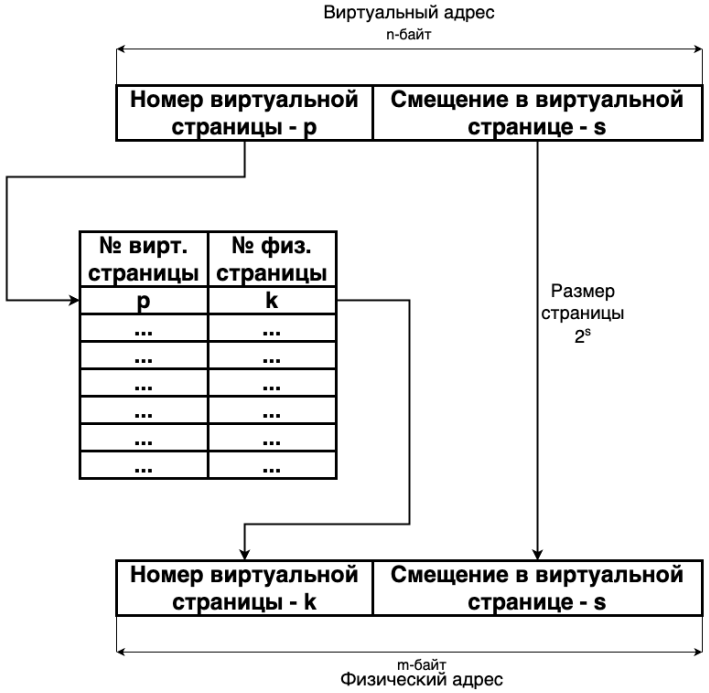


Рисунок 73. Страничное распределение памяти

Для преобразования виртуального адреса в физический диспетчер памяти ОС строит страничную таблицу (табл. 36).

Таблица 36. Таблица страничного распределения

№ вирт. стр.	V	R	M	A	№ физ. стр.	Карта диска
0	0					
1	0					
…						
$1001=9_{10}$	1				$10=2_{10}$	3 дор.2 сект.
…						
$2^8-1=255_{10}$						

Условные обозначения в таблице следующие:

- V — признак присутствия виртуальной страницы в ОП;
- R — признак использования страницы. Обычно — несколько разрядов. При каждом обращении к странице значение инкрементируется. Чем больше R, тем активнее идет обращение к странице и тем важнее оставить ее в ОП при необходимости вытеснения страницы на диск;
- M — признак модификации страницы. Равен 1, если страница модифицировалась, т.е. в нее производилась запись данных. При вытеснении такой страницы её необходимо сохранить на диск прежде записи на это место другой страницы;
- A — признак прав доступа к странице. Обычно составляет 2 разряда.

Обращение к таблице страниц может быть достигнуто быстрее при использовании буфера быстрого преобразования адреса TLB (Translation Lookaside Buffer). Если страничную таблицу хранить в памяти, то при каждом обращении к памяти надо обратиться прежде всего к таблице, т.е. требуется дополнительное обращение к памяти. Поэтому наиболее используемую часть страничной таблицы размещают в ассоциативной памяти, дорогой, но малой емкости. При этом поиск в таблице производится по определенным признакам и осуществляется до 10 раз быстрее, чем в обычной памяти.

Положительными качествами рассмотренного страничного распределения памяти являются его простота и быстродействие. Недостатками являются недостаточный учет прав доступа к информации, которая передается из диска в ОП и обратно одинаковыми страницами. Для устранения этого недостатка применяют сегментное распределение памяти.

9.3. Сегментное распределение памяти

Программиста интересует, в первую очередь, назначение информации, представленной в памяти сегментами с различными правами доступа. Поэтому сегментное распределение предусматривает передачу информации между диском и ОП целыми сегментами с одинаковыми правами доступа. При этом возможны ситуации, когда один и тот же сегмент (например, подпрограмма) используется одновременно двумя и более процессами. Поэтому не обязательно иметь много копий соответствующего кода в памяти каждого процесса, а достаточно иметь один такой сегмент в ОП и обращаться к нему из разных процессов по мере необходимости.

В случае сегментного распределения памяти (рис. 74) для каждого процесса создается таблица сегментов, которая имеет сведения о размере сегмента, нахождении его в ОП и управляющую информацию, позволяющую определять, какой сегмент следует выгрузить из ОП при необходимости загрузки нового сегмента.

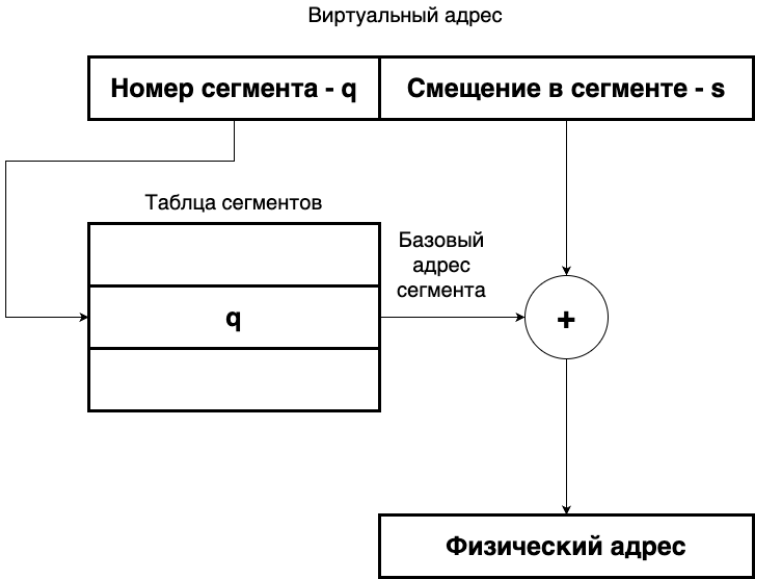


Рисунок 74. Сегментное распределение памяти

Фактически таблица сегментов аналогична таблице страниц при страничном распределении памяти. Отличие заключается в том, что сегменты имеют различный размер и находятся в виртуальной памяти не обязательно с начала страницы. Поэтому при вычислении физического адреса необходимо производить арифметические вычисления, учитывая базовый адрес сегмента в ОП и смещение внутри сегмента. Эти вычисления требуют больше времени по сравнению с вычислением физического адреса при страничном распределении, которое осуществляется посредством конкатенации номера страницы и смещения внутри страницы. Кроме того, недостатком сегментного распределения является эффект фрагментации ОП ввиду неодинаковости размера сегментов.

Способом устранения этих недостатков является переход к сегментно-страничному распределению памяти.

9.4. Сегментно-страничное распределение памяти

Сегментно-страничное распределение памяти совмещает достоинства обоих вышерассмотренных способов распределения памяти. При этом виртуальный адрес состоит из трех составляющих: номера сегмента q, номера страницы p и смещения внутри страницы s. Перемещение информации между диском и ОП осуществляется страницами одинакового размера. Для вычисления физического адреса используются две таблицы: таблица сегментов и таблица страниц сегмента.

Схема определения физического адреса показана на рис. 75.

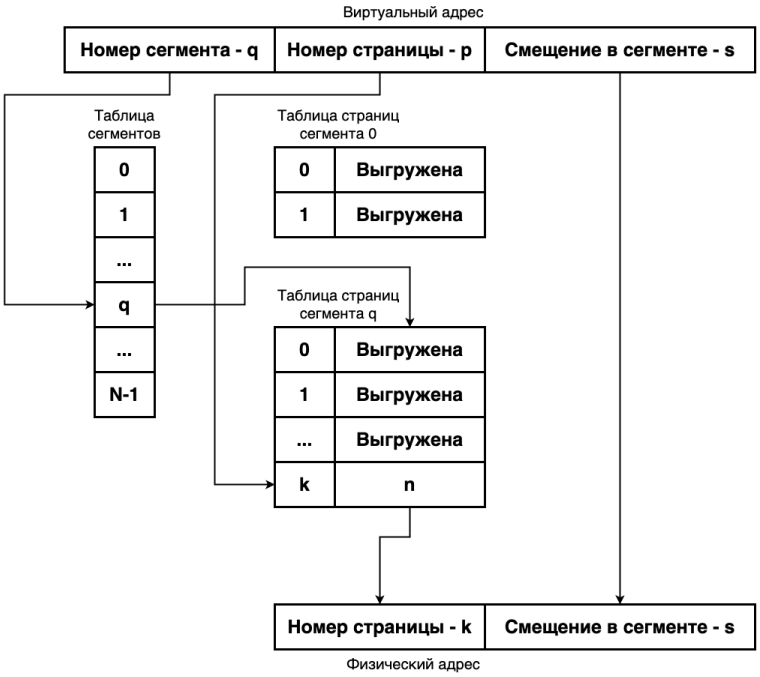


Рисунок 75. Сегментно-страничное распределение

Фрагментация памяти при таком распределении существенно уменьшается, поскольку передача информации осуществляется страницами одинакового размера, и неполной может быть только последняя страница сегмента. Кроме того, упрощается вычисление адресов, которое аналогично страничному распределению, но требует двойного обращения к памяти (используются две таблицы). По номеру сегмента выбирается таблица страниц сегмента, по номеру страницы в этой таблице выбирается номер страницы в физической памяти. Смещение внутри страницы виртуальной и физической памяти совпадают, поэтому используется операция конкатенации.

9.5. Перевод виртуального адреса в физический

Рассмотрим процесс перевода виртуального адреса в физический. Разбирать процесс будем на примере самого простого распределения — страничного.

Пусть виртуальная память имеет объем 16 Мбайт, это 2^{24} байт. Данное значение получается путем сложения степени двойки объема и размера. Объем выделенной памяти — 16, если представить это значение как степень двойки, то получится 4. Размер памяти исчисляется в Мбайт, приставка М (мега), если представить ее как степень двойки будет 20, при сложении двух полученных значений получим итоговое значение в виде степени двойки — 2^{24}. Аналогично, пусть физическая память имеет объем 32 Кбайт (2^{15} байт). Объем — 32, это 2^5. Размер

в данном случае исчисляется в К (кило), что меньше, чем мега, и равна значению 2^{10}.

Пусть страница имеет объем 4 Кбайт, аналогично это 2^{12} байт. Виртуальный адрес состоит из двух частей: 12 старших разрядов — номер страницы, 12 младших разрядов — смещение в странице. Физический адрес тоже состоит из двух частей: 3 старших разряда — номер страницы, 12 младших разрядов — смещение в странице.

Пусть требуется перенести виртуальную страницу под номером 1278, со смещением 2564, на страницу 28 в физическую память. Для окончательной записи адресов: переведем имеющиеся значения в двоичный вид. Перевод можно осуществлять любым известным способом. 1278 в десятичной системе счисления — 10011111110 в двоичной, значение 2564 — это 101000000100, значение 7 — это 111.

Запишем виртуальный и физический адреса в двоичном виде (рис. 76):

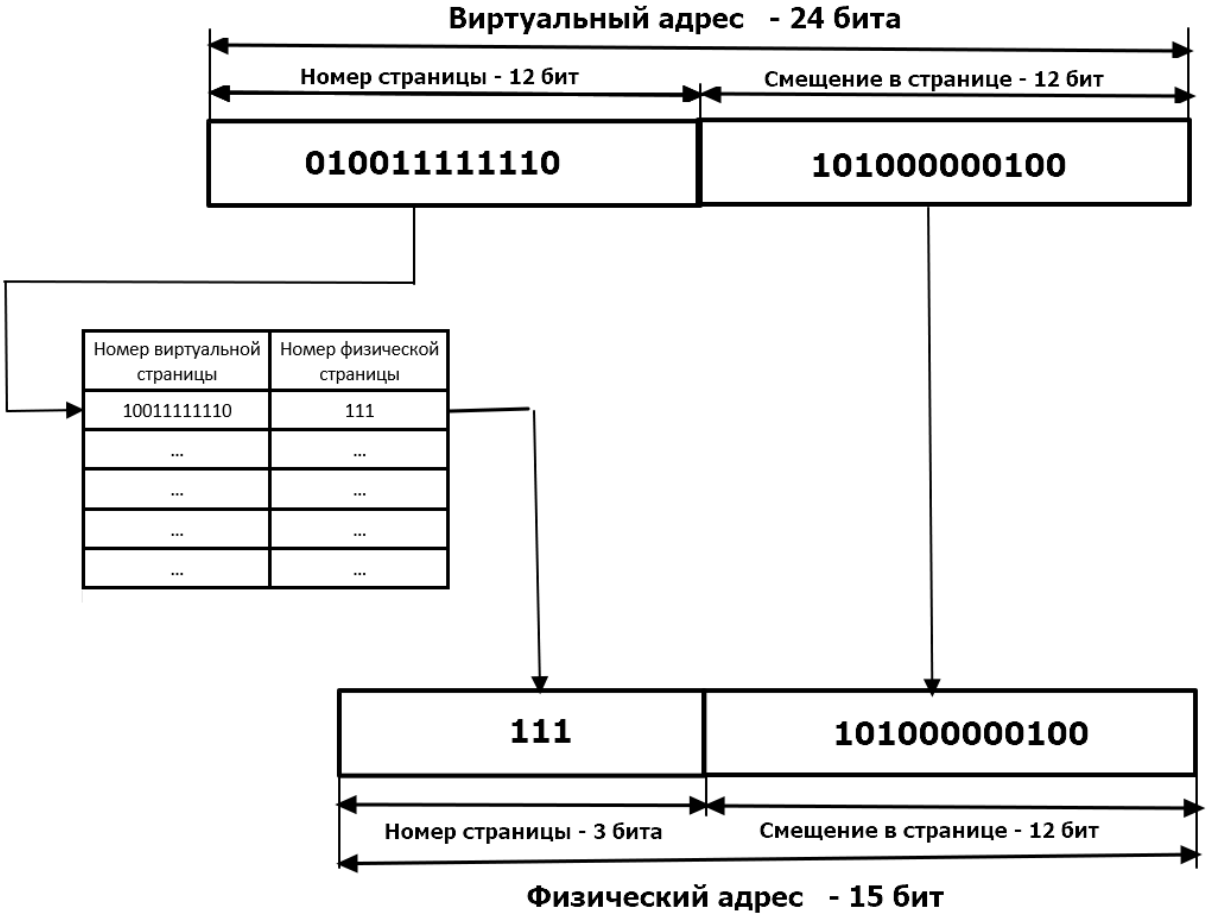


Рисунок 76. Виртуальная страница, перенесенная в физическую память

9.6. Задания для самостоятельного решения

Варианты заданий представлены в табл. 37.

Таблица 37. Варианты заданий для самостоятельного решения

№ варианта	Объем виртуальной памяти, байт	Объем физической памяти, байт	Размер страницы, байт	№ виртуальной страницы	Смещение, байт	№ физической страницы
1	8 М	512 К	2 К	35	123	33
2	16 М	1 М	4 К	48	259	17
3	32 М	4 М	8 К	46	306	23
4	1 М	256 К	16 К	99	462	7
5	512 К	128 К	1 К	77	239	32
6	4 М	512 К	16 К	12	145	21
7	2 М	512 К	32 К	24	289	8
8	8 М	256 К	64 К	23	177	2
9	16 М	1 М	2 К	45	153	22
10	32 М	32 К	4 К	24	99	6
11	1 М	1 М	8 К	7	53	19
12	512 К	128 К	16 К	32	49	5
13	4 М	256 К	2 К	15	69	31
14	2 М	16 К	2 К	13	121	5
15	8 М	512 К	4 К	14	257	13
16	16 М	64 К	8 К	25	215	3
17	32 М	512 К	16 К	27	217	23
18	1 М	256 К	1 К	29	39	19
19	512 К	128 К	16 К	24	119	7
20	4 М	512 К	32 К	23	187	4
21	2 М	128 К	8 К	12	305	13

ПРАКТИЧЕСКАЯ РАБОТА №10

10.1. Общая организация прерываний

Механизм прерывания в компьютере осуществляется специальными аппаратно-программными средствами. Аппаратные средства обработки прерывания, в процессоре ЭВМ, работают по принципу приостановки текущей программы и передачи управления подпрограмме обработки прерывания. Для сохранения информации из различных регистров процессора, во время прерывания и последующего возврата к основной программе, используются специальные механизмы. Обычно, сохранение содержимого счетчика команд и регистра флагов, а также других регистров, необходимых для обработки прерывания. Действия при обработке внешних прерываний, аналогичны, действиям при вызове обычной подпрограммы, за исключением того, что прерывание инициируется управляющим сигналом от контроллера внешнего устройства. Этот механизм позволяет обмениваться данными с внешними устройствами в реальном времени, не зависящем от выполнения основной программы. При этом прерывание программы по запросу внешнего устройства не должно влиять на саму программу, за исключением временного простоя для обработки прерывания.

Формирование сигнала прерывания — запроса внешнего устройства на обслуживание, происходит в их контроллерах — электронных схемах, обеспечивающих связь с процессором. В серийных ЭВМ обычно используется одноуровневая система прерываний — сигналы. Запрос прерывания от всех внешних устройств поступают на один вход процессора. Поэтому возникает проблема идентификации внешнего устройства, запросившего обслуживание. Основным способом, решающим проблему идентификации в большинстве современных ЭВМ в настоящее время, является использование векторов прерываний.

Внешнее устройство, запросившее обслуживание, само идентифицирует себя с помощью адреса своего вектора прерывания — ячейки основной памяти, в которой, хранится адрес начала программы обработки прерываний данного типа. Векторы всех обработчиков прерываний собраны в единую таблицу векторов прерываний, располагающуюся в самых младших адресах оперативной памяти, имеющую объем 1 Кбайт и содержащую четырехбайтные элементы для 256 обработчиков прерываний. Так как таблица всегда имеет нулевой начальный адрес и длину вектора в 4 байта, чтобы определить адрес вектора для прерывания типа i, достаточно просто умножить это значение на 4. Вектор прерывания выдается контроллером не одновременно с запросом на прерывание, а только по разрешению процессора (рис. 77).

Регистр прерываний составлен триггерами внешних устройств, устанавливаемыми в единичное состояние требованиями прерывания соответствующих контроллеров. Выходы триггеров поступают на входы приоритетного шифратора через элементы совпадения, вторые входы которых соединены с выходами регистра маски. Разряды этого регистра устанавливаются ОС в 0 при запрете прерывания соответствующего устройства и в 1, если прерывание разрешено.

При поступлении хотя бы одного требования прерывания на входы приоритетного шифратора он устанавливает в единичное состояние свой выход E и устанавливает код приоритета прерывания на других своих выходах в соответствии с таблицей соответствия (табл. 38).

Таблица 38. Таблица соответствий кодов приоритетов

I0	I1	I2	I3	Y	X	IST
1	X	X	X	0	0	1
0	1	X	X	0	1	1
0	0	1	X	1	0	1
0	0	0	1	1	1	1
0	0	0	0	0	0	0

Описанный выше процесс, можно изобразить в виде схемы, на рис. 77, проиллюстрированном ниже. Также, добавим в схему несколько дополнительных сигналов, а именно: IEN — Interrupt Enable — прерывание разрешено, IST — Interrupt Status — статус прерывания, INTACK — подтверждение прерывания.

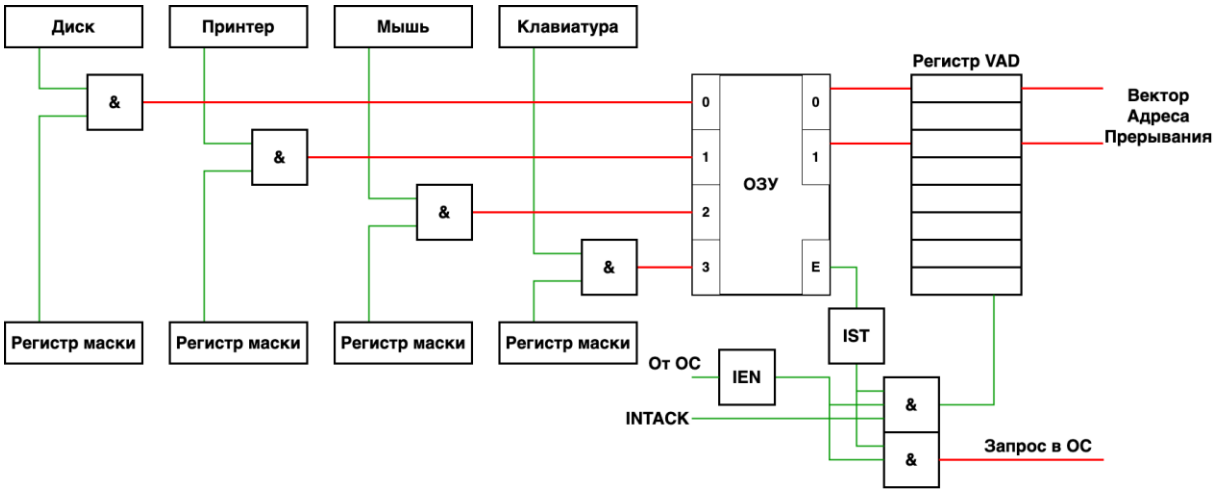


Рисунок 77. Схема обработки прерываний от внешних устройств

В регистре прерываний устройства подключены в соответствии с их приоритетами. Чем выше приоритет, тем на меньший по значению вход приоритет-

ного шифратора он поступает. Поэтому при одновременном поступлении требований прерывания от нескольких внешних устройств будет обрабатываться требование от устройства с высшим приоритетом. В начале оперативной памяти расположена таблица векторов прерываний. В каждой ячейке этой таблицы расположена команда безусловного перехода на начальный адрес соответствующей устройству программы обслуживания прерывания (ПОП).

10.2. Процесс обработки прерываний, со стороны памяти

Рассмотрим пример вычислительного процесса при обработке прерываний от внешних устройств. В качестве внешних устройств будут выступать простые периферийные устройства: жесткий диск, принтер, клавиатура и мышь. В итоге, для рассмотрения процесса обработки прерывания названия внешних устройств не важно.

Пусть программы обработки прерываний находятся по адресам от 101 до 500. Для программы обработки диска выделим адреса 101-200 (нулевой приоритет), для программ обработки принтера 201-300 (1-й приоритет), для программы обработки мыши 301-400 (2-й приоритет) и для клавиатуры 401-500 (3-й приоритет). Как можно заметить для удобства под каждое периферийное устройство выделено по 100 ячеек памяти. Исполняемый код основной программы будет располагаться в области памяти 701-1500. Так как при прерывании будет необходимо запоминать адрес текущей команды, добавим стек. Для корректного отображения процесса обработки прерывания составим схему (рис. 78).

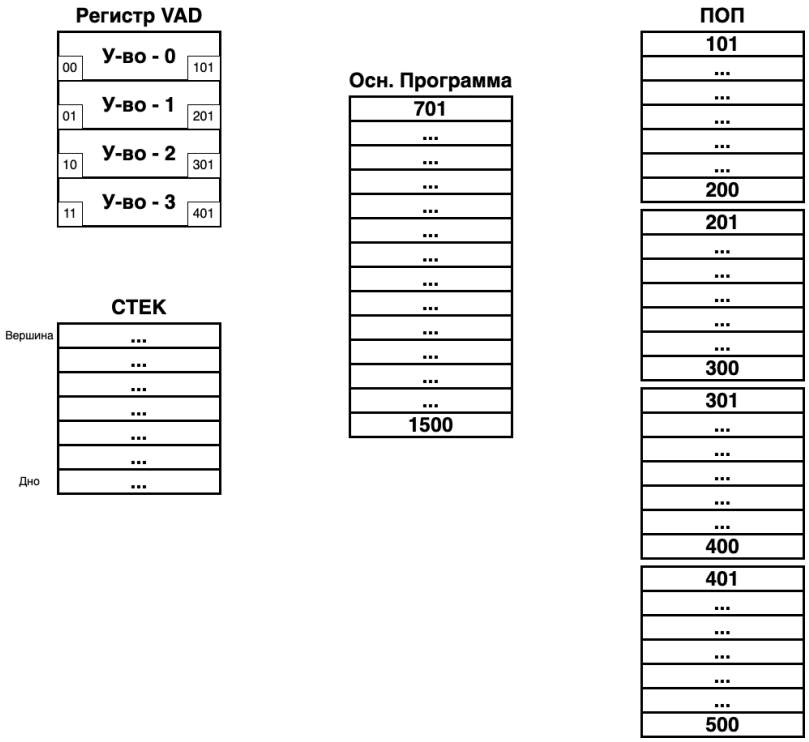


Рисунок 78. Схема устройства памяти для обработки прерываний

Рассмотрим пример: пусть при выполнении некоторой команды по адресу 809, основной программы, пришло прерывание от клавиатуры. Затем, при выполнении команды по адресу 442 ПОП клавиатуры, пришло прерывание от принтера. Изобразить последовательность обработки прерывания, используя заготовленную ранее схему (рис. 78).

Известно, что первое прерывание было получено от устройства «Клавиатура», при выполнении команды 809. Основная программа располагается в диапазоне 701-1500. Значит нулевой этап — это выполнение программы, до адреса 809. При поступлении сигнала о прерывании необходимо сделать два действия: закончить выполнение текущей команды и вычислить адрес следующей команды. В нашем случае адрес следующей команды — 810. Для того, чтобы не потерять этот адрес, занесем его в стек. Далее, необходимо узнать, по какому адресу находится программа-обработчик прерывания для устройства «Клавиатура», для этого необходимо обратиться в регистр VAD, на котором к этому моменту уже сформировался необходимый адрес, и считать его, в нашем случае, адрес будет только один, потому что только одно устройство запросило прерывание — это адрес 400.

Программа переходит в адрес 400 и начинает выполняться. По условию задачи, приходит еще одно прерывание, на этот раз от устройства «Принтер». Принтер в условии задачи имеет приоритет 1, а клавиатура приоритет 3, что означает, что прерывание от принтера должно быть обработано раньше. Повторяем процесс сохранения адреса. Завершаем выполнение команды 442, заносим в стек адрес 443. В регистре VAD сформировался адрес 200, который соответствует программе обработчику принтера.

По условию задачи, никакие устройства больше прерываний не запрашивали, полностью выполняем ПОП принтера, доходя до адреса 300. После чего обращаемся в стек, и берем верхний адрес, которым оказывается адрес 443, переходим по нему, и выполняем до конца ПОП клавиатуры. Когда выполнение ПОП клавиатуры дойдет до адреса 500, повторим чтение адреса из стека, и это позволит вернуться к выполнению основной программы, начиная с адреса 810. Ход процесса обработки прерывания от двух устройств с разными приоритетами продемонстрирован на рис. 79.

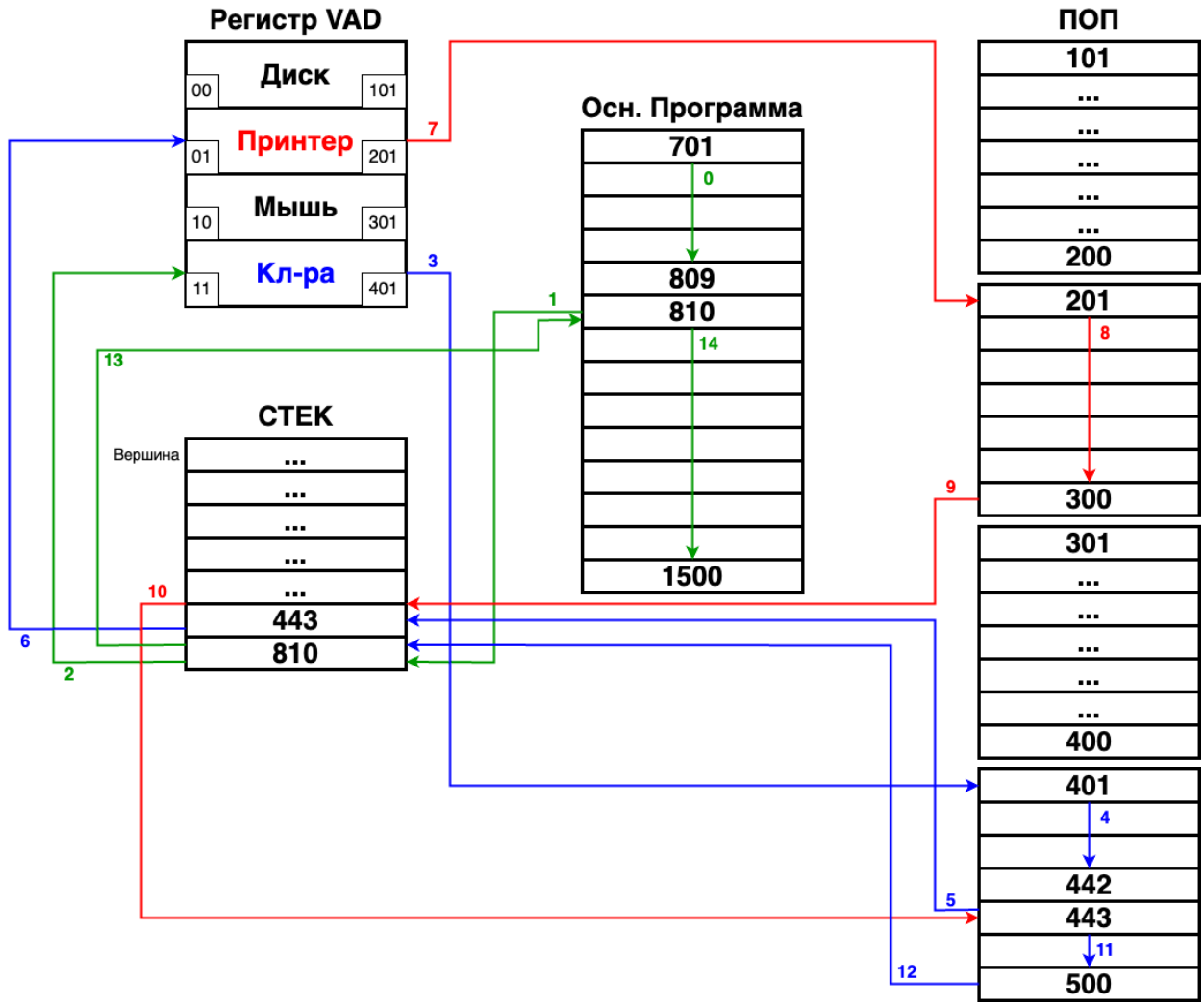


Рисунок 79. Обработка прерываний от двух устройств

10.3. Задания для самостоятельного выполнения

Варианты задач:

Вариант №1

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Клавиатура.

При выполнении команды 919 прикладной программы пришло прерывание от манипулятора. Далее, при выполнении команды 395 ПОП манипулятора пришло прерывание от диска.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №2

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Клавиатура.

При выполнении команды 725 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 436 ПОП клавиатуры пришло прерывание от манипулятора.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №3

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Клавиатура.

При выполнении команды 1223 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 478 ПОП клавиатуры пришло прерывание от принтера.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №4

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Тачпад.

При выполнении команды 1105 прикладной программы пришло прерывание от манипулятора. Далее, при выполнении команды 356 ПОП манипулятора пришло прерывание от диска.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №5

ПУ: 0. Диск 1. Плоттер 2. Манипулятор 3. Клавиатура.

При выполнении команды 1453 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 447 ПОП клавиатуры пришло прерывание от манипулятора.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №6

ПУ: 0. Диск 1. Принтер 2. Тачпад 3. Клавиатура.

При выполнении команды 1238 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 410 ПОП клавиатуры пришло прерывание от принтера.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №7

ПУ: 0. Диск 1. Плоттер 2. Манипулятор 3. Клавиатура.

При выполнении команды 1122 прикладной программы пришло прерывание от манипулятора. Далее, при выполнении команды 339 ПОП манипулятора пришло прерывание от диска.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №8

ПУ: 0. Диск 1. Плоттер 2. Манипулятор 3. Клавиатура.

При выполнении команды 1300 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 440 ПОП клавиатуры пришло прерывание от манипулятора.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №9

ПУ: 0. Диск 1. Принтер 2. Тачпад 3. Клавиатура.

При выполнении команды 962 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 497 ПОП клавиатуры пришло прерывание от принтера.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №10

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Тачпад.

При выполнении команды 784 прикладной программы пришло прерывание от манипулятора. Далее, при выполнении команды 315 ПОП манипулятора пришло прерывание от диска.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №11

ПУ: 0. Диск 1. Плоттер 2. Манипулятор 3. Клавиатура.

При выполнении команды 956 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 487 ПОП клавиатуры пришло прерывание от манипулятора.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №12

ПУ: 0. Диск 1. Принтер 2. Плоттер 3. Клавиатура.

При выполнении команды 1125 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 472 ПОП клавиатуры пришло прерывание от принтера.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №13

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Тачпад.

При выполнении команды 957 прикладной программы пришло прерывание от манипулятора. Далее, при выполнении команды 381 ПОП манипулятора пришло прерывание от диска.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №14

ПУ: 0. Диск 1. Плоттер 2. Манипулятор 3. Клавиатура.

При выполнении команды 1325 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 460 ПОП клавиатуры пришло прерывание от манипулятора.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №15

ПУ: 0. Диск 1. Принтер 2. Плоттер 3. Клавиатура.

При выполнении команды 1147 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 499 ПОП клавиатуры пришло прерывание от принтера.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №16

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Клавиатура.

При выполнении команды 999 прикладной программы пришло прерывание от манипулятора. Далее, при выполнении команды 343 ПОП манипулятора пришло прерывание от диска.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №17

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Клавиатура.

При выполнении команды 966 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 406 ПОП клавиатуры пришло прерывание от манипулятора.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №18

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Клавиатура.

При выполнении команды 1313 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 489 ПОП клавиатуры пришло прерывание от принтера.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №19

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Клавиатура.

При выполнении команды 862 прикладной программы пришло прерывание от манипулятора. Далее, при выполнении команды 392 ПОП манипулятора пришло прерывание от диска.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №20

ПУ: 0. Диск 1. Принтер 2. Манипулятор 3. Клавиатура.

При выполнении команды 1488 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 415 ПОП клавиатуры пришло прерывание от манипулятора.

Изобразить на рисунке ход выполнения вычислительного процесса.

Вариант №21

ПУ: 0. Диск 1. Принтер 2. Плоттер 3. Клавиатура.

При выполнении команды 1292 прикладной программы пришло прерывание от клавиатуры. Далее, при выполнении команды 422 ПОП клавиатуры пришло прерывание от принтера.

Изобразить на рисунке ход выполнения вычислительного процесса.

ПРАКТИЧЕСКАЯ РАБОТА №11

11.1. Оперативное запоминающее устройство

Существует два основных типа RAM (*random-access memory* — память с произвольным доступом): *статическая* и *динамическая*, или сокращённо SRAM (static random access memory) и DRAM (dynamic random-access memory).

SRAM для той же задачи применяет транзисторы, которые работают с той же скоростью, что и логические блоки процессора (примерно в 10 раз быстрее, чем DRAM). В статической оперативной памяти (SRAM), данные хранятся не в виде заряда на конденсаторе, а в паре перекрестно связанных инверторов (это логический вентиль, который реализует логическое отрицание), поэтому SRAM не нужно обновлять. Однако внутренняя конструкция каждой ячейки SRAM требует шести транзисторов, а для ячейки DRAM один транзистор.

DRAM использует микроскопические конденсаторы для хранения данных в виде электрического заряда, для сохранения данных необходимо постоянное обновление памяти.

Обновление памяти, процесс периодического чтения информации из области памяти компьютера и немедленной перезаписи прочитанной информации в ту же область без изменений с целью сохранения информации. DRAM могут изготавливаться в корпусе чипа (это называется встроенной (embedded) DRAM), однако процессоры довольно малы, поэтому в них не удастся поместить много памяти.

Технологии памяти DRAM.

В чипе DRAM каждый бит данных памяти хранится как наличие или отсутствие электрического заряда на небольшом конденсаторе на чипе. С течением времени заряды в ячейках памяти утекают, поэтому без обновления сохраненные данные в конечном итоге будут потеряны. Чтобы предотвратить это, внешняя схема периодически считывает каждую ячейку и перезаписывает ее, восстанавливая заряд на конденсаторе до исходного уровня. Каждый цикл обновления памяти обновляет последующую область ячеек памяти, таким образом, повторно обновляя все ячейки в последовательном цикле. Этот процесс выполняется автоматически в фоновом режиме схемой памяти и прозрачен для пользователя. Во время цикла обновления, память недоступна для обычных операций чтения и записи, но в современной памяти это "накладное" время недостаточно велико, чтобы значительно замедлить работу памяти.

Поля DRAM в основном состоят из двух электронных компонентов:

переключателя, представленного MOSFET (*МОП-транзистором*)

накопителя, представляющего собой канавочный конденсатор.

Вместе они образуют так называемую *ячейку памяти*, каждая из которых содержит 1 бит данных. Очень приблизительная схема ячейки показана ниже (рис. 80):

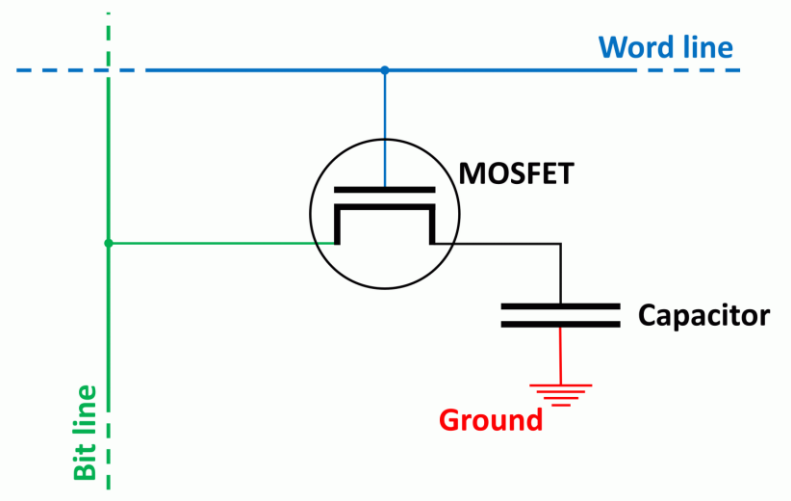


Рисунок 80. Ячейка памяти

Синими (word line — числовая шина) и зелёными (bit line — разрядная линия) линиями обозначены соединения, подающие напряжение на МОП-транзистор (MOSFET) и конденсатор (capacitor). Они используются для считывания и записи данных в ячейку, и первой всегда срабатывает вертикальная (разрядная (bit line)) линия.

Канавочный конденсатор, по сути, используется в качестве сосуда для заполнения электрическим зарядом — его пустое/заполненное состояние даёт нам 1 бит данных: 0 — пустой, 1 — полный (конденсаторы не способны хранить этот заряд вечно и со временем он утекает).

Это означает, что каждую ячейку памяти нужно постоянно обновлять *по 15-30 раз в секунду*, хотя сам этот процесс довольно быстр: для обновления набора ячеек требуется всего несколько наносекунд. К сожалению, в чипе DRAM *множество* ячеек, и во время их обновления считывание и запись в них невозможна. К каждой линии подключено несколько ячеек (рис. 81).

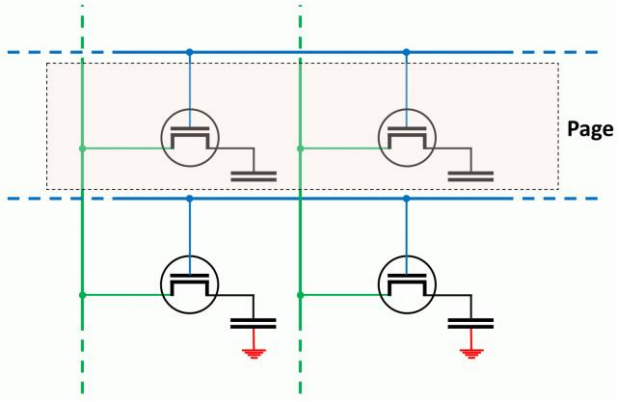


Рисунок 81. Соединенные ячейки памяти

Эта схема неидеальна, потому что для каждого столбца ячеек используется две разрядные линии — если бы изобразили всё, то схема бы стала слишком неразборчивой.

Полная *строка* ячеек памяти называется *страницей* (page), а длина её зависит от типа и конфигурации DRAM. Чем длиннее страница, тем больше в ней бит, но и тем большая электрическая мощность нужна для её работы; короткие страницы потребляют меньше мощности, но и содержат меньший объём данных.

Однако нужно учитывать и ещё один важный фактор. При считывании и записи на чип DRAM первым этапом процесса является активация всей страницы. Строка битов (состоящая из нулей и единиц) хранится в *буфере строки*, который по сути является набором усилителей считывания и защёлок, а не дополнительной памятью. Затем активируется соответствующий столбец для извлечения данных из этого буфера.

Если страница слишком мала, то чтобы успеть за запросами данных, строки нужно активировать чаще; и наоборот — большая страница предоставляет больше данных, поэтому активировать её можно реже. И даже несмотря на то, что длинная строка требует большей мощности и потенциально может быть менее стабильной, лучше стремиться к получению максимально длинных страниц.

Если собрать вместе набор страниц, то мы получим один *банк* (Bank) памяти DRAM (рис. 82). Как и в случае страниц, размер и расположение строк и столбцов ячеек играют важную роль в количестве хранимых данных, скорости работы памяти, энергопотреблении и так далее.

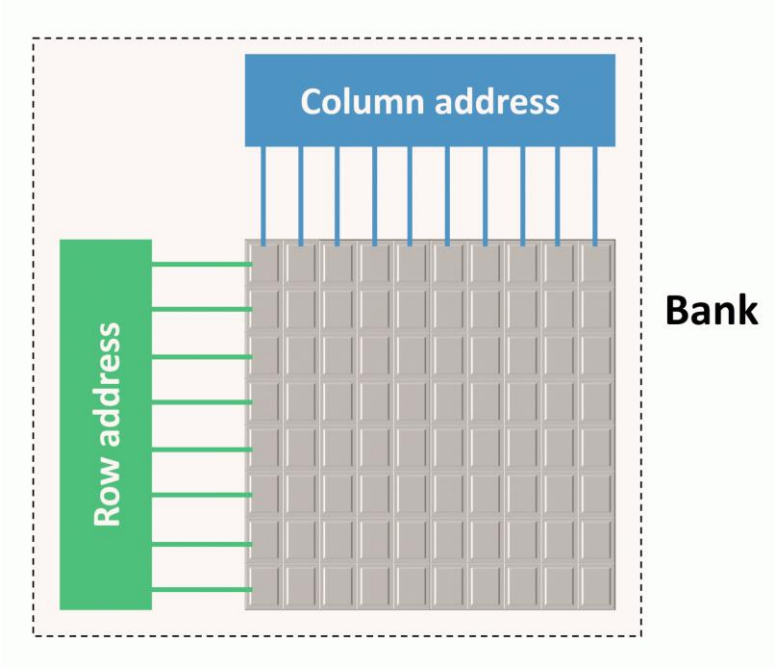


Рисунок 82. Банк памяти DRAM

Например, схема может состоять из 4 096 строк и 4 096 столбцов, при этом полный объём одного банка будет равен 16 777 216 битам или 2 мегабайтам. Но не у всех чипов DRAM банки имеют квадратную структуру, потому что длинные страницы лучше, чем короткие. Например, схема из 16 384 строк и 1 024 столбцов даст нам те же 2 мегабайта памяти, но каждая страница будет содержать в четыре раза больше памяти, чем в квадратной схеме.

Все страницы в банке соединены с системой адресации строк (то же относится и к столбцам) и они контролируются сигналами управления и адресами для каждой строки/столбца. Чем больше строк и столбцов в банке, тем больше битов должно использоваться в адресе.

Для банка размером 4 096 x 4 096 для каждой системы адресации требуется 12 бит, а для банка 16 384 x 1 024 потребуется 14 бит на адреса строк и 10 бит на адреса столбцов. Стоит заметить, что обе системы имеют суммарный размер 24 бита. Если бы чип DRAM мог предоставлять доступ к одной странице за раз, то это было бы не особо удобно, поэтому в них упаковано несколько банков ячеек памяти. В зависимости от общего размера, чип может иметь 4, 8 или даже 16 банков — чаще всего используется 8 банков. Все эти банки имеют общие шины команд, адресов и данных, что упрощает структуру системы памяти (рис. 83). Пока один банк занят работой с одной командой, другие банки могут продолжать выполнение своих операций. Весь чип, содержащий все банки и шины, упакован в защитную оболочку и припаян к плате. Она содержит электропроводники, подающие питание для работы DRAM и сигналов команд, адресов и данных.

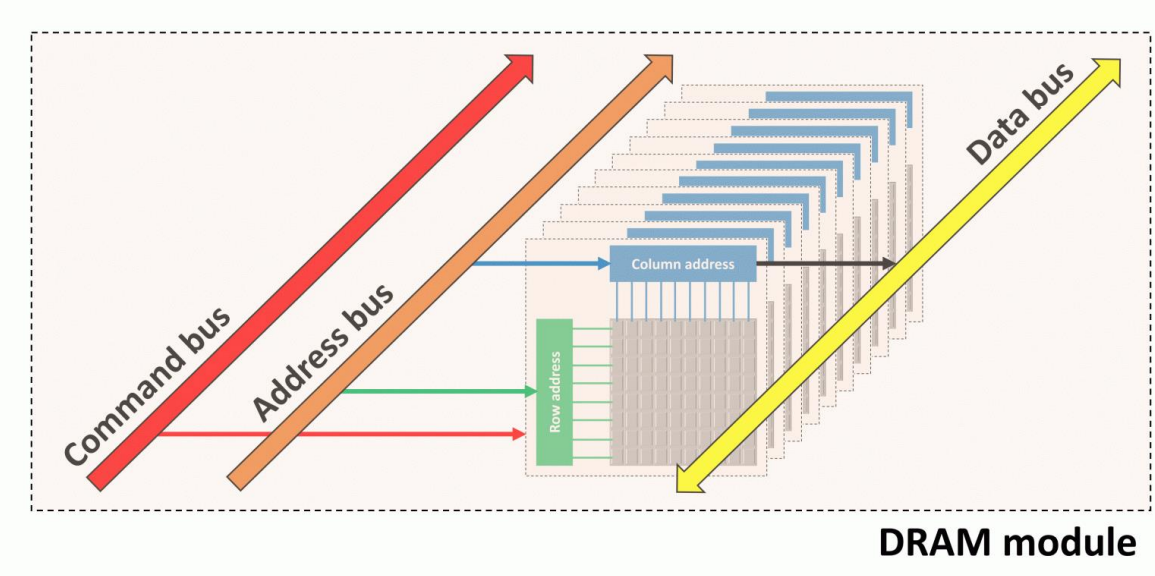


Рисунок 83. Модуль DRAM

11.2. Пример решения задачи

В качестве примера рассмотрим следующую задачу: требуется построить ОЗУ 16x2 имея микросхему 4x1.

Решение:

1. Для выполнения задания нужно сначала построить по условно-графическому обозначению (УГО) обе схемы. Они представлены на рис. 84 и 85.
2. При построении схемы 16x2 из 4x1 требуется сначала увеличить разряды, следовательно, будет построена на базе элемента 4x1 схема 4x2. Результат представлен на рис. 86.
3. На базе созданного элемента 4x2 и используя дешифратор, построить схему 16x2. A0, A1, D0 и D1 работают синхронно. Вход CS управляется с помощью дешифратора. Так, например, если будет требоваться 10 ячейка, то значения разрядов от A3 до A0 будут соответственно 1010. Потребуется RAM, к входу CS которого подсоединен провод от второго выхода дешифратора. Результат представлен на рис. 87.
4. В случае если требуется отобразить схему с общим коллектором (ОК), то необходимо соединить все выходы из всех RAM, в один провод и поставить знак ОК. Если же требуется отобразить схему с 3 состояниями, то требуется соединить все выходы в один провод и поставить знак стрелки. Примеры обоих случаев приведены в рис. 88.

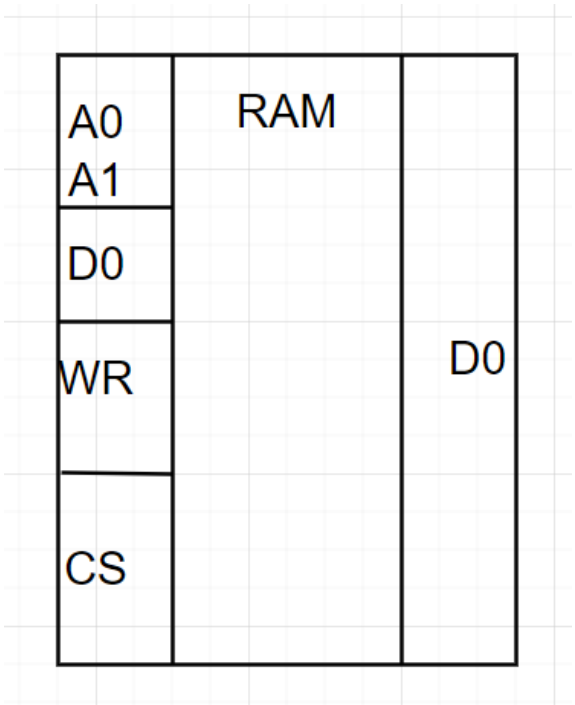


Рисунок 84. Микросхема ОЗУ 4x1

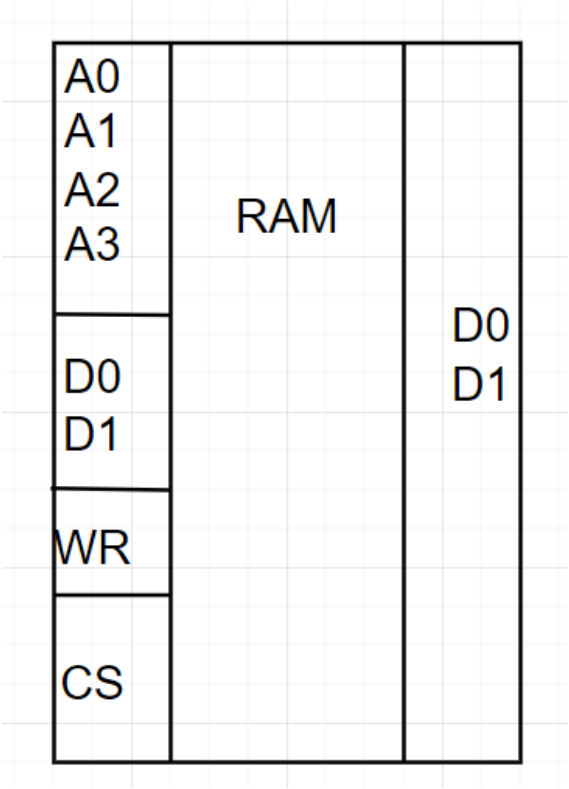


Рисунок 85. Микросхема ОЗУ 16x2

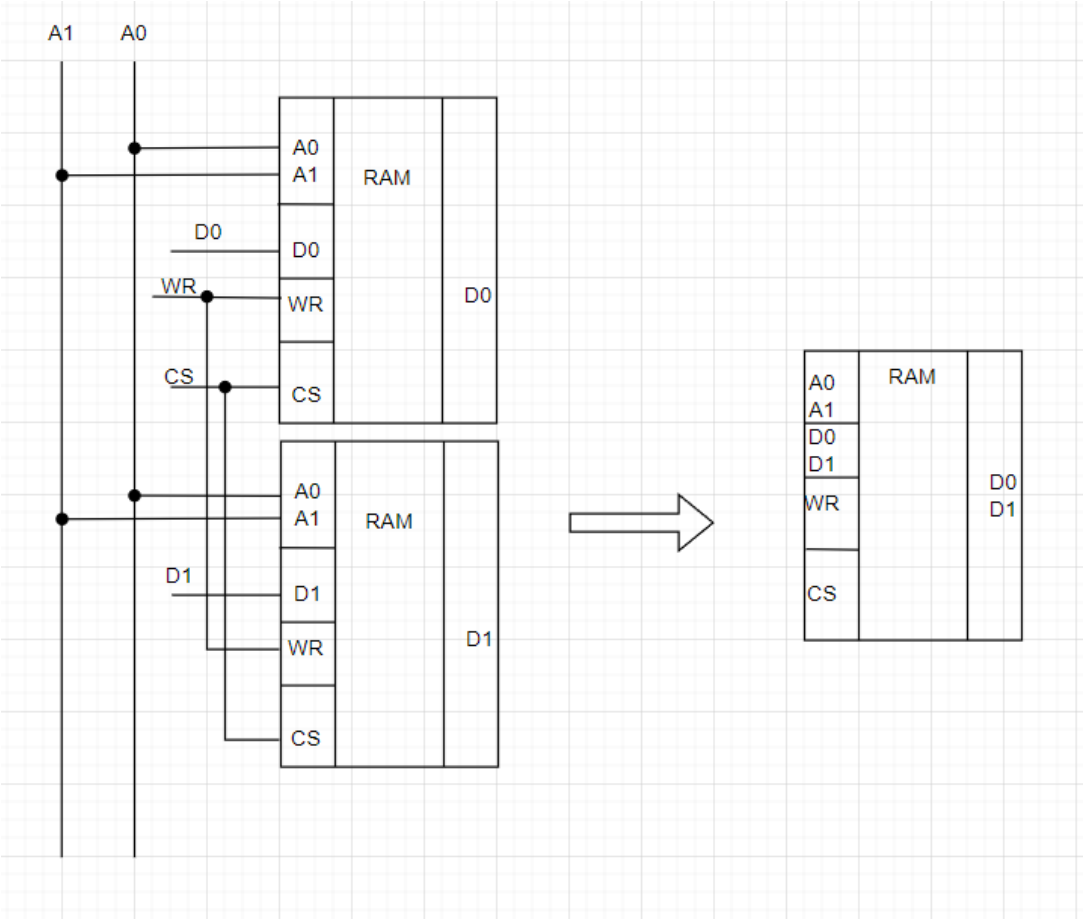


Рисунок 86. Получение ОЗУ 4x2 на базе микросхемы 4x1

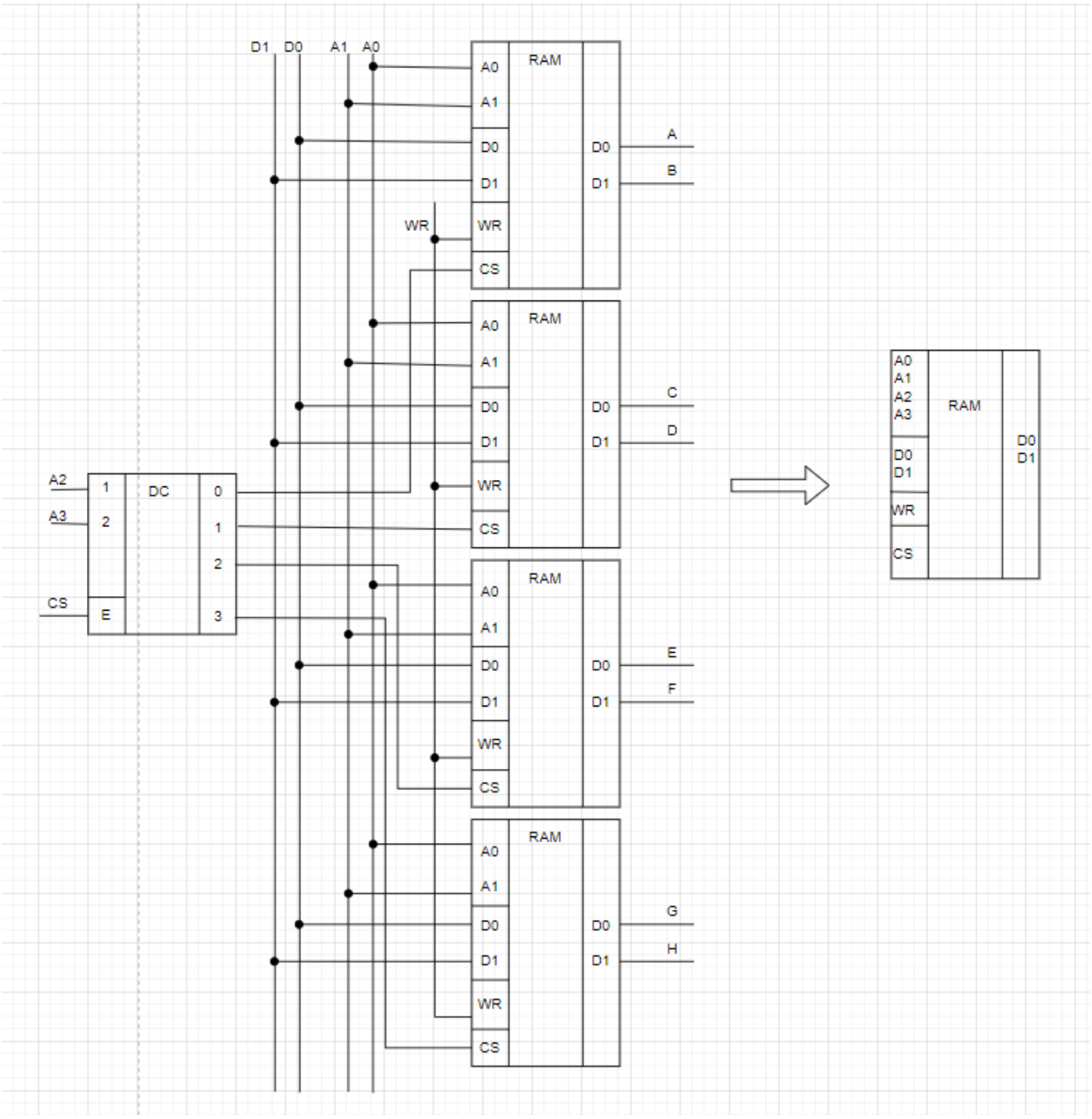


Рисунок 87. Получение ОЗУ 16x2 на базе микросхемы 4x2

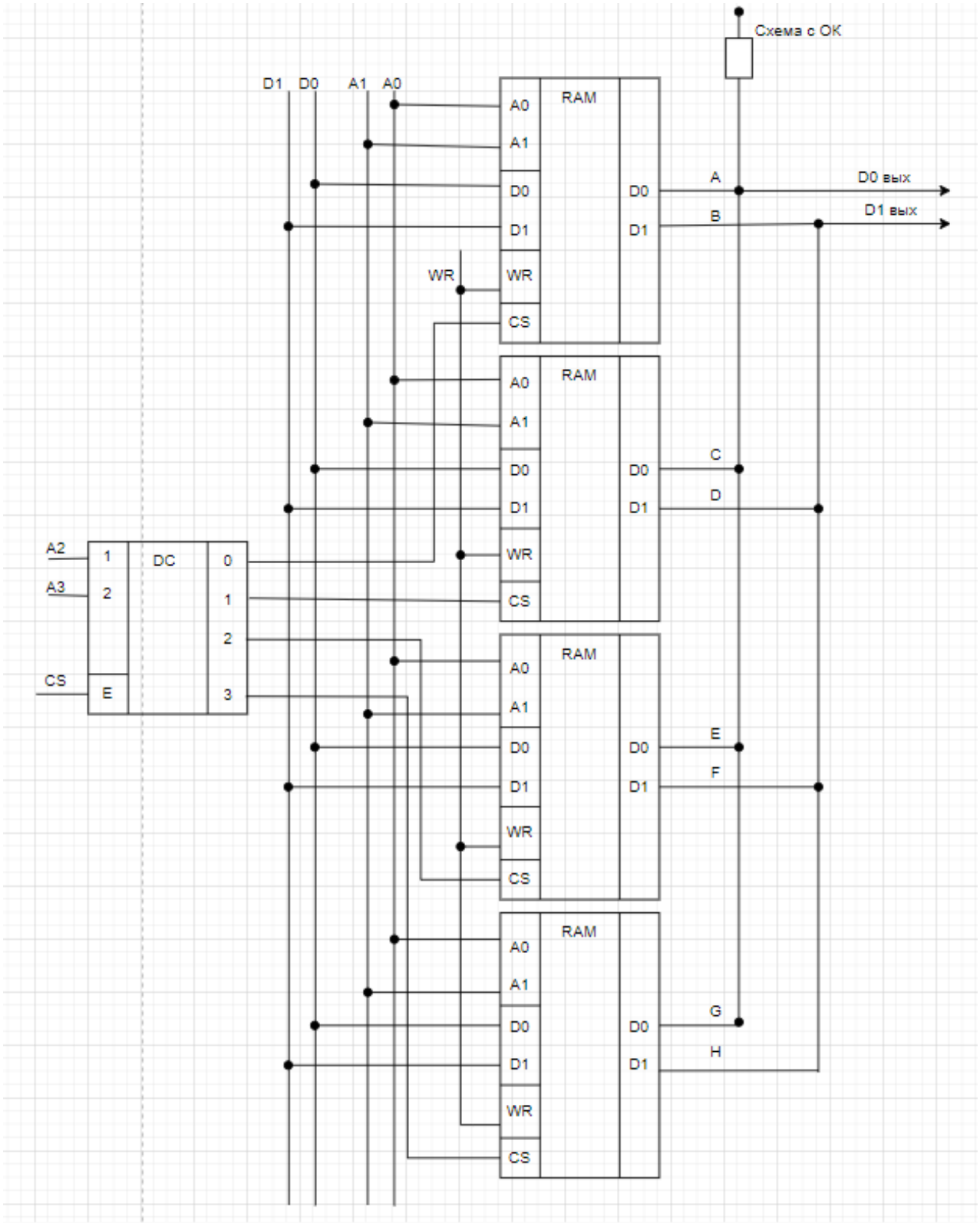


Рисунок 88. Получение ОЗУ 16x2 на базе микросхемы 4x2 D0 с общим коллектором D1 с 3 состояниями

11.3. Задачи для самостоятельного выполнения

Задания для самостоятельного выполнения представлены в табл. 39.

Таблица 39. Список вариантов для самостоятельного решения

№ варианта	Задание
1	Построить ОЗУ 16х4 на базе ИС 4х1 с 3 состояниями
2	Построить ОЗУ 32х4 на базе ИС 8х2 с 3 состояниями
3	Построить ОЗУ 64х2 на базе ИС 32х1 с 3 состояниями
4	Построить ОЗУ 16х2 на базе ИС 8х1 с 3 состояниями
5	Построить ОЗУ 32х4 на базе ИС 8х1 с 3 состояниями
6	Построить ОЗУ 64х2 на базе ИС 16х1 с 3 состояниями
7	Построить ОЗУ 64х4 на базе ИС 16х2 с 3 состояниями
8	Построить ОЗУ 32х2 на базе ИС 16х1 с 3 состояниями.
9	Построить ОЗУ 16х4 на базе ИС 8х1 с 3 состояниями
10	Построить ОЗУ 64х4 на базе ИС 32х2 с 3 состояниями
11	Построить ОЗУ 32х4 на базе ИС 16х1 с 3 состояниями
12	Построить ОЗУ 16х4 на базе ИС 4х2 с 3 состояниями
13	Построить ОЗУ 32х2 на базе ИС 8х1 с 3 состояниями
14	Построить ОЗУ 64х4 на базе ИС 16х1 с 3 состояниями
15	Построить ОЗУ 32х4 на базе ИС 16х2 с 3 состояниями
16	Построить ОЗУ 64х4 на базе ИС 32х1 с 3 состояниями
17	Построить ОЗУ16х2 на базе ИС 4х1 с 3 состояниями
18	Построить ОЗУ 16х4 на базе ИС 8х2 с 3 состояниями
19	Построить ОЗУ 16х4 на базе ИС 8х2 с ОК
20	Построить ОЗУ 16х4 на базе ИС 4х2 с ОК
21	Построить ОЗУ 16х2 на базе ИС 4х1 с ОК
22	Построить ОЗУ 16х4 на базе ИС 4х1 с ОК
23	Построить ОЗУ 32х4 на базе ИС 8х2 с ОК
24	Построить ОЗУ 64х2 на базе ИС 32х1 с ОК
25	Построить ОЗУ 16х2 на базе ИС 8х1 с ОК
26	Построить ОЗУ 32х4 на базе ИС 8х1 с ОК
27	Построить ОЗУ 64х2 на базе ИС 16х1 с ОК
28	Построить ОЗУ 64х4 на базе ИС 16х2 с ОК
29	Построить ОЗУ 32х2 на базе ИС 16х1 с ОК
30	Построить ОЗУ 16х4 на базе ИС 8х1 с ОК
31	Построить ОЗУ 64х4 на базе ИС 32х2 с ОК
32	Построить ОЗУ 32х4 на базе ИС 16х1 с ОК
33	Построить ОЗУ 16х4 на базе ИС 4х2 с ОК
34	Построить ОЗУ 32х2 на базе ИС 8х1 с ОК

Окончание табл. 39

№ варианта	Задание
35	Построить ОЗУ 64х4 на базе ИС 16х1 с ОК
36	Построить ОЗУ 32х4 на базе ИС 16х2 с ОК
37	Построить ОЗУ 64х4 на базе ИС 32х1 с ОК
38	Построить ОЗУ 16х2 на базе ИС 4х1 с ОК
39	Построить ОЗУ 16х4 на базе ИС 8х2 с ОК
40	Построить ОЗУ 16х2 на базе ИС 8х1 с ОК

ЗАКЛЮЧЕНИЕ

В настоящем учебно-методическом пособии приведены материалы одиннадцати практических занятий по дисциплине «Архитектура вычислительных машин и систем». Для каждого практического занятия имеются краткие теоретические сведения, дополняющие материал лекционных занятий, разбирается пример выполнения практических заданий, приводятся варианты заданий для самостоятельного решения студентами.

СПИСОК ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Схемотехника устройств компьютерных систем. Ч. 1 [Электронный ресурс]: лабораторный практикум / Н. А. Дуксин, Д. В. Люлява, И. Е. Тарасов. — М.: РТУ МИРЭА, 2023. — Электрон. опт. диск (ISO)

2. Инструментальные средства разработки программно-аппаратных комплексов [Электронный ресурс]: учебное пособие / И. Е. Тарасов. — М.: РТУ МИРЭА, 2021. — Электрон. опт. диск (ISO)

3. Разработка программно-аппаратного обеспечения информационных и автоматизированных систем [Электронный ресурс]: учебное пособие / Д. С. Потехин, И. Е. Тарасов. — М.: РТУ МИРЭА, 2022. — Электрон. опт. диск (ISO)

4. Архитектура вычислительных машин и систем [Электронный ресурс]: учебное пособие / А. Г. Мусихин, Н. А. Смирнов. — М.: РТУ МИРЭА, 2021. — Электрон. опт. диск (ISO).

5. Методы проектирования цифровых устройств в составе инфокоммуникационных систем [Электронный ресурс]: учебное пособие / Д. С. Потехин. — М.: РТУ МИРЭА, 2022. — Электрон. опт. диск (ISO)

6. Схемотехника устройств компьютерных систем. Ч. 1 [Электронный ресурс]: лабораторный практикум / Н. А. Дуксин, Д. В. Люлява, И. Е. Тарасов. — М.: РТУ МИРЭА, 2023. — Электрон. опт. диск (ISO)

7. Коников, А.И. Электронные вычислительные машины: учеб. -метод. пособие / О.М. Баранова; Нац. исследоват. Моск. гос. строит. ун-т; А.И. Коников. — Электрон. изд. — Москва: Изд-во МИСИ-МГСУ, 2021. — 39 с.: ил. — Электрон. дан. и прогр. (5,2 Мб). — ISBN 978-5-7264-2859-8. — ISBN 978-5-7264-2860-4. — URL: https://lib.rucont.ru/efd/786123 (дата обращения: 29.03.2024)

8. Овчеренко, В.А. Периферийные устройства информационных систем. Физические принципы организации и интерфейсы ввода-вывода: учеб. пособие / В.Г. Токараев; В.А. Овчеренко. — Новосибирск: Изд-во НГТУ, 2018 — 75 с.: ил. — ISBN 978-5-7782-3625-7. — URL: https://lib.rucont.ru/efd/774730 (дата обращения: 29.03.2024)

9. Фомин, Д.В. Основы компьютерной электроники: учебное пособие / Д.В. Фомин. — 2-е изд. — Москва: Директ-Медиа, 2019. — 109 с. — ISBN 978-5-4499-0152-1. — URL: https://lib.rucont.ru/efd/798852 (дата обращения: 29.03.2024)

10. Микропроцессорная техника: учеб. пособие / В.В. Иванов. — Самара: Изд-во Самарского университета, 2019. — 80 с.: ил.

11. Лиманова Н.И. Архитектура вычислительных систем и компьютерных сетей: учеб. пособие / Поволж. гос. ун-т телекоммуникаций и информатики;

Н.И. Лиманова. — Самара: Изд-во ПГУТИ, 2017. — 197 с.: ил. — URL: https://lib.rucont.ru/efd/641678 (дата обращения: 29.03.2024)

12. Исследование работы конвейера команд [Электронный ресурс]: метод. указания по выполнению лаб. работ / Е. В. Бражникова, Ф. А. Игнатьев, Д. А. Карпов. — М.: РТУ МИРЭА, 2018 .

Сведения об авторах

Мусихин Александр Григорьевич, кандидат технических наук, доцент кафедры вычислительной техники института информационных технологий Российского технологического университета — МИРЭА.

Морозов Василий Алексеевич, ассистент кафедры вычислительной техники института информационных технологий Российского технологического университета — МИРЭА.

Гололобов Александр Алексеевич, ассистент кафедры вычислительной техники института информационных технологий Российского технологического университета — МИРЭА.

Пономарев Артем Николаевич, старший преподаватель кафедры вычислительной техники института информационных технологий Российского технологического университета — МИРЭА.